



The PYroMat User and Developer Handbook

Third Edition, v2.2.5 and later

Christopher R. Martin
Associate Professor of Mechanical Engineering
Penn State University

September 18, 2026

Contents

Foreword	5
Nomenclature	5
1 Introduction	8
1.1 The Properties	9
1.1.1 Property Groups	9
1.1.2 Density, ρ	14
1.1.3 Specific volume, v	16
1.1.4 Temperature, T	17
1.1.5 Pressure, p	17
1.1.6 Internal Energy, e	18
1.1.7 Enthalpy, h	19
1.1.8 Entropy, s	20
1.1.9 Free (Helmholtz) energy, f	21
1.1.10 Gibbs energy, g	22
1.1.11 Specific Heats, c_p c_v	23
1.1.12 Speed of sound, a	25
2 Getting started	26
2.1 Installation	26
2.1.1 Prerequisites	26
2.1.2 Installation with a package manager	27
2.1.3 Installing in popular IDEs	29
2.1.4 Manual installation with <code>git</code>	30
2.1.5 Manual installation from Sourceforge	30

2.2	Using PYroMat	31
2.2.1	Importing	31
2.2.2	Retrieving substance data	31
2.2.3	Searching for substances with <code>search()</code>	32
2.2.4	Finding substances with <code>info()</code>	34
2.2.5	In-line documentation	35
2.3	Property interface	35
2.3.1	Property method arguments	35
2.3.2	Default values	37
2.3.3	Inverse methods	38
2.3.4	Tips and tricks	38
3	Configuration	41
3.1	The <code>pm.config</code> instance	41
3.1.1	Making temporary changes to <code>config</code>	44
3.2	Configuration files	45
4	Units	47
4.1	Unit definitions	48
4.2	Setup	49
4.3	Constants	50
4.4	<code>Conversion Class</code>	52
4.5	Fundamental Units	53
4.5.1	Time	53
4.5.2	Length	54
4.5.3	Mass and Weight	55
4.5.4	Molar	59
4.5.5	Matter	62
4.5.6	Temperature	63
4.6	Derived Units	65
4.6.1	Force	66
4.6.2	Energy	67
4.6.3	Pressure	69
4.6.4	Volume	71

5	The PYroMat modules	74
5.1	The class registry module, <code>reg</code>	74
5.2	The data module, <code>dat</code>	76
5.2.1	The <code>load()</code> function	76
5.2.2	Data files	77
5.2.3	Tools for working with data files	79
5.3	The utility module, <code>utility</code>	79
5.3.1	PYroMat error types	79
5.3.2	Redundancy tools	81
5.3.3	Other tools	81
6	Ideal Gases	82
6.1	Properties of ideal gases	83
6.1.1	Ideal gas law	83
6.1.2	Internal energy	86
6.1.3	Enthalpy	88
6.1.4	Specific heats	88
6.1.5	Entropy and enthalpy revisited	89
6.1.6	Formation properties	92
6.1.7	Speed of sound	95
6.1.8	Other properties	96
6.1.9	Properties of mixtures	96
6.2	The ideal gas collection	100
6.2.1	The Shomate equation: <code>ig</code>	101
6.2.2	The NASA polynomial: <code>ig2</code>	102
6.2.3	The ideal gas mixture: <code>igmix</code>	105
7	Multi-phase substance models	108
7.1	Span and Wagner Equation of State	109
7.1.1	Nondimensionalization	110
7.1.2	Ideal gas portion of free energy	110
7.1.3	Residual portion of free energy	114
7.1.4	Derivatives of free energy	115
7.2	Calculation of properties	119
7.2.1	Pressure	120
7.2.2	Entropy	120
7.2.3	Internal energy	121

7.2.4	Enthalpy	121
7.2.5	Gibbs free energy	121
7.2.6	Specific heats	121
7.2.7	Speed of sound	123
7.2.8	Liquid-vapor line	124
7.3	The multi-phase collection	125
7.3.1	The model coefficients	125
7.3.2	The <code>mp1</code> class	126
7.3.3	The <code>mp2</code> class	129
7.3.4	Benchmarking	137
7.4	Sources implemented	139
8	Numerical routines	140
8.1	Polynomials of two variables	141
8.1.1	Modifying polynomials for non-integer and negative powers	141
8.1.2	Representation of polynomials in data	144
8.1.3	Efficient evaluation of the polynomial	146
8.1.4	Efficient evaluation of derivatives	147
8.1.5	Implementation of the algorithm	148
8.2	Polynomials in one dimension	151
8.3	Iteration with <code>iter1</code>	152
8.4	Iteration with <code>hybrid1</code>	152
8.4.1	Bisection iteration	155
8.4.2	The <code>hybrid1</code> candidates	155
8.4.3	Standard candidate selection	157
8.4.4	Paranoid candidate selection	159
	Bibliography	164

Foreword

My name is Chris Martin, and I am the founding author of PYroMat, and I am proud to collaborate with some wonderful people to continuously expand its capabilities. If you're here, you have enough interest in this project to dive into its documentation, so I really appreciate you. This is the first edition of the version 3 documentation. If you find mistakes or errors that need to be corrected in the second edition, please don't hesitate to reach out to me¹.

If you've derived a latte's worth of enjoyment or extracted a gallon of gasoline (or a liter of petrol) worth of mileage out of the project, consider supporting us to help keep it going. Just go to <http://pyromat.org/support> to see ways to help.

In the meantime, happy calculating!

Nomenclature

These variables and units are restricted to derivations in this document. Because PYroMat has a user-configurable unit system, any of these may be expressed in alternate units.

The following are parameters with units.

¹Christopher R. Martin Ph.D.

Associate Professor of Mechanical Engineering
Penn State University; Altoona College
crm28@psu.edu

Symb.	Units	Description
a	m/s	Speed of sound
c	J/kg/K	Specific heat
e	J/kg	Internal energy
f	J/kg	Free energy
g	J/kg	Gibbs energy
h	J/kg	Enthalpy
k	J/K	Boltzmann constant
M	kg	Mass
N	count	Number of moles
n	m ⁻³	Number density
p	Pa	Pressure
q	J kg ⁻¹	Normalized heat addition
R	J kg ⁻¹ K ⁻¹	Gas constant
R_u	J kmol ⁻¹ K ⁻¹	Universal gas const.
s	J/kg/K	Entropy
T	K	Temperature
v	m ³ /kg	Specific volume
W	kg/kmol	Molar mass
ρ	kg/m ³	Density

The following are dimensionless parameters.

Symb.	Def.	Description
α	f/RT	ND Free energy
δ	ρ/ρ_c	ND Density
τ	T_c/T	ND Temperature
γ	c_p/c_v	Sp. heat ratio
y_i	m_i/m	Mass fraction
χ_i	N_i/N	Mole fraction
π	p/p_c	ND Pressure

The following are subscripts used throughout the document.

Symb.	Description
x_c	Critical point
x_f	Formation property (e.g. enthalpy of formation)
x_p	Constant-pressure (sp.heat)
x_t	Triple point
x_v	Constant-volume (sp.heat)
x°	Property at reference pressure
$\bar{x}$	Property in molar or number units

Special constants used in more fundamental calculations are defined throughout this document. Many are listed explicitly in Table 4.1 in Chapter 4.

Chapter 1

Introduction

PYroMat is a Python-based package for calculating the thermodynamic properties of fluids. That includes liquids, gases, and plasmas. It is written in pure Python, and the core algorithms only depend on the Numpy package for back-end numerical and array support. This document is intended to serve as a reference manual for interpreting, using, or even writing your own PYroMat data sets.

Probably, no reader will need to digest this handbook in its entirety. The goal is to include all the important information in a single portable document. Chapter 2 on Getting Started is probably the one that most users will need, since it describes installation and basic use of the software.

For users who want a review of the thermodynamic properties, this introduction goes into detail on the fundamental definitions for the properties used. First, we talk briefly about the ideas of *primary* and *basic properties*, which PYroMat uses to organize its work on the back-end. Then, we will briefly define each property with a modest attempt to describe its origins and relevance.

For the curious or for developers who may want to add their own substance models, the inner workings of the package are described in some detail in Chapters 5, 6, 7, and 8.

1.1 The Properties

Here, we begin with a brief review of the thermodynamic properties of liquids and gases that PYroMat calculates. For a detailed development of these properties, the reader should consult an introductory text on thermodynamics¹.

In the author’s opinion, contemporary undergraduate texts on thermodynamics do the reader a disservice by ignoring the deeply intuitive (and terribly important) underpinnings provided by the kinetic theory of gases. Kinetic theory and thermodynamics have a complicated relationship because they were developed in parallel (often in tragic contention with one another), but the limits imposed by many of the common assumptions (such as ideal or perfect gas) cannot be deeply understood without kinetic theory. To the interested reader, the author would recommend Jeans’s 1949 introductory text². It is old, but it is brief, accessible, inexpensive, and approaches the subject only requiring that the reader have a grasp of calculus, geometry, and introductory mechanics.

PYroMat is primarily concerned with the *thermodynamic* properties listed in Table 1.1. There is no single set of units used to express these properties since PYroMat uses a user-configurable unit system. The units systems given in Table 1.1 are itemized in Table 1.2. All of Chapter 4 is devoted to describing PYroMat’s treatment of units.

1.1.1 Property Groups

A detailed equation of state model can make equal use of almost any pair of properties to define the others, and since version 2.2.0, PYroMat is quite flexible about which two are used. While this kind of convenience is part of the purpose of a high-level software tool, users may notice that some combinations are faster than others, some produce more numerical inaccuracies than others, and some are simply not possible.

¹For example, Cengel and Boles, *Thermodynamics*, McGraw Hill. Any edition will do.

²James Jeans, *Introduction to the Kinetic Theory of Gases*, Cambridge University Press, 1949.

Table 1.1: Thermodynamic properties, their symbols, and their unit systems

Symbol	In-Code	Units	Description
T	T	T	Temperature
p	p	P	Pressure
ρ	d	M / V	Density
x	x	-	Quality
R	R	E / M / T	Ideal gas constant
W	mw	Ma / Mo	Molecular weight
a	a	L / t	Speed of sound
e	e	E / M	Internal energy
f	e	E / M	Free (Helmholtz) energy
g	e	E / M	Gibbs energy
h	h	E / M	Enthalpy
s	s	E / M / T	Entropy
c_p	cp	E / M / T	Constant-pressure specific heat
c_v	cv	E / M / T	Constant-volume specific heat
γ	gam	-	Specific heat ratio

Table 1.2: Classes of units, their entry in `pm.config`, and their defaults.

Unit	config entry	Default	Description
E	unit_energy	kJ	Energy
L	unit_length	m	Length
M	unit_matter	kg	Matter (molar or mass)
Ma	unit_mass	kg	Mass
Mo	unit_molar	kmol	Molar
P	unit_pressure	bar	Pressure
t	unit_time	s	Time
T	unit_temperature	K	Temperature
V	unit_volume	m ³	Volume

To help make sense of the different property combinations, PYroMat organizes properties into groups based on the information that can be gained from them.

- Basic properties: These are the most physically intuitive properties – usually obtainable from direct physical measurement. In general, specifying the thermodynamic state with these is faster and more numerically precise.
 - Temperature, T ,
 - Pressure, p ,
 - Density, ρ ,
 - Specific volume, v ,
 - Quality, x .
- Primary properties: Every model uses two basic properties to uniquely define a thermodynamic state, from which all the others can be calculated. Specifying the thermodynamic state with these is always the fastest, most precise method.
 - Ideal gases: T, p ,
 - Multi-phase models: T, ρ .
- Energy properties: These are different ways of quantifying the energy contained in the system. Specifying pairs of energy properties is usually inadequate for defining the thermodynamic state, so it is almost always disallowed.
 - Temperature, T ,
 - Internal energy, e ,
 - Enthalpy, h ,
 - Free (Helmholtz) energy, f ,
 - Gibbs energy, g .
- Inverse properties: In order to determine the state (in primary properties) from these properties, PYroMat either needs to perform numerical iteration, or (in rare exceptions) a precise algebraic inverse is available.

- Ideal gases: ρ , e , f , g , h , s ,
- Multi-phase models: p , e , f , g , h , s .

Ideal gas property combinations supported by the `ig`, `ig2`, and `igmix` classes are listed in Table 1.3. Combinations are marked with “X” when they would either be self-contradictory or redundant. Every combination along the diagonal is obviously redundant, but simultaneously specifying specific volume and density is also redundant. As we show in Chapter 6, for an ideal gas, specifying internal energy, enthalpy and temperature are all theoretically equivalent.

The multi-phase property combinations supported by the `mp1` and `mp2` classes are shown in Tables 1.4 and 1.5 respectively. Almost all combinations that are theoretically valid are supported. A “o” represents a combination that is not supported, though there may be some cases where it is theoretically possible. The theory behind these combinations is somewhat more nuanced than in ideal gases.

In PYroMat, a property combination is disallowed if there are at least two distinct states that can be found providing the same combination. For example, Figure 1.1 shows a surface plot of density plotted against temperature and enthalpy for two-phase acetone. Except in the liquid region, the surface is very badly behaved numerically, but the important criterion that causes us to reject it as a valid property combination is that it wraps back on itself. For many values of T and h , and there are two values of ρ that produce the same value of h . Therefore, h, T does not specify a unique state.

Despite this problem, T, h and T, e are both allowed in `mp1`, and they can only be expected to give reasonable results in (or near) the liquid region. On the other hand, f and g are not permitted as arguments, and any combination of the inverse group is disallowed for reasons of numerical stability. For example it is theoretically valid to specify enthalpy and entropy together, but it requires iteration in two dimensions to extract the T, ρ state. This presents a significant challenge for global numerical stability.

The `mp2` class addresses the global numerical stability issue by stashing a table of property values that can be referenced to establish initial estimates for properties prior to local numerical polishing. It recognizes f and g in its arguments, but it avoids the numerical problems seen

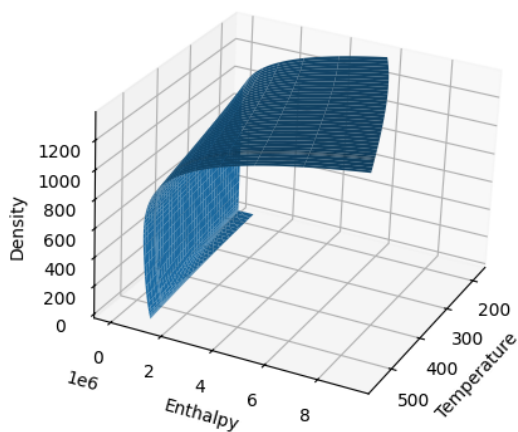


Figure 1.1: A surface plot of density plotted against temperature and enthalpy for a two-phase mixture of acetone.

Table 1.3: Property combinations that are supported by ideal gas classes in version 2.4.5. ✓ = supported, X = not valid, o = not supported.

	T	p	d	v	e	h	s
T	X	✓	✓	✓	X	X	✓
p	✓	X	✓	✓	✓	✓	✓
d	✓	✓	X	X	✓	✓	✓
v	✓	✓	X	X	✓	✓	✓
e	X	✓	✓	✓	X	X	✓
h	X	✓	✓	✓	X	X	✓
s	✓	✓	✓	✓	✓	✓	X

Table 1.4: Property combinations that are supported by the `mp1` class in version 3.0.1. ✓ = supported, X = not valid, o = not supported.

	T	p	d	v	x	e	h	s
T	X	✓	✓	✓	✓	✓	✓	✓
p	✓	X	✓	✓	✓	✓	✓	✓
d	✓	✓	X	X	X	✓	✓	✓
v	✓	✓	X	X	X	✓	✓	✓
x	✓	✓	X	X	X	X	X	X
e	✓	✓	✓	✓	X	X	X	o
h	✓	✓	✓	✓	X	X	X	o
s	✓	✓	✓	✓	X	o	o	X

in `mp1` by disallowing any combination of properties from the energy group.

1.1.2 Density, ρ

It is convenient to begin our discussion of properties with density since it is the easiest to define.

Density is the quantity of a substance per unit volume it occupies in space. It can be described as a number of molecules or mass per unit space. When described with molar units, it is usually called concentration, but PYroMat does not make that distinction.

Table 1.5: Property combinations that are supported by the multi-phase classes in version 3.0.1. ✓ = supported, X = not valid, o = not supported, E = energy group.

	T	p	d	v	x	E	s
T	X	✓	✓	✓	✓	o	✓
p	✓	X	✓	✓	✓	✓	✓
d	✓	✓	X	X	X	✓	✓
v	✓	✓	X	X	X	✓	✓
x	✓	✓	X	X	X	X	X
E	o	✓	✓	✓	X	o	✓
s	✓	✓	✓	✓	X	✓	X

When the unit volume is very tiny, this is a poorly defined quantity. For example, as the volume shrinks to be about the same size as the distance between molecules, the density one might measure would vary hugely as individual molecules entered and left the region of space. However, as the volume grows within a thermodynamically homogeneous region, the ratio of matter to volume converges to a consistent well-defined value.

This introduces the idea that the study of thermodynamics is essentially a careful study of averages over a large population of mechanical bodies. It is important that we study a quantity of a substance large enough that the quasi-random motions of individual molecules do not weigh heavily in our measurements. On the other hand, our measurements must consider a region of space small enough that the properties do not vary significantly. We may consider a region to be thermodynamically homogeneous if any two of its halves exhibit identical thermodynamic properties.

The density of the gas in molecules per unit volume is

$$n = \frac{N}{V}. \quad (1.1)$$

where V is the volume of the region, and N is the number of molecules. These numbers are extremely large, and early in the history of chemistry, there was no way to know the true number of molecules in a sample anyway. So, the use of molar quantities was of great utility.

The density in molar units is a slight variation on n ,

$$\bar{\rho} = \frac{N}{N_a V} = \frac{\bar{N}}{V}, \quad (1.2)$$

where N_a is Avagadro's number, and $\bar{N}$ is the number of moles in the sample. As we will see in Section 4.5.4, other molar units exist, but the same formula applies. In this way, the density of molecules may be expressed either in a literal count per unit volume, n , or in a number of moles per unit volume, $\bar{\rho}$.

In mass units, the density is merely multiplied by the molecular weight of the species,

$$\begin{aligned} \rho &= m_0 \frac{N}{V} \\ &= \frac{\bar{N}W}{V} = \frac{NW}{N_a V} \end{aligned} \quad (1.3)$$

when m_0 is the mass of a single molecule, and W is the more commonly used molecular weight in atomic mass units.

Since ρ is not available in the ASCII character set, PYroMat uses `d` to represent density.

1.1.3 Specific volume, v

Specific volume is the volume occupied by a unit of matter, and is calculated as the inverse of density.

$$v \equiv \frac{1}{\rho}. \quad (1.4)$$

Density is the property with the clearer fundamental definition. After all, how much space can be said to be occupied by a gas molecule? Formulations that address the average distance between molecules essentially reduce to defining specific volume as the inverse of density.

However, specific volume is mathematically identical to other intensive properties because it expresses the quantity of interest (volume) per unit matter. That makes it extremely convenient for a number of thermodynamic calculations.

1.1.4 Temperature, T

Temperature scales were originally developed as quantitative means for describing hot and cold, but they were developed before we had more physical descriptions for their meaning. After all, the existence of molecules and atoms was still being hotly debated while temperature scales were already in wide scientific and engineering use.

We now understand temperature to be an observable measure of the molecular translational kinetic energy of a substance. In a gas, the molecules are free to translate through space, and temperature is proportional to their kinetic energy. In a liquid or solid, the same energy manifests as molecular vibration within the confines of the intermolecular forces.

For a gas,

$$\langle \frac{1}{2}mu^2 \rangle = \frac{3}{2}kT, \quad (1.5)$$

Here, m is the mass of an individual molecule, $\langle u^2 \rangle$ is the mean square of velocity, k is the Boltzmann constant, and T is the temperature in absolute units.

The ITS-90 temperature scale establishes an international standard for the definition of temperature in terms of the triple points of various pure substances. Many of the property models included in PYroMat were formulated when the previous ITPS-68 was the international temperature scale, but the changes were so minute that the uncertainties in the properties usually dominate[1, p.10]. As a result, they are treated as interchangeable for the purposes of this handbook.

See chapter 4.5.6 for more information on the formal definition of temperature.

1.1.5 Pressure, p

Pressure is the static force exerted by a fluid on a surface. It is quantified in force per unit area of the surface, and it always acts normal to the surface facing into the surface (away from the fluid).

In a gas, pressure is due to the impact of molecules on the surface. Pressure may be increased by increasing either their velocity (temperature) or their quantity (density). Because these effects are linear in

a gas, this leads to the famous ideal gas relationship between density, temperature, and pressure.

In a liquid or solid, intermolecular forces that cause pressure are persistent instead of momentary (due to collisions in a gas). Under these conditions, even slight changes in intermolecular spacing causes huge changes in pressure, making the substance quite stiff in comparison to gases. In this case as well, increasing temperature makes molecules vibrate more violently in their equilibrium with each other, so at a consistent average density, the force applied to a neighboring surface will increase. This is why substances appear to expand as they are heated.

1.1.6 Internal Energy, e

Energy can be stored in a fluid in many ways. In gases, for example, the molecules translate with great speed (see temperature), the molecules vibrate and rotate, and there is incredible energy stored in the chemical bonds of molecules. However, all of these are dwarfed by the energy contained in the forces binding the nucleus of each atom.

We account for *all* of these energies simultaneously with the internal energy, e , which has units energy per matter (e.g. J/kg). It is neither practical nor necessary to tally all of these energies in an absolute fashion. Especially since most applications will have no release of nuclear energy, it is practical, instead, to describe how the substance's energy changes relative to some reference state. This is not unlike defining a reference height for gravitational potential energy calculations in classical mechanics. The choice is arbitrary and has no bearing on the result, but it can drastically simplify the calculations.

Therefore, we say that the internal energy, e is the sum of vibrational, rotational, translational, chemical, and nuclear energies contained in a thermodynamically homogeneous unit matter, subtracted by the same sum at some reference state.

Some readers will be scandalized that PYroMat uses e in favor of the traditional u for internal energy. This is a genuine (if futile) attempt to untangle the web of contradictory variable use between thermodynamics and fluid mechanics. u is reserved for velocity, v for volume, and e for energy.

1.1.7 Enthalpy, h

When a fluid of any kind is flowing, it carries its internal energy with it, but it also does mechanical work as it flows. The mechanical work done by a moving fluid is $p dV$, where p is the pressure exerted and dV is a differential volume displaced by the fluid. If we were to imagine that the volume were displaced while the fluid is expanding or contracting, the same idea applies to a fluid whether it is flowing or not. Per unit mass, this can be expressed as $p dv$ (when v is specific volume). Integrated over an isobaric (constant-pressure) process, this becomes simply $p v$.

It is a matter of convenience for engineers and physicists that deal with fluid power, that we define enthalpy as the sum of internal energy and fluid power,

$$h \equiv e + p v = e + \frac{p}{\rho}. \quad (1.6)$$

Of course, most processes aren't *actually* isobaric, so extra terms will tend to appear in energy balances (like $dh - v dp$).

Enthalpy is most commonly used in the analysis of any flowing fluid such as in heat exchangers, combustors and burners, chemical reactors, compressors, turbines, valves, etc... Even though internal energy might be argued to be more “fundamental,” enthalpy is usually tabulated instead because of its widespread use.

Since internal energy and enthalpy are related, their reference states *must* be consistent with one another. By tradition in both the NIST-JANAF and NASA data sets, the enthalpy of a substance is defined at 298.15 K and 1 bar, and values at other conditions are calculated from specific heat and the physics of ideal gases. Since the PYroMat multi-phase models permit only pure substances, their reference states do not need to be interchangeable. By tradition, the internal energy and entropy of these substances are defined as zero for the liquid at the triple point.

See the ideal gas (Chapter 6) and multi-phase (Chapter 7) chapters for more information on how the reference states are handled.

1.1.8 Entropy, s

The idea of entropy is born with the Clausius statement of the Second Law of Thermodynamics, which says that a reversible cyclic series of processes that include heat transfer must obey

$$\oint \frac{\delta q}{T}_{rev.} = 0, \quad (1.7)$$

where T is the temperature of the substance and δq is the addition of heat. It is important to emphasize that this is only true of processes that result in a continuous cycle where the fluid ends at the same thermodynamic state from which it began (like in an engine or refrigeration cycle).

The first law would be satisfied by any cycle where the work and heat transfer merely summed to zero, but the Clausius equality implies something deeper. Heat is not a property of the substance, but heat added reversibly in ratio with the temperature consistently returns to zero when the substance returns to its original thermodynamic state. That implies the existence of a new (and terribly important) property. Clausius termed that property entropy,

$$ds \equiv \frac{\delta q}{rev. \ T} \quad (1.8)$$

From the first law of thermodynamics, $\delta q = de + pdv$, and we have a way to calculate entropy from the other properties

$$Tds = de + pdv \quad (1.9)$$

$$= dh - vdp \quad (1.10)$$

This calculation is performed by putting the substance through a hypothetical reversible process in which there is neither chemical reaction, nor field work (e.g. gravitational or electrical).

Just like with temperature, there is a parallel (mathematically consistent) definition of entropy from statistical mechanics. The above definition of entropy had long been in use, when Boltzmann found that Entropy can also be calculated from the probability of the substance obtaining a thermodynamically equivalent state. The probability of

gas molecules spontaneously exhibiting a specific set of positions and velocities is astronomically low, but there is an immense quantity of dissimilar positions and velocities that would give the gas precisely the same temperature, internal energy, pressure, density, and other properties. For a given volume populated with a given substance with a given number of molecules with a given amount of total energy, how numerous are those states in comparison with all the other possible states?

Entropy is high when the number of equivalent states is high. Entropy is low when the number of equivalent states is low. The idea was fiercely resisted when Boltzmann and his contemporaries first argued it. From one point of view, it is astonishing that the idea was first discovered by brilliant insight *without* statistics.

Just like internal energy, it is theoretically possible to tally all of the possible states and calculate the probability of each, but that task is neither practical nor necessary. Instead, it is convenient to define the entropy as zero at some reference state and then deal merely in changes in entropy as defined by (1.8). In the case of ideal gases, the entropy is known to be precisely zero at absolute zero temperature, but some multi-phase models (like water) set entropy (and internal energy) of the liquid phase to zero at the triple point.

1.1.9 Free (Helmholtz) energy, f

The free energy is written in terms of internal energy and entropy,

$$f \equiv e - Ts. \quad (1.11)$$

While free energy has long been used as a tool for calculating the mechanical work available from a substance at a given temperature, modern equation-of-state substance models use f as the property from which all others are calculated.

There is tremendous potential for confusion, because various authors have also referred to Gibbs energy as “free energy.” Here, we strictly reserve that phrase to refer to $e - Ts$.

Sources can be found using a , ϕ , α , and many other characters to represent free energy. We adopt f for its relationship to the word “free,” and to reserve these variables for other uses.

1.1.10 Gibbs energy, g

The Gibbs energy is defined as,

$$\begin{aligned} g &\equiv e + \frac{p}{\rho} - Ts \\ &= h - Ts \\ &= f + \frac{p}{\rho}. \end{aligned} \tag{1.12}$$

It has historically been called *free enthalpy* or Gibbs free energy as well. Here, we adopt the recommendations of the IUPAC and refer to it simply as *Gibbs energy*.

Its use is primarily in systems with chemical or phase reactions. In these applications, properties are written in terms of the *chemical potential*, μ_i , which is the contribution of each constituent to the internal energy,

$$\begin{aligned} dE &= TdS - pdV + \sum_i \mu_i dm_i \\ &= TdS - pdV + \sum_i \bar{\mu}_i dN_i \end{aligned}$$

Note that the properties in these formulae are extensive (total energy rather than energy per unit matter).

Here, chemical reactions or phase transitions cause changes in the mass, m_i , or moles, N_i , of each substance, which can consume or release energy. So, for a hypothetical reaction with constant entropy and volume,

$$\mu_i = \left(\frac{\partial E}{\partial m_i} \right)_{s,v} = \left(\frac{\partial e}{\partial y_i} \right)_{s,v}. \tag{1.13}$$

Because reactions conserve mass, it is equally valid to write the chemical potential in terms of the intensive properties. This only works because $d(em) = mde$. The same is not true of $d(\bar{e}N)$, because dN is not necessarily zero in a chemical reaction.

Specifying properties in terms of volume and entropy in a reacting experiment is not particularly practical. Examining chemical potential

in terms of enthalpy merely transposes the expression into constant entropy and pressure. However, when we examine the differential of g ,

$$dg = (de) + (pdv + vdp) - (Tds + sdT),$$

and substitute to find the chemical potentials,

$$\begin{aligned} dg - (pdv + vdp) + (Tds + sdT) &= Tds - pdv + \sum_i \mu_i dy_i \\ dg &= vdp - sdT + \sum_i \mu_i dy_i. \end{aligned}$$

Temperature and pressure are among the most commonly specified properties when performing thermodynamic calculations, so it is extremely convenient to write the chemical potential in terms of Gibbs energy,

$$\mu_i = \left(\frac{\partial g}{\partial y_i} \right)_{T,p}. \quad (1.14)$$

In chemical reactions, it is unusual to work in intensive units, since the number of moles changes. Instead, this relationship is usually expressed extensively.

$$\mu_i = \left(\frac{\partial G}{\partial m_i} \right)_{T,p}. \quad (1.15)$$

$$\bar{\mu}_i = \left(\frac{\partial G}{\partial N_i} \right)_{T,p}. \quad (1.16)$$

1.1.11 Specific Heats, c_p c_v

Of the properties so far defined, only temperature, pressure, and density can be directly measured. The specific heats are vitally important to the study of a substance, because they can be conveniently directly measured and the other properties calculated from them.

Specific heat, c , is the amount of heat per mass of a substance, δq , required to obtain a small increase in temperature, δT ,

$$c \equiv \frac{\delta q}{\delta T}. \quad (1.17)$$

Especially when dealing with gases, we have to be more specific because substances have a tendency to expand when they are heated. One measures a different value for specific heat depending on whether or not expansion is allowed.

For any process, energy will be conserved, so

$$\delta q = de + pdv. \quad (1.18)$$

Here, δq is heat added per mass of the substance, de is the change in internal energy, p is the pressure, and dv is the change in volume per unit mass.

When heat is applied in such a manner that the substance's volume is constant, $dv = 0$, the mechanical work is zero, and all of the heat is stored as internal energy.

$$\begin{aligned} \delta q|_v &= de \\ &= \left(\frac{\partial e}{\partial T} \right)_v dT \end{aligned} \quad (1.19)$$

Thus, the constant-volume specific heat is, by definition,

$$c_v \equiv \left(\frac{\delta q}{\delta T} \right)_v = \left(\frac{\partial e}{\partial T} \right)_v. \quad (1.20)$$

In this process, the substance's pressure will rise (sometimes sharply) as it is heated in a rigid container.

When one considers addition of heat under constant pressure, a trick application of the chain rule for the term, pv , lets us transition the differential on volume into a differential on pressure. The definition for enthalpy (1.6) appears naturally.

$$\begin{aligned} \delta q &= de + pdv + vdp - vdp \\ &= d(e + pv) - vdp \end{aligned} \quad (1.21)$$

When heat is added while pressure is constant (like in an atmospheric gas), $dp = 0$, and

$$\begin{aligned} \delta q|_p &= dh \\ &= \left(\frac{\partial h}{\partial T} \right)_p dT \end{aligned} \quad (1.22)$$

Thus, constant-pressure specific heat is, by definition,

$$c_p \equiv \left(\frac{\delta q}{\delta T} \right)_p = \left(\frac{\partial h}{\partial T} \right)_p. \quad (1.23)$$

1.1.12 Speed of sound, a

Like specific heats, the speed of sound is the property of a substance that can be measured in the laboratory to high precision, making it a valuable tool. Pressure wave propagation in a substance obeys the equation,

$$\frac{\partial^2 p}{\partial t^2} + \left(\frac{\partial p}{\partial \rho} \right)_s \frac{\partial^2 p}{\partial x^2} = 0. \quad (1.24)$$

A wave with wavenumber, k , propagating at constant speed, a , might take the form

$$p = \sin \left(k(x + at) \right).$$

This satisfies (1.24) only when

$$a^2 \equiv \left(\frac{\partial p}{\partial \rho} \right)_s. \quad (1.25)$$

So, wave speed in a substance is the square root of the isentropic rise in pressure with increasing density.

Wave speed is usually denoted with the variable, c , which we reserve for specific heats. Authors can be found using w and a , and we have selected the latter.

Chapter 2

Getting started

In this chapter, we discuss everything you should need to get your own installation of PYroMat working and providing properties.

2.1 Installation

PYroMat is distributed primarily through the Python package index¹, but it can also be downloaded and installed “manually” if you have Python’s `setuptools` installed. PYroMat is written in plain Python, so there is no need to compile to binaries. All data are encoded in `json` format (<https://docs.python.org/3/library/json.html>), and the configuration files are executable python code. The result is that all of the code, data, and configuration files are in human readable text using only the ASCII character set.

2.1.1 Prerequisites

Obviously, a Python interpreter needs to be installed on your system, but PYroMat is deliberately designed to minimize the number of pre-requisites. Only Numpy version 1.7 or later is required.

It is worth noting that PYroMat was originally developed in Python 2, but as of January 2020, Python 2 was officially deprecated, and PYroMat testing and support on Python 2 halted. On a number of

¹Python package index: <https://pypi.org/>

popular Linux systems, Python 2 is still the system default, so users should take note. If your system has both Python 2 and Python 3 installed, it may be necessary to replace `python` in the instructions below with `python3` to specify the major version.

The examples that follow use a `$` to represent the prompt of a command line. On Linux, it might appear `user@machine ~ $` and on Windows it might appear `C:\>`.

2.1.2 Installation with a package manager

Global installation with pip was once the most common way to install packages. Python uses a package called `pip`² to manage its own dependencies. You can use it to automatically download and install PYroMat and its dependencies with a single command.

```
$ python -m pip install --upgrade pip
$ python -m pip install --upgrade pyromat
```

This makes sure the package manager itself is up-to-date and installs or upgrades PYroMat to its latest released version. In many systems, these commands are run globally, so you will need administrator permissions. Otherwise, consider using a virtual environment (see below).

Backwards compatibility is a high priority in PYroMat development, but major version changes (e.g. 1.X.X to 2.X.X to 3.X.X) allow changes to the user layer that might break some users' codes. Users who need to enforce some specific rules can use the boolean operators to specify limits on the desired version.

```
$ python -m pip install pyromat==2.2.0
$ python -m pip install 'pyromat<3.0.0'
```

See https://pip.pypa.io/en/stable/user_guide/ for more information.

Virtual environments are used pretty universally these days. They are self-contained Python interpreters and packages, so users don't need to install their packages in the same interpreter that is used to run system scripts. Virtual environments can be great tools

²pip docs: <https://pip.pypa.io>

for educators and students working on machines without administrator privileges and for developers who want to contain their testing. As of Python 3.5, `venv`³ is the Python command for creating a virtual environment.

(1) First, make sure `venv` is installed and updated. On a Linux installation, the best way to do this is usually to use the system's native package manager system. For example, on Ubuntu or Mint,

```
$ apt update
$ apt install python3-venv
```

On systems without a package manager (like Windows),

```
$ python -m pip install --upgrade venv
```

(2) Create your environment

```
$ python -m venv <VENV DIRECTORY>
```

This command will create a directory and place in it all the files needed for an independent little world of Python. The `VENV DIRECTORY` placeholder should be replaced with the path where you want your environment to be placed.

(3) “Activate” the virtual environment. This redirects terminal commands to the binaries in this virtual environment instead of the system-wide Python interpreter. In Linux or MacOS distributions, these are located in a directory, `<VENV DIRECTORY>/bin/`, and on Windows, `<VENV DIRECTORY>\Scripts\`. On a Linux distribution (in a bash terminal),

```
$ source <VENV DIRECTORY>/bin/activate
```

and on Windows

```
C:\> <VENV DIRECTORY>\Scripts\activate.bat
```

On most systems, the terminal will now appear

```
(<VENV DIRECTORY>) $
```

to remind the user that your commands are being executed in the environment, and not on the system.

(4) Install PYroMat and its dependencies

³venv docs: <https://docs.python.org/3/library/venv.html>

```
(<VENV DIRECTORY>) $ pip install --upgrade pyromat
```

This should automatically install both PYroMat and the latest version of Numpy.

(5) Start the interpreter, and start using PYroMat. Here is what that looks like on my system.

```
(<VENV DIRECTORY>) $ python
Python 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC
 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for
more information.
>>> import pyromat as pm
>>> pm.config['install_dir']
'/home/calcul/Documents/test-env/lib/python3.12/site-
packages/pyromat'
```

I have called up PYroMat's 'install_dir' parameter to verify that I am using the copy of PYroMat that is located in my virtual environment. For me, that was an environment called `test-env`.

For more information, see the `venv` documentation.

2.1.3 Installing in popular IDEs

In an Anaconda installation start the “Anaconda Prompt” to bring up a terminal, and use the `conda` command much like `pip`⁴.

```
$ conda install pyromat
```

This works inside Anaconda’s own virtual environment with the advantage that you don’t need to set it up yourself.

In a Spyder installation, the IDE will be using either a virtual environment or the system’s global Python interpreter. Provided you have appropriate permissions, the IPython console will usually allow you to simply install packages

```
In [1]: pip install pyromat
```

If that isn’t working, there may be something unusual about your installation. Refer to Spyder’s documentation⁵ for guidance.

⁴Anaconda: <https://www.anaconda.com/docs/getting-started/working-with-conda/packages/install-packages>

⁵Spyder: <https://docs.spyder-ide.org/current/index.html>

In a Thonny installation, users can use a graphical package manager⁶ to automatically install in Thonny's own virtual environment without ever needing to think about it. For many educators, this may be the best option, since it is easy to use, and does not require administrator privileges. In the top menu, navigate to "Tools & Manage Packages." In the search bar, type "pyromat." The manager should find PYroMat. Click "Install."

2.1.4 Manual installation with git

This method of installation is not recommended. So-called manual installation is also quite easy using `git` if you already have Python's `setuptools`⁷ installed and updated.

Just clone the `git` repository, then navigate into the root directory and run the `setup.py` installation script. This is the stage where an error will be generated if there is a problem with `setuptools`.

```
$ git clone https://github.com/chmarti1/pyromat.git
$ cd pyromat
$ python setup.py install
```

2.1.5 Manual installation from Sourceforge

This method has largely fallen out of favor, but it is still a perfectly valid way to perform an installation. Like the GitHub method, this method only works if you have `setuptools` installed and updated (see the footnote below).

Download the latest version of PYroMat from <https://sourceforge.net/projects/pyromat/>. Select whichever compression type that suits you (e.g. zip, bz2, gzip). Then, extract the package, creating a `pyromat` directory.

Bring up a command prompt (see above for how to do that on your system), and navigate to the extracted directory using the appropriate `cd` commands. Finally, execute the Python setup script.

```
$ cd /path/to/your/dir
```

⁶Thonny: <https://thonny.org/>

⁷Setuptools: <https://setuptools.readthedocs.io/en/latest/>

```
$ python setup.py install
```

2.2 Using PYroMat

This section assumes that users already have a basic familiarity with the Python command line. For users that are just getting started, try sampling the official Python Tutorial (<https://docs.python.org/3/tutorial/index.html>).

First, open a command prompt (terminal) and start Python in interactive mode.

```
$ python3 # On my system, I need to specify 3
Python 3.6.9 (default, Jan 26 2021, 15:33:00)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for
more information.
>>>
```

2.2.1 Importing

Import PYroMat like any other package.

```
>>> import pyromat as pm
```

It automatically loads its modules, and automatically seeks out and loads data.

Once loaded, there are only three functions in the base PYroMat package that most users will ever need: `get()`, `search()`, and `info()`. These are tools that interact with the PYroMat data collection to retrieve data class instances and print information about them. All of the rest of the functionality is offered by the class instances themselves.

2.2.2 Retrieving substance data

PYroMat identifies substances by a unique string called the substance identifier (ID). The substance ID string for ideal gas nitrogen is `'ig.N2'`. The `ig` prefix indicates that the substance model belongs to PYroMat's ideal gas collection. The rest of the string is the substance's chemical formula using Hill notation.

Hill notation removes the ambiguity in how a chemical formula is represented by mandating that all substances with the same atomic composition will be represented by the same strings. The atomic contents are listed in order of carbon, hydrogen, and then all others in alphabetical order.

By deliberately omitting all chemical structure from the notation, there will be collisions between compounds with the same atomic composition in dissimilar arrangements. So far, these collisions are rare in the PYroMat data collection, and they are addressed by appending an underscore and an index. For example, the CNN radical and methanetetraylbis-amidogen both have the same chemical formula, so the former has the ID string, `ig.CN2`, and the latter is listed under `ig.CN2_1`.

Given an ID string, the `get` function returns the corresponding class instance. This instance provides all of the property methods. This example calculates the ideal gas enthalpy of nitrogen at 492 K.

```
>>> n2 = pm.get('ig.N2')
>>> n2.h(T=492)
array([202.68455864])
```

In this example, the variable, `n2`, is an ideal gas class instance with all the methods (like `h()`) needed to calculate its properties.

2.2.3 Searching for substances with `search()`

There are about 1,000 unique substance models in PYroMat version 2.2.0, so manually skimming through the list is not a very efficient means of finding the species you want. The `search()` function returns sets of substances that match the criteria supplied. Those sets can either be used directly, or the `info()` function can display more information (see below).

Searching is done by specifying keywords to the `search()` function. Each one adds a new criterion that further narrows the search. The results are

The **name** keyword is used to search for an exact match with part of the substance ID or for a case-insensitive match with one of the substance's names. For example,

```
>>> pm.search(name='Acetylene')
{<ig2, ig.C2F2>, <ig2, ig.C2HCl>, <ig2, ig.C2HF>, ...}
>>> pm.search(name='C2H')
{<ig2, ig.C2HCl>, <ig2, ig.C2HF>, <ig2, ig.C2HN>, ...}
```

These show how a name string can be used to match part of one of a substance's common name *or* part of the substance ID string.

The **pmclass** is the name of the class that is used to implement the data model for each substance. As of version 3.0.1, there are four classes implemented: **ig**, **ig2**, **igmix**, **mp1**, and **mp2**. These are described in detail in sections 6.2.1, 6.2.2, 6.2.3, ??, and ?? respectively.

The **collection** keyword allows users to specify the collection to which species must belong. This forces the substance id string to begin with “collection.” For example **collection='ig'** forces all of the substances returned to belong to the ideal gas collection.

The **InChI**[2] and **CAS**[3] substance identifiers can also be used to narrow the search. Since these are unique substance identifiers, they will only return multiple entries when the same substance is represented in multiple collections. To specify these, use **inchi=...** or **cas=...** as keyword arguments. It is important to keep in mind that as of version 3.0.1, not all entries have had these values added to the data, so some results may not come back as expected.

Probably the most powerful of the search tools, the **contains** keyword allows users to specify the atomic contents of the substance. Values may be the string name of an element, or lists of element name strings, or dictionaries with element names and their precise amounts. For example, these queries return all substances (1) with any amount of iron, (2) with any amount of bromine and carbon, and (3) with two hydrogen, one oxygen, and any nonzero amount of carbon.

```
>>> pm.search(contains='Fe')
{<ig, ig.Br4Fe2>, <ig, ig.FFe>, <ig2, ig.H2FeO2>, ...}
>>> pm.search(contains=['Br','C'])
{<ig, ig.CBrF3>, <ig, ig.CBr4>, <ig, ig.CBr>, ...}
>>> pm.search(contains={'H':2, 'O':1, 'C':None})
{<ig2, ig.C2H2O>, <ig2, ig.CH2O>}
```

Because the **search()** function returns a Python **set** instance, applications that demand more nuanced search operations can use the **set**

operations to combine the results of separate searches. For example, to find substances that contain two hydrogen *or* three carbon,

```
>>> h2 = pm.search(contains={'H':2})
>>> c3 = pm.search(contains={'C':3})
>>> h2.union(c3)
{<ig, ig.H2P>, <ig2, ig.C6H2>, <ig2, ig.CH2Cl2>, ...}
```

For more tips and tricks with sets, see the Python documentation <https://docs.python.org/3/tutorial/datastructures.html#sets>.

2.2.4 Finding substances with info()

When called without any arguments, the `info()` function lists all of the substances available, and it lists all of the property methods available for each. That probably won't be useful for most users, so the function also passes any keyword arguments to `search()` internally to narrow down the set.

```
>>> pm.info(contains=['Br','C'])
PYroMat
Thermodynamic computational tools for Python
version: 2.2.0
-----
ID          : class : name                               : properties
-----
ig.CBr      : ig     : Bromomethylidyne             : T p d v cp cv gam e h s mw R
ig.CBr4     : ig     : Carbon tetrabromide          : T p d v cp cv gam e h s mw R
ig.CBrF3    : ig     : Bromotrifluoromethane        : T p d v cp cv gam e h s mw R
ig.CBrN     : ig     : Cyanogen bromide             : T p d v cp cv gam e h s mw R
```

Alternately, an iterable of substance instances (like the one generated by `search()`) may be passed directly to `info()`.

The table shows each substance listed by its ID string and one of its common names (if one is included in the data). There is also a list of the properties currently offered in the data model.

If a substance ID string, a single substance instance, or a set (or other iterable) with only one entry is passed, the `info()` function prints detailed information on that substance.

```
>>> pm.info('ig.N2')
***
Information summary for substance: "ig.N2"
***
N
2

Names : Nitrogen
        Nitrogen gas
        N2
```

```

UN 1066
UN 1977
Dinitrogen
Molecular nitrogen
Diatomic nitrogen
Nitrogen-14
Molecular Weight : 28.01348
CAS number : 7727-37-9
InChI string : InChI=1S/N2/c1-2
Data class : ig2
Loaded from : /home/chris/Documents/pyromat/src/pyromat/data/ig2/N2.hpd
Last updated : 21:11 April 20, 2022

```

The supporting data for this object were adapted from: B. McBride, S. Gordon, M. Reno, "Coefficients for Calculating Thermodynamic and Transport Properties of Individual Species," NASA Technical Memorandum 4513, 1993.

2.2.5 In-line documentation

While it is also designed to run efficiently in scripts, all aspects of PYroMat were designed with ease of use from the command line in mind. Most users will first learn PYroMat through the command line and then go on to write scripts that automate their calculations.

With that in mind, every class instance, every method, and every module has in-line documentation that can be accessed using Python's built-in `help()` function. For example, try typing:

```

>>> help(n2)
>>> help(n2.h)
>>> help(pm)

```

2.3 Property interface

The property methods that belong to the many substance class instances use flexible arguments that are as standardized as is practical. Details about the individual properties and the theory behind them is included in chapters 6 and 7, but there are some general rules that apply to all substances in PYroMat.

2.3.1 Property method arguments

With a few special exceptions, a thermodynamic state can be specified with any two properties (except specific heats). For example,

```
>>> n2 = pm.get('ig.N2')
>>> T = 452.
>>> p = 14.
>>> n2.s(T=T, p=p)
array([6.49072181])
>>> n2.s(T=T, d=n2.d(T=T, p=p))
array([6.49072181])
```

Or, with multi-phase data,

```
>>> n2 = pm.get('mp.N2')
>>> n2.s(T=T, p=p)
array([6.48696146])
>>> n2.s(T=T, d=n2.d(T=T, p=p))
array([6.48696146])
>>> n2.s(T=100, x=0.5)
array([4.18094321])
```

The last line uses quality to specify a 50/50 mixture of liquid and vapor nitrogen at 100 K.

This works with all basic properties (see section ??) T , p , ρ , v , and x (multi-phase only). In ideal gases, specifying e or h is equivalent to specifying T , so they can be expressed in almost any combination, just like a basic property. However, simultaneously specifying h , e , and/or T to an ideal gas property triggers an exception. Meanwhile, multi-phase classes do not tolerate simultaneously specifying h , e , and s ; not because it is theoretically invalid, but because it causes potentially unstable numerical iteration. These nuances are tabulated in Tables 1.3 and ??.

Other properties like e , h , and s , may be specified as well, but only one at a time with a basic property. For example, PYroMat does not support property evaluations with h and s simultaneously. Instead s or h may be specified with one of the basic properties like T .

```
>>> n2.h(T=100, x=0.5)
array([7.27884341])
>>> n2.h(s=4.18090744, T=100)
array([7.27884313])
>>> n2.T(s=4.18090744, h=7.27884313)
...
pyromat.utility.PMParamError: Properties may not be
specified together: s, h
```

Note that some numerical accuracy is lost in the process of inverting the property. Specifying a property like entropy is also far more computationally expensive than specifying basic properties like temperature, density, or pressure. As a result, it is best practice to calculate temperature, density, pressure, or quality and calculate other properties in terms of those.

```
>>> import pyromat as pm
>>> n2.T(s=n2.s(T=300,p=2.5), p=2.5)
array([299.99994672])
```

2.3.2 Default values

It is not unusual that users may want vaguely defined properties. For example, "what is the specific heat of water?" That question has an infinite number of answers depending on the state of the water, so a property method really can't answer it. On the other hand, when users want properties without specifying a state, they usually mean "...at standard conditions."

All property methods apply default primary property values when properties are unspecified. For example,

```
>>> h2o = pm.get('mp.H2O')
>>> h2o.cp()
array([4.18131499])
```

tells the user that the specific heat of liquid water is about 4.18 kJ/kg/K at 273.15 K and 1.01325 bar. That default temperature and pressure can be changed by setting the `def_T` and `def_p` parameters in `pm.config`. See chapter 3 for more information on configuring PYro-Mat.

When only one property is specified, the second property will revert to a default value. If temperature is specified, then pressure reverts to its default. If any other property is specified, then temperature reverts to its default and pressure is unspecified. For example,

```
>>> h2o.cp(T=540) # T=540K, p=1.01325bar
array([1.99666454])
>>> h2o.cp(p=0.01) # T=273.15K, p=0.01bar
array([1.87429701])
```

Table 2.1: Inverse property methods and their availability by class

	ig	ig2	mp1
T_h	✓	✓	✓
T_s	✓	✓	✓
d_s	□	□	✓
p_s	✓	✓	□

```
>>> h2o.cp(d=.01) # T=273.15K, d=.01kg/m3
array([1.87870465])
```

Note that the default values are specified in whatever units PYroMat has been configured to use. By default, PYroMat uses K, kJ, kg, bar, m³. If the unit system is changed, the default values should also be changed to reflect the intended values in the new unit system. See chapter 4 for more information on units and chapter 3 for more information on configuring PYroMat.

2.3.3 Inverse methods

Before version 2.2.0, the interface only accepted *basic properties* (see section ??) T , p , ρ , and x . To handle cases where users had secondary properties like entropy or enthalpy, methods like `T_s` were created to calculate “temperature from entropy” and one other property. Originally, `T_s`, `p_s`, `d_s`, `T_h`, and other *inverse property* methods were specific to the class and the property. The new interface introduced in version 2.2.0 obsoleted these methods, but they are still available for reverse compatibility. Table 2.3.3 shows the historical inverse property methods that are still available on the basic classes. They should not be used in new codes.

New codes designed to determine temperature (or any other property) from enthalpy or entropy should merely use a direct call to `T()` or other desired methods.

2.3.4 Tips and tricks

There are some guidelines that users can use to obtain dramatically better performance out of PYroMat.

Avoid calculating property values in a for loop. Instead, construct property values as arrays, lists, tuples, or other iterables and pass them to a single property method call. For example,

```
>>> import numpy as np
>>> import pyromat as pm
>>> h2o = pm.get('mp.h2o')
>>> ##### DON'T DO THIS #####
>>> h = []
>>> for T in np.linspace(300,1000,101):
...     h.append(h2o.h(T))
...
>>> ##### Do this instead #####
>>> T = np.linspace(300,1000,101)
>>> h = h2o.h(T)
```

On many systems, the first code segment can take four or five seconds to run! The second code segment consistently takes a small fraction of a second to run and makes for much cleaner code.

PYroMat is written to assume that all inputs and outputs are multi-dimensional arrays. Simple floating point scalars are taken to be special cases. Programming numerical codes like property evaluation in plain Python has a steep numerical penalty, but much of that can be regained by leaning on Numpy's (<https://numpy.org>) compiled back-end for efficient numerical methods on arrays.

Ideal gases are faster than the mp1 model. Ideal gases only have to evaluate a polynomial. The multi-phase mp1 model requires multiple parallel polynomials with computationally expensive exponential terms. Single equation-of-state multi-phase models sacrifice computational speed in favor of a single model that works well in liquid, gas, near critical, and super-critical states.

Prefer temperature and density when working with multi-phase substances. The mp1 class calculates all properties (including pressure) in terms of temperature and density. When another combination of properties is specified (e.g. temperature and pressure) an iterative routine has to run first to isolate temperature and density. That is why the example above takes so long to run; it requires two iterative algorithms run in series. When a series of properties are needed at a given state, it is much faster to calculate the density first and pass

temperature and density to all of the successive property methods.

Using the `state()` method is faster than separate calls to properties. In all classes, the `state()` method minimizes the number of back-end polynomial evaluations to calculate the properties at a state. If a user needs more than two properties, chances are good that it is worth it to just go ahead and calculate them all. Multi-phase properties especially require the calculation of a number of intermediate parameters that can be re-used in successive property evaluations.

The mp1 property methods can return quality too. When running a property method like `T()`, it is often a good idea to go ahead and calculate the quality by setting the optional keyword `quality=True`. All of these routines have to calculate the saturation properties anyway, so one might as well save the redundant steps.

```
>>> T,x = h2o.T(s=4.331, p=1.01325, quality=True)
>>> print(T,x)
[373.12429581] [0.50004124]
```

Chapter 3

Configuration

There are parameters that most users will never need to change, but that some users may find irritating or even limiting. For example, no unit system ever seems to make everyone happy. There is a configuration system that allows users to alter settings that are honored by the PYroMat substance model classes.

3.1 The `pm.config` instance

The PYroMat configuration system is implemented by a custom class instance called `config`. It mimics a dictionary in that it has a series of string parameters that have values associated with them. Evoking the configuration object prints its contents in a table.

```
>>> import pyromat as pm
>>> pm.config
      config_file : ['/home/chris/Documents/PYroMat/
      pyromat/defaults.py']
      config_verbose : False
      dat_dir : ['/home/chris/Documents/PYroMat/
      pyromat/data']
      dat_exist_fatal : False
      dat_overwrite : True
      dat_recursive : True
      dat_verbose : False
      ...
```

Unlike a dictionary, the `PMConfig` instance enforces certain rules that are specific to each of the configuration entries.

- Configuration entries can be neither created nor deleted.
- The value of a read-only entry (marked “R” in Table 3.1) cannot be changed.
- The values of an appendable entry (marked “A” in Table 3.1) cannot be overwritten. Instead, new values are appended to the end of a list of values.
- Entry values must be of an appropriate data type.

Table 3.1 lists the configuration parameters, their hard-coded initial values when the configuration instance is first created, and a brief description for each. Entries to which special rules apply are marked with an (A) for appendable or (R) for read-only.

Table 3.1: Configuration parameters, their rules, and their descriptions. Parameters that are appendable are marked with (A), and parameters that are read-only are marked with (R). Paths are referenced relative to the PYroMat installation directory and not the user working directory.

Entry (Rule)	Initial	Description
<code>config_file</code> (A)	<code>['../config.py']</code>	String file paths to load for configuration values
<code>config_verbose</code>	<code>False</code>	Print descriptions of the configuration load process?
<code>dat_dir</code> (A)	<code>['../data']</code>	String paths to directories to scan for <code>.hpd</code> files
<code>dat_exist_fatal</code>	<code>False</code>	Raise an exception if there are two definitions for the same substance id?
<code>dat_overwrite</code>	<code>True</code>	Overwrite older substance ids with newer ones?
<code>dat_recursive</code>	<code>True</code>	Recurse into sub-directories when looking for data files?
<code>dat_verbose</code>	<code>False</code>	Print descriptions of the data load process?

Table 3.1: continued...

Entry (Rule)	Initial	Description
def_T	298.15	Default temperature in units, def_T_unit
def_T_unit	'K'	Default temperature unit
def_oob	numpy.nan	Value to use when the state is out-of-bounds
def_p	1.01325	Default pressure in units, def_p_unit
def_p_unit	'bar'	Default pressure unit
error_verbose	True	Is verbose PYroMat error printing enabled?
install_dir (R)	'./'	Path to the PYroMat installation
reg_dir (A)	['./registry']	A list of directories to scan for .py files containing PYroMat classes
reg_exist_fatal	False	Raise an exception if there are two definitions for the same class?
reg_overwrite	True	Overwrite older class definitions with new ones?
reg_verbose	False	Print descriptions of the registry class discovery process?
unit_energy	'kJ'	Unit string for energy
unit_force	'N'	Unit string for force
unit_length	'm'	Unit string for length
unit_mass	'kg'	Unit string for mass
unit_matter	'kg'	Unit string for matter
unit_molar	'kmol'	Unit string for mole count
unit_pressure	'bar'	Unit string for pressure
unit_temperature	'K'	Unit string for temperature
unit_time	's'	Unit string for time
unit_volume	'm3'	Unit string for volume
version (R)	<version>	The PYroMat installation version string
warning_verbose	True	Is verbose PYroMat warning printing enabled?

3.1.1 Making temporary changes to config

Configuration entry values can be retrieved and written like dictionary values. In this example, we check the version string and change the temperature units to Fahrenheit.

```
>>> import pyromat as pm
>>> pm.config['version']
'2.2.0'
>>> pm.config['unit_temperature'] = 'F'
```

Parameters that are appendable do not behave the same way. When they are written to, they accumulate new values. For example,

```
>>> pm.config['config_file'] = '/etc/pyromat.py'
>>> pm.config['config_file']
['/home/chris/Documents/PYroMat/pyromat/config.py', '
/etc/pyromat.py']
```

Observe that instead of overwriting the previous configuration file, the new value was appended to the list.

Undoing a configuration change can be done using the `restore_default()` method. When it is used as an entry method, it only applies to that entry.

```
>>> pm.config['unit_temperature']
'F'
>>> pm.config.restore_default('unit_temperature')
>>> pm.config['unit_temperature']
'K'
```

The “default” values honored by the `restore_default()` method are hard-coded, so if there is a flaw somewhere in the configuration process, the system can be restored to functional settings by calling `restore_default()` with no argument. Then, all of the parameters will be reset to their defaults.

In the next section, we will discuss how to write configuration files that override these defaults. They will be automatically loaded when PYroMat is imported, but the `config.load()` method will manually repeat the configuration load process.

3.2 Configuration files

To make changes to the configuration that persist when PYroMat is reloaded, values should be written to configuration files. PYroMat configuration files are Python scripts that are expected to define variables with the same names as the configuration parameters. Any variables created that are recognized as valid PYroMat configuration parameters are read into the configuration. Any variables that are unrecognized or that have values with invalid types result in a `PMParmError` exception.

Loading all of its configuration files is the very first thing PYroMat does when it is loaded by the Python import system. First, `pm.config` is initialized with hard-coded initial values listed in Table 3.1. Even if all other aspects of the configuration process fail, the system will still be able to function with these basic parameters.

Then, the configuration load process begins. Configuration files are read one-by-one in the order they are listed in `config_file` until they have all been loaded. By default, the only path in `config_file` is `./config.py`, located in the PYroMat installation directory, but each time a configuration file is read in, new configuration files can be added. In this way, configuration files can add more configuration files. The loading algorithm protects users are protected against infinite loads by checking for files that reference each other or themselves.

The `config.py` file located in the base installation directory is heavily commented with instructions for administrators to make their own changes. Many administrators may want to add a configuration file in `/etc/` or in users' home directory somewhere. The example below shows a `config.py` file that admins might want to place in the PYroMat install directory to allow users to apply their own settings and write their own data models without a virtual environment.

This is an example of a configuration script that might appear in `config.py`.

```
# This adds two new configuration files.
# The first is a global configuration file in /etc.
# The second is in a hidden directory in the user's
# home directory. The order is important. Placing
# the user's settings last means that their options
# will overwrite the global options.
```

```

config = ['/etc/pyromat.py', '~/.pyromat/config.py']

# These add registry and data directories in the
# user's home directories.
reg_dir = '~/.pyromat/registry'
dat_dir = '~/.pyromat/data'

# These entries reconfigure PYroMat to use the
# imperial system of units. These entries
# could appear anywhere in the configuration
# process.
unit_temperature = 'F'
unit_length = 'ft'
unit_volume = 'ft3'
unit_pressure = 'psi'
unit_energy = 'BTU'
unit_matter = 'lb'
unit_force = 'lb'
unit_molar = 'lbmol'

```

For more information on the units, see [Chapter 4](#). For more information on the registry, data files, and the load process, see [Chapter 5](#).

Chapter 4

Units

PYroMat handles a wide variety of units automatically. The configuration system (see Chapter 3) allows users to select the units they prefer when interacting with the PYroMat user layer. Meanwhile the raw data that form the back-end substance models are coded in standard units unique to each class. The `pyromat.units` module provides the tools that translate between them. Interested users can use the back-end algorithms for their own purposes to convert between most units users need for their calculating pleasures.

A list of all currently available units is available by typing

```
>>> import pyromat as pm
>>> help(pm.units)
```

or use the `show()` method

```
>>> pm.units.show()
```

For a `dict_keys` instance listing the supported units, use a conversion instance's `get()` method,

```
>>> pm.units.length.get()
dict_keys(['km', 'm', 'cm', 'mm', 'um', 'nm', 'A', 'in', 'nmi', 'ft', 'yd', 'mile', 'mi'])
```

The unit systems used by the property methods and their corresponding configuration entries are listed in Tables 1.1 and 1.2.

This chapter infuses a little history along with the information PYroMat users are likely to find useful when interacting with the unit

conversion system.

There may seem to be a chauvinism in the author's choice of some units to include (such as the esoteric US customary volumetric units), and some of the sources are somewhat biased in favor of the US National Institute of Standards and Technology (NIST). The author acknowledges this bias. It is hoped that the reader will be reassured that first and foremost, units common in international scientific and engineering fields were included. The inclusion of some (perhaps out-dated) units peculiar to the US is mostly an indulgence in the author's curiosity.

4.1 Unit definitions

The Bureau International des Poids et Mesures (BIPM) International Committee for Weights and Measures (CIPM) is the entity responsible for the definition of the fundamental units on which the SI system is based, and from which almost all other units trace their definition. Nearly all other units in wide use are now derived from the SI units in some way, so rigorous treatment of their formal definition is worth some attention. There are countless web-based unit conversion calculators, but few use sufficient care in the derivation of their conversion factors to be trusted for serious work.

The second (time), meter (length), kilogram (mass), ampere (electrical current), kelvin (temperature), mole (quantity), and candela (luminous intensity) are the fundamental units in terms of which all other measures are derived. Contemporary definitions for these units are constructed so that certain important physical constants (like Planck's or Boltzmann's constants) may be represented exactly with finite precision, so the conversion factors used in PYroMat are, by definition, exact.

Before the turn of the twentieth century, the United States and the British governments had adopted the international yard and avoirdupois pound. This move re-defined the pound and the yard (and all those based on them) in terms of the meter and kilogram using an exact relationship with finite precision. The contemporary definitions for those fundamental units are constructed from highly stable natural phenomena may be independently reproduced in a laboratory, but this idea is

quite recent. [4]

This establishes a web of precise definitions of units for US customary, imperial, and SI systems. However, there are still popular measures (like the inch water column for pressure or scf for quantity of a gas) that depend on establishing so-called “standard” conditions whose precise definitions are often neglected (and are far from standard).

For example, water and mercury column measures for pressure depend on the density of a fluid and the local strength of gravity, both of which are subject to change. So-called “standard” pressure is often 1atm (1.01325bar), but is precisely 1bar for the NIST-JANAF data. Worse still, standard temperature usually refers to 273.15K (0°C), but the NIST-JANAF tables use 298.15K (25°C), and there are some engineering applications that still use 70°F. There are at least a half-dozen definitions for the calorie, which are construed from the specific heat of water at different conditions. These inconsistencies are far too severe to ignore, so great care is required when pulling data from multiple sources into a single database.

4.2 Setup

To maintain transparency in regard to the standards used to derive unit conversions, the PYroMat units system includes a **setup** function that is responsible for constructing all of the unit conversion functions. Users can inspect the constants and may change them at any time by re-calling the **setup()** function with new arguments. Its behavior is fully documented and can be found by evoking **help** as below.

```
>>> import pyromat as pm
>>> pm.units.setup(Tstd=273.15, pstd=1.01325, \
...               g=9.80665, dh2o = 999.972, dhg=13595.1)
>>> help(pm.units.setup)
```

It should be emphasized that there is a difference between the standard parameters used to construct the PYroMat unit system and the default parameters determined by **pm.config**. The standard conditions are only used to construct the unit conversion system, they are always given in the units documented below, and they are ignored outside of the **units** module. On the other hand, the default parameters

are used in property methods when arguments are omitted, and they are interpreted in whatever units are currently configured. See the configuration chapter (Chapter 3) for more information.

Standard temperature and pressure, `Tstd` and `pstd`, *must* be entered in units of Kelvin and bar respectively (regardless of the units configured in `pm.config`). The standard atmosphere was set to precisely 1.01325 bar by the BIPM General Conference on Weights and Measures (CGPM) in 1901 [5], and has been almost universally adopted. The default standard temperature is less universal, but is usually 273.15 K, the freezing point of water at one atmosphere of pressure.

The CGPM did not establish a standard acceleration due to gravity for the purpose of weights and measures until 1954. It is set to be precisely 9.80665 m s^{-2} [6]. This is a mean of gravitational forces experienced at sea level at a latitude of 45 degrees. This value can be found to agree precisely with the official conversions for pounds mass, pounds force, kilogram, and newton.

The densities of water and mercury, `dh2o` and `dhg`, must be entered in units of kg m^{-3} . The default for water is its density at atmospheric pressure and 4°C. The default for mercury is its density at atmospheric pressure and 0°C.

4.3 Constants

The `setup` function is called when the `units` module is first imported to initialize the PYroMat unit constants listed in Table 4.1 and declare all the unit conversion routines. These constants are used widely throughout the PYroMat system, so altering them is only recommended for advanced users who can anticipate the effects they will have.

Regardless of experience, constants should never be changed manually. They should only ever be changed by re-calling the `setup` function since the unit conversion routines will need to be updated to reflect the new value.

The values in this table are, by the definitions of units, exact (unless noted with $\approx$). Similarly, the conversions in the following sections are also exact unless they are otherwise noted.

Table 4.1: PYroMat constants and their default values. Values with a * are affected by calls to `setup()`.

Sym.	Name	Value		Description
g^*	<code>const_g</code>	9.80665	m s^{-2}	Gravity at 45° lat.
h	<code>const_h</code>	$6.62607015 \times 10^{-34}$	J s	Plank constant
k	<code>const_k</code>	1.380649×10^{-23}	J K^{-1}	Boltzmann constant
N_a	<code>const_Na</code>	$6.02214076 \times 10^{23}$	mol^{-1}	Avagadro number
N_c	<code>const_Nc</code>	$\approx 6.24150934 \times 10^{18}$	C^{-1}	Electrons per Coulumb
$\bar{\rho}_{std}^*$	<code>const_nstd</code>	44.61504819783	mol m^{-3}	Standard molar density
p_{std}^*	<code>const_pstd</code>	1.01325	bar	Standard pressure
q	<code>const_q</code>	$1.602176634 \times 10^{-19}$	C	Fundamental charge
R_u	<code>const_Ru</code>	≈ 8.314462618	$\text{J mol}^{-1} \text{K}^{-1}$	Universal gas constant
T_{std}^*	<code>const_Tstd</code>	273.15	K	Standard temperature
$\rho_{\text{H}_2\text{O}}^*$	<code>const_dh2o</code>	999.972	kg m^{-3}	Std. density of water
ρ_{Hg}^*	<code>const_dhg</code>	1,3595.1	kg m^{-3}	Std. density of mercury

4.4 Conversion Class

Unit conversions are performed by a callable `Conversion` class. With the exception of the `matter` and `temperature_scale` functions (described below), each of the conversions is performed by a callable `Conversion` class instance defined by the `setup` function at import.

As a callable, `Conversion` instances mimic a function that accepts up to five arguments,

```
>>> conversion_instance(value=1., from_units=None, \
...     to_units=None, exponent=None, inplace=None)
```

The `value` may be a scalar or a numpy array of values to be converted. By default, it is 1, so that if no value is specified, the result will automatically be a conversion factor between the given units.

The `from_units` and `to_units` keywords accept strings identifying the units in the conversion. In the event that one of them is omitted, each `Conversion` instance is initialized to check for an entry establishing an default unit in the `pm.config` system. In the sections that follow, each of the unit classes is listed with the name of its instance, the configuration entry that identifies its default unit, and the default that is configured in PYroMat at installation.

The `exponent` keyword allows for powers of units that are not unity. For example, units of velocity are not explicitly included, but it is still easy to convert from meters per second to miles per hour by

```
>>> import pyromat as pm
>>> speed_ms = 10.
>>> temp = pm.units.length(speed_ms, 'm', 'mile')
>>> speed_mph = pm.units.time(temp, 's', 'hr',
...     exponent=-1)
>>> print(speed_mph)
22.369362920544024
```

Most users will never need the `inplace` keyword, but when it is set to `True` and the `value` is a numpy array, the original values will be overwritten with the result of the conversion.

`Conversion` instances also support item recall, so that the conversion factor for any unit may be obtained simply

```
>>> print(pm.units.volume['m3'] / pm.units.volume['L']
      )
1000.
```

This indicates that there are 1000 liters per cubic meter. Most users will not need to use this capability, but for those that do, it is important to always use ratios of units of the same class. Each item recall returns a scalar value indicating the *relative* size of the respective unit, but relative to what? In the case of volume, each value indicates the unit's value in cubic meters, but that is an arbitrary choice.

For a complete list of the supported units in a unit class, use the `get()` method.

```
>>> pm.units.energy.get()
dict_keys(['J', 'kJ', 'cal', 'kcal', 'BTU_ISO', 'eV',
          'BTU'])
```

4.5 Fundamental Units

Fundamental units are those that have precise definitions based on observable phenomena in nature. They are defined by the BIPM.

4.5.1 Time

Conversion instance:	<code>pm.units.time</code>
pyromat.config entry:	<code>'unit_time'</code>
Default:	<code>'s'</code>

To alter the PYroMat unit for time,

```
>>> import pyromat as pm
>>> pm.config['unit_time'] = 'min'
```

Time is the most fundamental measure defined by the BIPM. “[The second] is defined by taking the fixed numerical value of the caesium frequency $\Delta\nu_{\text{Cs}}$, the unperturbed ground-state hyperfine transition frequency of the caesium-133 atom, to be 9,192,631,770 when expressed in the unit Hz, which is equal to s^{-1} .”[7, p.130] Most of the other fundamental units are constructed in terms of it.

Table 4.2: Time units recognized by PYroMat

Setting	Value	Description
year	31,536,000 s	year
day	86,400 s	day
hr	3,600 s	hour
min	60 s	minute
s	1 s	second
ms	0.001 s	millisecond
us	10^{-6} s	microsecond
ns	10^{-9} s	nanosecond

The other units of time recognized by the PYroMat system are well known, and have simple definitions in terms of the second. It is worth emphasizing that the year and the day recommended by NIST [8] presume precisely 24 hours to a day and 365 days to a year. This ignores the roughly quarter-day annual disagreement between the year and the sidereal year (the time for the Earth to orbit the sun).

Their configuration strings, their values in terms of the second, and their names are listed in Table 4.2.

Time is implemented in the unit conversion system for completeness, but as of PYroMat version 2.1.0, it is not used directly by any of the property methods.

Listing 4.1: Time Conversion Example

```
>>> pm.units.time(30, 's', 'min')
0.5
```

4.5.2 Length

Conversion instance:	<code>pm.units.length</code>
pyromat.config entry:	<code>unit_length</code>
Default:	<code>'m'</code>

To alter the PYroMat unit for length,

```
>>> import pyromat as pm
>>> pm.config['unit_length'] = 'in'
```

“[The meter] is defined by taking the fixed numerical value of the speed of light in vacuum, c , to be 299,792,458 when expressed in the unit m s^{-1} [...]” [7, p.131] All other metric length units are trivial to derive from the meter.

The international yard (and by extension, the inch, foot, and mile) were defined exactly in terms of the meter before the end of the nineteenth century, but their contemporary values were not adopted until the middle of the twentieth century. [4] Now one yard is defined to be precisely 0.914 4 meters, which results in the better known relationship, one inch is precisely 25.4 millimeters.

The meter was once defined to be a fixed ratio of a great circle around the Earth. This made it of great use for navigation, and for the same reason, the nautical mile is still in common use. The meter was one ten millionth of one quarter of a meridian, making the circumference of the Earth precisely 40,000 km by definition [4]. The nautical mile was also primarily used for navigation, but it was set to the distance covered by one arc minute of latitude (of which there are 21,600 in a circle). Therefore, the nautical mile was exactly 40,000,000 m / 21,600 or approximately 1,851.851 85 m. In the middle of the twentieth century, the United States Departments of Commerce and Defense jointly declared the nautical mile to be redefined as precisely 1,852m [9].

The length scales supported by PYroMat and their values in terms of the meter are listed in table 4.3.

Example:

```
>>> pm.units.length(12, 'in', 'm')  
.127
```

4.5.3 Mass and Weight

Conversion instance:	<code>pm.units.mass</code>
pyromat.config entry:	<code>unit_mass</code>
Default:	<code>'kg'</code>

To alter the PYroMat unit for mass,

```
>>> import pyromat as pm
```

Table 4.3: Length units recognized by PYroMat

Setting	Value	Description
km	1,000 m	kilometer
m	1 m	meter
cm	0.01 m	centimeter
mm	0.001 m	millimeter
um	1×10^{-6} m	micrometer
nm	1×10^{-9} m	nanometer
Å	1×10^{-10} m	Angstrom
in	0.025 4 m	inch
ft	0.304 8 m	foot
yd	0.914 4 m	yard
mile	1,609.344 m	statute mile
mi	Alternate of mile	
nmi	1,852 m	nautical mile

```
>>> pm.config['unit_mass'] = 'lb'
```

“[The kilogram] is defined by taking the fixed numerical value of the Planck constant h to be $6.626\,070\,15 \times 10^{-34}$ when expressed in the unit J s, which is equal to $\text{kg m}^2 \text{s}^{-1}$ ” [7, p.131]. Like many of the remaining fundamental units, the kilogram is defined in terms of length and time, making it dependent on their definitions as well.

Definitions for the pound and units derived from it are often confused by conflicting definitions of the term “weight.” For example, in NIST special publications it is possible to find “[...] the weight of a body in a particular reference frame is defined as the force that gives the body an acceleration equal to the local acceleration of free fall in that reference frame” [8, p.23] and “In general usage, the term ‘weight’ nearly always means mass, and this is the meaning given the term in U.S. laws and regulations” [10, p.10]. Unfortunately, to this day, the pound continues to exist with a kind of mass/force duality that is occasionally clarified by the invention of the “pound-mass,” lbm, and the “pound-force,” lbf. The distinction became especially important as the precision of measuring instruments exceeded the consistency of the strength of gravity over the Earth’s surface.

At the time, where the governing bodies were concerned, the earliest precise definitions of the pound in the UK and the US were primarily concerned with the quantity of precious metals in coinage¹. At the time, mass and force seemed virtually interchangeable under the assumption that a mass would always possess the same weight. Especially because there was not internationally recognized standard for the measure of force, this lead countless industries to adopt the pound as a unit of force (including in derived units of pressure like psi), but since 1893, the United States’ definition of the avoirdupois pound was set to a fixed fraction of the kilogram – indisputably a measure of mass. The contemporary value for the avoirdupois pound was established in the middle of the twentieth century, and is precisely 0.453 592 37 kg of mass.[9] From this, the imperial ton (precisely 2,000 lb) and the long ton (precisely 2,240 lb) are derived.

A slug is the unit of mass that has a weight of precisely one pound-force. It is primarily useful as the nearest imperial equivalent to the kilogram – a mass unit that works without adjustment in Newton’s laws. As is discussed in Section 4.6.1, the pound-force is the quantity of force needed to support one pound mass in “standard Earth gravity.” Therefore, 1 lb of mass, m_1 , expressed in slugs, would obey the equation

$$1\text{ lbf} = m_1 g. \quad (4.1)$$

Solving for m_1 with g expressed in feet and seconds, this gives a value of approximately 0.031 080 950 slugs per pound. The `setup()` calculates this factor in terms of kilograms implicitly from the gravity constant, the definition of the pound (mass) and the definition of the foot.

The atomic mass unit (u) or Dalton (Da) is defined as precisely 1/12 the mass of a carbon 12 atom at rest. It is used to describe the tiny masses of molecules, atoms, and sub-atomic particles. Its definition makes a particle’s mass expressed in atomic mass units very nearly identical to its molar mass in grams. Prior to version 3.0.1, `setup()` calculated the value of one u so that 1000 moles of a substance would have precisely the same mass in kilograms, which gave 1 u a mass of

¹This footnote is my method of suppressing the urge to discuss (at length) the development of the troy pound that preceded the avoirdupois pound that is the standard today.

Table 4.4: Mass units recognized by PYroMat

Setting	Value	Description
t	1,000 kg	metric ton or tonne
Mg	1,000 kg	megagram
kg	1 kg	kilogram
g	0.001 kg	gram
mg	1×10^{-6} kg	milligram
ton	907.184 74 kg	(short) ton
kip	453.592 37 kg	kilopound or “kip”
lbm	0.453 592 37 kg	pound-mass
lb	Alternate form of lbm	
oz	0.028 349 523 125 kg	ounce
slug	$\approx 14.593\ 902\ 9$ kg	slug
u	$1.660\ 539\ 068\ 925\ 2 \times 10^{-27}$ kg	atomic mass unit
Da	Alternate form of u	

about $1.660\ 539\ 067\ 17 \times 10^{-27}$ kg. In the time since PYroMat was originally written, official recommendations for this value can be found to vary as measurements of atomic mass have increased in accuracy. As of version 3.0.1, PYroMat has changed to adopt the 2022 CODATA recommendation of $1.660\ 539\ 068\ 925\ 2 \times 10^{-27}$ kg [?, pg. 54], which is only different by about 1 part per billion.

The troy pound and the troy ounce predate the avoirdupois pound as a measure primarily used for quantities of precious metal in coinage [4, p.6]. Today they are rarely used outside of these specialized applications. They are omitted from PYroMat to avoid confusion and because they are not often used in scientific or engineering applications.

Table 4.4 shows the mass units recognized by PYroMat and their values in kilograms. Note that only the atomic mass unit and the slug have been rounded. The rest of the relationships are precise by definition.

Example:

```
>>> pm.units.mass(2, 'lb', 'kg')
0.90718474
```

4.5.4 Molar

Conversion instance:	<code>pm.units.molar</code>
pyromat.config entry:	<code>unit_molar</code>
Default:	<code>'kmol'</code>

To alter the PYroMat unit for molar quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_molar'] = 'lbmol'
```

“One mole contains exactly $6.022\,140\,76 \times 10^{23}$ elementary entities. This number is the fixed numerical value of the Avogadro constant, N_a , when expressed in the unit mol^{-1} and is called the Avogadro number.” [7][p.134]. Because a mole is a unit of counting, it is equally valid to refer to a mole of planets (about right for our observable universe) as it is to refer to a mole of gas.

Historically, the mole was the quantity of a substance with a total mass in grams equal to the mass of the elementary entities expressed in atomic mass units (see Section 4.5.3). This relationship is echoed by more recently invented alternatives such as the kilogram-mole (kmol) or the pound-mole (lbmol), which enjoy the same relationships with their mass-based namesakes.

Neglecting relativistic effects, if the mass of a single molecule is m_0 , a mass, m , of many such molecules must contain

$$N = \frac{m}{m_0} \quad (4.2)$$

molecules.

When $N = N_a$, the mass of the group expressed in grams will very nearly equal the mass of the molecule expressed in atomic mass units. Therefore, the gram-mole is

$$N_g = \frac{1\text{g}}{1\text{u}} \approx N_a. \quad (4.3)$$

Because both the kilogram and the atomic mass unit have precise definitions (see Section 4.5.3), the relationship between the gram-mole and the mass of a mole of particles is merely approximate rather than precise. In fact, it would only be possible to define an integer precisely

respecting this ratio if the masses of the molecules were precisely equal to integer multiples of fundamental particles – close, but not precise. However, because N_a is defined to nine significant figures, the error is on the order of one part per billion.

```
>>> import pyormat as pm
>>> (pm.units.mass['g'] / pm.units.mass['u'] - pm.
    units.const_Na)/pm.units.const_Na
-1.054689657647338e-09
```

The same process may be applied for any other unit of mass, so the quantity required for m to be expressed in kilograms is

$$N_{\text{kg}} = \frac{1\text{kg}}{1\text{u}}. \quad (4.4)$$

When we consider the ratio of kg to g, even the tiny disagreement in the relationship between N_a and the atomic mass unit cancels, so

$$\frac{N_{\text{kg}}}{N_{\text{g}}} = \frac{1\text{kg}}{1\text{g}} = 1000. \quad (4.5)$$

The moles are converted by the same conversion factors as their mass equivalents, so a kilogram-mole contains precisely 1000 times as many elementary particles as the gram-mole. To express a quantity, N , in molar units, its quantity only needs to be divided by the quantity of the corresponding mole type. For example, in kilogram moles,

$$\bar{N} = \frac{N}{N_{\text{kg}}}. \quad (4.6)$$

For describing quantities of a gas, standard (or normal) volumetric units are used so widely that the problematic aspects of their definitions are worth tolerating. A standard (or normal) volume is the quantity of an ideal gas that would occupy that volume at standard conditions. US Customary and imperial units use the word “standard” and metric units use the word “normal,” but the meaning is the same.

Regardless of the mass of the elementary particles, an ideal gas has a consistent number density (concentration) at given conditions,

$$\bar{\rho}_{\text{std}} = \frac{p_{\text{std}}}{R_u T_{\text{std}}}, \quad (4.7)$$

Table 4.5: Molar units recognized by PYroMat. Values with a * are affected by calls to `setup()`.

Setting	Value	Description
<code>kmol</code>	1 kmol	kilogram-mole
<code>mol</code>	0.001 kmol	gram-mole
<code>lbmol</code>	0.453 592 37 kmol	pound-mole
<code>n</code>	$\approx 1.660\,539\,06 \times 10^{-27}$ kmol	count
<code>Nm3*</code>	$\approx 0.044\,615\,033\,4$ kmol	normal cubic meters
<code>Ncum*</code>	Alternate form of <code>Nm3</code>	
<code>NL*</code>	$\approx 44.615\,033\,4 \times 10^{-6}$ kmol	normal liters
<code>Ncc*</code>	$\approx 44.615\,033\,4 \times 10^{-9}$ kmol	normal cubic centimeters
<code>scf*</code>	$\approx 0.001\,263\,357\,06$ kmol	standard cubic feet
<code>sci*</code>	$\approx 0.731\,109\,408 \times 10^{-6}$ kmol	standard cubic inches

given in mole count per unit volume. Obviously, precise and consistent definitions for standard conditions are essential here. By default PYroMat uses $p_{std} = 1.013\,25$ bar and $T_{std} = 273.15$ K. When $\bar{\rho}_{std}$ is expressed in gram-moles per cubic meter, it is approximately 44.615 033 4.

To calculate the number of moles in a standard volume, one need only multiply by the volume in question. So, a normal liter contains .044 615 033 4 moles (when standard temperature and pressure are as above).

Table 4.5 shows the molar units and their values expressed in kilogram-moles. The values marked with * are dependent on the standard conditions provided when `setup()` was last called.

Even though the mole is the BIPM standard for quantity of a substance, PYroMat uses the kmol or kilogram-mole as the default molar unit for self consistency. For example, the molecular weight property methods return mass per molar units. If the mass units were set to kg and the molar units were set to mol, then the molar mass of diatomic nitrogen would be reported as something near 28,000. Setting the two consistently prevents this unusual result.

Example:

```
>>> pm.units.molar(1.5, 'scf', 'kmol')
```

4.5.5 Matter

Function:	<code>pm.units.matter</code>
pyromat.config entry:	<code>unit_matter</code>
Default:	<code>'kg'</code>

To alter the PYroMat unit for matter quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_matter'] = 'lbm'
```

Molar and mass units provide parallel methods for quantifying amounts of matter. Thermodynamic properties can be expressed in either, so PYroMat uses a third class of units, *matter*, which is an amalgamation of all molar and mass units. A unit matter may be any of the units listed in Tables 4.4 and 4.5. Since every molar unit can be related to the kilogram-mole and every mass unit can be related to the kilogram, it is only important that we establish how the kilogram is related to the kilogram-mole.

The ratio between kilograms and kilogram moles is a property of the molecule, and is called its molar or molecular weight, W . It is typically expressed in terms of atomic mass units per molecule, but it is the same in kilograms per kilogram moles. The mass, m of a molar quantity, $\overline{N}$, in kilogram moles then, is

$$m = \overline{N}W. \quad (4.8)$$

Unlike any of the other unit conversions, this one depends on the properties of the substance itself. As a result, it is implemented in a custom function that adds molecular weight (in u per molecule), `mw`, as a mandatory argument in addition to the other standard `Conversion` arguments.

```
>>> matter(value, mw, from_units=None, \
... to_units=None, exponent=None, inplace=False)
```

If the to- and from-units are in the same molar or mass class, then the function merely calls the appropriate `Conversion` instance.

These examples consider a molecule with molecular weight exactly 2.0 u (H_2):

```
>>> import pyromat as pm
>>> pm.units.matter(1, 2, 'kg', 'kmol')
0.5
>>> pm.units.matter(1, 2, 'kg', 'lbmol')
1.1023113109243878
>>> pm.units.matter(1, 2, 'kg', 'lb')
2.2046226218487757
>>> pm.units.mass(1, 'kg', 'lb')
2.2046226218487757
```

4.5.6 Temperature

Conversion instance:	<code>pm.units.temperature</code>
Scale function:	<code>pm.units.temperature_scale</code>
pyromat.config entry:	<code>unit_temperature</code>
Default:	<code>'K'</code>

To alter the PYroMat unit for temperature quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_temperature'] = 'K'
```

This history of temperature as a measure of hot and cold extends back longer than thermodynamics as a rigorous theory. Without knowing what underlying principles caused substances to be hot or cold, a Celsius scale could still be reliably constructed on a thermometer by marking its readings in ice water and boiling water and by dividing the space between into one hundred equal increments.

With the discovery that temperature indicates the mean translational kinetic energy of the molecules of an ideal gas, the kinetic theory of gasses gives the relationship,

$$\frac{1}{2}m_0 \langle u^2 \rangle = \frac{3}{2}kT. \quad (4.9)$$

The coefficient, k , which is now known as Boltzmann's constant, establishes the proportionality between kinetic energy and temperature. This also provides the idea of an *absolute* temperature scale; one in

which zero temperature corresponds to zero energy rather than an arbitrary choice (like the freezing point of water at standard pressure). In this way, Boltzmann’s constant completely determines an absolute temperature scale in terms of the other units.

The BIPM stipulates that “[the kelvin] is defined by taking the fixed numerical value of the Boltzmann constant k to be 1.380649×10^{-23} when expressed in the unit J K^{-1} , which is equal to $\text{kg m}^2 \text{s}^{-2} \text{K}^{-1}$ [...]” [7, p.133] This makes the value for the Boltzmann constant exact by definition.

The realization that temperature is a measure of certain portions of a system’s energy is especially useful in plasma physics and related fields. In these studies, it is useful to express temperature directly as an energy, or electron-volts (eV). The temperature of a substance expressed in electron-volts is the electrical potential that can be built up due to thermal motions of electrons, and is equal to kT/q , when q is the fundamental charge.

The rankine scale is the US customary and imperial equivalent to the kelvin, with one 1.8 R per 1 K. The Celsius scale uses the same increments as the kelvin scale, but its zero is set 273.15 K, the freezing point of water at 1 atmosphere. The Fahrenheit scale uses the same increment as the rankine, but its zero is set so that the freezing point of water at standard pressure occurs at precisely 32°F.

Scales with offsets defy the unit conversion rules implemented by the `Conversion` instance; that all measurements can be converted between the units merely by multiplying by a factor. Obviously, when converting values for a temperature, it is important to handle these offsets correctly, but when considering derivatives involving temperature or other changes in temperature, the offsets must be ignored. As a result, there are two temperature-based unit conversion tools.

The `pm.units.temperature` is a `Conversion` instance and should only be used in cases where changes or derivatives in temperature are being considered (e.g. in specific heat or entropy). In these cases, offsets can be ignored, and the conversion factor rules are observed.

When temperatures are being converted between scales so that the offsets between them must be respected, the `pm.units.temperature_scale` function should be used instead. It is a function that mimics the

Table 4.6: Temperature units recognized by PYroMat

Setting	Value	Description
K	1 K	kelvin
C	1 K	degree Celsius
R	5/9 K	rankine
F	5/9 K	degree Fahrenheit
eV	$\approx 86.173\,332\,6 \times 10^{-6}$ K	electron-volt

Conversion call signature, but that is specially written to handle temperature scales.

```
>>> pm.units.temperature_scale(value, \
...     from_units=None, to_units=None, \
...     inplace=False)
```

Just like the **Conversion** instances, it respects the `unit_temperature` configuration parameter for any unspecified units. However, it makes no sense to refer to a temperature scale with any exponent other than 1, so the exponent parameter is absent.

The temperature scales and their respective differential values in kelvin are listed in Table 4.6. Note that the offsets are not included in this table.

Examples:

```
>>> pm.units.temperature(2, 'C', 'F')
3.6
>>> pm.units.temperature_scale(2, 'C', 'F')
array(35.6)
```

4.6 Derived Units

Derived units are ones that are defined in terms of the fundamental units. They have no fundamental definition in nature, but are instead calculated in terms of the fundamental units.

4.6.1 Force

Conversion instance:	<code>pm.units.force</code>
pyromat.config entry:	<code>unit_force</code>
Default:	<code>'N'</code>

To alter the PYroMat unit for force quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_force'] = 'kgf'
```

Force is a concept that finds rigorous definition in Newton’s laws of motion, so it is fitting that the principle force unit be called the newton. It is the force required to accelerate one kg of mass by one m/s every s [7, p.137], or simply one kg m s⁻².

Force units that are derived from measures of mass (like kilogram-force and pound-force) can be found to be defined in several (roughly equivalent) ways. The unit force may be the force required to accelerate the respective unit mass at one “gee,” or it may be the force required to support the respective unit mass at rest under “Earth’s gravity.” The ambiguity in the definition is obviously centered around a “gee” or Earth’s gravity. In 1901, the BIPM adopted precisely $g = 9.80665 \text{ m s}^{-2}$ as standard Earth gravity[5], and that value has been in common use ever since. Because the force needed to support a mass varies over the surface of a planet that is neither perfectly spherical, nor uniform in density, and that spins with significant speed, the convention of equating weight and mass through local gravity is deeply problematic. For that reason, the value 9.80665 m s^{-2} , regardless of how well it reflects local gravity, has been adopted as the universal relationship between the unit mass and its respective unit force².

These units depend on the value assigned to g in the last call to `setup()` (see Section 4.2 of this chapter).

²In technical communities, it is impossible to operate long without enduring a diatribe on the evils of inches when millimeters are so obviously superior. While not factually incorrect, it is my personal opinion that those emotional discharges are misdirected, so long as kgf remains in use. Some historical excuse may be made for the mass/force duality of the pound, but the kilogram has no such refuge. I include kgf here as a peace offering – from an engineer originally trained in inches and pounds.

Table 4.7: Force units recognized by PYroMat. Values with a * are dependent on the constants defined by `setup()`.

Setting	Value	Description
kN	1,000 N	kilonewton
N	1 N	newton
mN	0.001 N	millinewton
dyne	1×10^{-5} N	dyne
kgf*	9.80664 N	kilogram-force
ton	$\approx 8,896.443$ 23 N	tons
kip	4,448.221 62 N	kip
lbf*	≈ 4.448 221 62 N	pound-force
lb*	Alternate for lbf	
oz*	≈ 0.278 013 851 N	ounce

The dyne is mostly found in older scientific publications still using the centimeter-gram-second (CGS) system of units, and is precisely 1 g cm s^{-2} . It is almost never in modern use, but writings on the theories of gases from the nineteenth century used CGS units commonly. Some users may find it useful to be able to quickly translate between classic texts and our most modern writings³.

Table 4.7 lists the supported force units and their values in Newtons.

4.6.2 Energy

Conversion instance:	<code>pm.units.energy</code>
pyromat.config entry:	<code>unit_energy</code>
Default:	<code>'kJ'</code>

To alter the PYroMat unit for energy quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_energy'] = 'kcal'
```

The Joule is the SI measure of energy, and it is defined as one N m, or one $\text{kg m}^2 \text{s}^{-2}$ [7, p.137]. The calorie is of historical importance to the scientific community, but its use has fallen out in favor of the Joule.

³I certainly do.

The British thermal unit (BTU) is still broadly used in some industries (especially heating and refrigeration), but few authoritative definitions for unit systems recognize either in their contemporary standards.

The calorie has a number of alternate definitions that has made the unit somewhat problematic. One set of definitions is the energy required to raise a gram of water one degree Celsius at different “standard” conditions. Another is 1/100 of the energy required to raise a gram of water from its ice point to boiling at atmospheric pressure. In 1956, the Fifth International Conference on the Properties of Steam set the “international table calorie” to be precisely 4.1868 J. The calorie in broadest contemporary use is the thermochemical calorie, which seems to be universally understood to be precisely 4.184 J [8], though an original source for that definition is difficult to find. Because it is the unit adopted by the International Unions of Pure and Applied Chemistry and Physics (IUPAC and IUPAP) [11] PYroMat also adopts the thermochemical calorie.

Whatever definition is used for the calorie, the BTU may be derived from it by adjusting the quantity of water to be one pound and the temperature rise to be one degree Fahrenheit. The BTU so derived from the thermochemical calorie is approximately 1,054.350 26 J.

It is important to emphasize that it is difficult to find contemporary authorities that certify the BTU or calorie for commercial use. The ISO standard that historically defined the calorie (ISO 31-4) was withdrawn and superseded by ISO 80000-5, which makes no mention of the calorie or the BTU. NIST’s special publication number 811 from which the conversion is lifted specifically lists the calorie as an “unacceptable unit.”

The electron-volt is the energy required to move a single electron through a one volt potential. Often used as a fundamental unit in atomic and sub-atomic physics, it is expressed in Joules as 1V times the fundamental charge, q , so `setup()` simply calculates its value in J.

The erg is the centimeter-gram-second (CGS) system unit of energy. It corresponds to one $\text{g cm}^2 \text{s}^{-2}$ or 10^{-7} J.

Table 4.8: Energy units recognized by PYroMat

Setting	Value	Description
MJ	1×10^6 J	kilojoule
kJ	1,000 J	kilojoule
J	1 J	joule
cal	4.184 J	calorie
kcal	4,184 J	kilocalorie
BTU	$\approx 1,054.350$ 26 J	British thermal unit
erg	1×10^{-7} J	erg
eV	1.602 176 634 $\times 10^{-19}$ J	electron-volt

4.6.3 Pressure

Conversion instance:	<code>pm.units.pressure</code>
pyromat.config entry:	<code>unit_pressure</code>
Default:	<code>'bar'</code>

To alter the PYroMat unit for pressure quantities,

```
>>> import pyromat as pm
>>> pm.config['unit_pressure'] = 'Pa'
```

Pressure is a force exerted per unit area. In addition to its typical use describing fluid forces on surfaces, pressure units are also used to describe stresses and surface loads in solids. The SI unit for pressure is the pascal, which is defined as one $\text{kg m}^{-1} \text{s}^{-2}$ [7, p.137]. The bar is precisely 100,000 Pa, and is in broad use thanks to its proximity to atmospheric pressure.

The “standard atmosphere” was adopted by the (CGPM) in 1954 to be precisely 101,325 newtons per square meter [6]. It is intended to be indicative of a global mean barometric pressure (adjusted to sea level). This definition is in broad use both scientifically [11] and commercially [8] as value of one atm. The standard atmosphere can be adjusted using the `pstd` parameter of `setup` (see Section 4.2 of this chapter).

Liquid column units for pressure are defined as the increase in pressure observed beneath a column of liquid of some height. Like bourdon tube gauges, these pressures are measured relative to the ambient, and are also sometimes called gauge pressure. Provided sufficient measures

are taken to avoid the impacts of meniscus, the liquid column pressure, p , may be calculated for a liquid with mass density, ρ , under a uniform gravitational acceleration, g , with a column height, h ,

$$p = \rho gh. \quad (4.10)$$

For measures of high precision, it is obviously necessary to define a standard gravity and standard densities for the column fluids.

By default, PYroMat uses the acceleration of free fall in Earth gravity, g , to be precisely 9.80665 m s^{-2} . This value was internationally adopted in 1901 by the third General Conference on Weights and Measures (CGPM)[5] and is still in broad use as a “standard” value [10, p.45] [8, p.5]. However, it should be emphasized that actual pressures produced by liquid columns will vary significantly with altitude, proximity to dense geological formations, and especially latitude.

It is common to calculate the mmH₂O with a convenient and easy-to-remember $1,000 \text{ kg m}^{-3}$ water density. The so-called 4°C mmH₂O is based on a water density of $999.972 \text{ kg m}^{-3}$, which is close enough to the convention for the vast majority of applications. Actual water density in the “ambient” range 20°C to 25°C can be as low as 997 kg m^{-3} , so practical realizations of this unit in a laboratory setting are unlikely to be more accurate than 0.3% – still better than inexpensive commercial gauges. By default PYroMat uses the 4°C mmH₂O, but users may change this convention by altering the `dh2o` parameter in the `setup` function (see Section 4.2 of this chapter). When in doubt, users should calculate their own water column pressures in Pa or bar based on the known conditions in the lab.

There is also a variety of conventional values established around the mmHg column. The definition most commonly used is the 0°C mmHg, which is adopted by PYroMat by default. A density of $13,595.1 \text{ kg m}^{-3}$ for mercury with the above value for g results in a conversion factor identical to the value officially adopted by NIST [8, p.52] to the precision given.

Because they are identical to practical precision, the Torr and the mmHg are often treated as identical units, but there is a difference in definition. The Torr is defined so that 1 atm is precisely 760 Torr. As seen in Table 4.9, they are not quite identical, but they are so close, few

Table 4.9: Pressure units recognized by PYroMat. Values with a * are dependent on the constants defined by `setup()`.

Setting	Value	Description
Pa	1 Pa	pascal
kPa	1,000 Pa	kilopascal
MPa	1,000,000 Pa	megapascal
bar	100,000 Pa	bar
atm*	101,325 Pa	atmosphere
Torr*	≈ 133.322 368 Pa	Torr
mmHg*	≈ 133.322 387 Pa	mm mercury column
inHg*	$\approx 3,386.388$ 64 Pa	inches mercury column
mmH2O*	≈ 9.806 375 41 Pa	mm water column
inH2O*	≈ 249.081 936 Pa	inches water column
psi	$\approx 6,894.757$ 29 Pa	pounds per square inch
ksi	$\approx 6,894,757.29$ Pa	kips per square inch
psf	≈ 47.880 259 0 Pa	pounds per square foot

practical measurements will ever mandate a distinction. However, the distinction means that the Torr does not depend on the choice for the density of mercury or gravity. Instead, it only depends on the definition of the standard atmosphere.

As mentioned above in the discussion on water column, “gauge” measurements of pressure are often made relative to the ambient conditions. These kinds of pressures should never be used in thermodynamic calculations, so to avoid confusion, PYroMat only works in “absolute” pressure, which is actual force per actual area. To convert back-and-forth between absolute pressure and the gauge measurements that are easier in the laboratory, the `units` module includes the `abs_to_gauge()` and `gauge_to_abs()` functions. See their in-line documentation for more information.

4.6.4 Volume

Conversion instance:	<code>pm.units.volume</code>
pyromat.config entry:	<code>unit_volume</code>
Default:	<code>'m3'</code>

To alter the PYroMat unit for volumetric quantities,

```
>>> import pyormat as pm
>>> pm.config['unit_volume'] = 'gal'
```

Volume is the measure of the size of a region in three-dimensional space. SI volumetric units are entirely derived from cubes of the linear units, with the liter being equivalent to 0.001 m^3 . These are quite simply defined as the region of space occupied by a cube having the corresponding unit length on each of its sides.

Historically the English units for fluid ounce, pint, quart, and gallon were based on the volume occupied by specific weights of water. By contrast, the English wine gallon was first implemented in 1707 as precisely 231 cubic inches, and though the UK abandoned it in favor of the imperial gallon (the volume occupied by 10 lb of water) in 1824, the wine gallon was adopted as the official gallon by the United States Treasury Department in 1832 [4]. Today, the US gallon is still defined as precisely 231 cubic inches (implicitly deriving it from the meter), and the imperial gallon is precisely 4.546 09 liters (also derived from the meter). Though the source of these measures has converged to the meter, the world is left with these divergent vestiges of early attempts to construct rigorously defined units of volume.

The “customary” volumetric measurements quart (one quarter gallon), pint (one half quart), cup (one quarter of a pint), the fluid ounce (one eighth of a cup), tablespoon (one half of a fluid ounce), and teaspoon (one third of a tablespoon), all follow from the gallon⁴. It should be emphasized that these are “fluid” volumes, and are distinct from the system of “dry” volumes in historical commercial use in agriculture. Their name stems from their historical definitions. For example, the “fluid ounce” was once known as the volume occupied by one ounce of distilled water, and its modern definition relating it to the gallon is within 4% of its historical definition.

Table 4.10 shows the volumetric units recognized by PYroMat.

⁴There are parallel definitions for imperial volumes derived from the imperial gallon. Perhaps I am betraying some chauvinism by including the US customary and not the UK equivalents. Personally, I sleep well with the assurance that I am probably the only user to calculate the density in ounces per ounce – a feat that should not, on its face, be possible.

Table 4.10: Volumetric units recognized by PYroMat

Setting	Value	Description
cum	1 cum	cubic meter
m3	Alternate for cum	
cc	$\times 10^{-6}$ cum	cubic centimeter
cm3	Alternate for cc	
cumm	1×10^{-9} cum	cubic millimeter
mm3	Alternate for cumm	
L	.001 cum	liter
mL	1×10^{-6} cum	milliliter
uL	1×10^{-9} cum	microliter
cuin	$0.163\ 870\ 64 \times 10^{-5}$ cum	cubic inch
in3	Alternate for cuin	
cuft	$0.028\ 316\ 846\ 591$ cum	cubic foot
ft3	Alternate for cuft	
USgal	$0.003\ 785\ 411\ 783$ cum	US gallon
gal	Alternate for USgal	
qt	$0.946\ 352\ 946 \times 10^{-3}$ cum	customary quart
pt	$0.473\ 176\ 473 \times 10^{-3}$ cum	customary pint
cup	$\approx 0.236\ 588\ 237 \times 10^{-3}$ cum	customary cup
floz	$\approx 29.5735295624 \times 10^{-6}$ cum	customary fluid oz
oz	Alternate for floz	
tbsp	$\approx 14.786\ 764\ 8 \times 10^{-6}$ cum	customary tablespoon
tsp	$\approx 4.928\ 921\ 59 \times 10^{-6}$ cum	customary teaspoon
UKgal	$0.004\ 546\ 09$ cum	imperial gallon

Chapter 5

The PYroMat modules

The core of PYroMat’s functionality is split across four modules; **reg**, **dat**, **utilty**, and **units**. The **units** module already has its own chapter (Chapter 4), so this chapter is devoted to a description of the back-end and how it retrieves the models and their data.

When the PYroMat package is first imported, the load process is completed in three steps: (1) configuration, (2) registry, and (3) data. In the configuration stage, PYroMat loads configuration files, which are described in detail in Chapter 3. In the registry stage, PYroMat searches for Python code that defines the classes that handle the substance models. Finally, in the data stage PYroMat searches for the ***.hpd** files that define the substance data.

5.1 The class registry module, **reg**

The **reg** module only has three members of interest to a user – the **registry** dictionary, the **regload()** function, and the **PYroMatModel** class. As a module, **reg** is responsible for maintaining the **registry** dictionary, which contains all of the classes that provide the substance models. Each entry of this dictionary is a child class of the **PYroMatModel** class. When data files are loaded (see the **dat** module below), each substance data set is required to identify its class with a string. That string is used as the key to the **registry** dictionary, which is expected to provide the correct class for handling that substance’s data. As of version

Table 5.1: Default installation classes available as of version 3.0.1.

Class	Section	Description
'ig'	6.2.1	Ideal gas with the Shomate c_p equation
'ig2'	6.2.2	Ideal gas with the NASA c_p polynomials
'igmix'	6.2.3	Static ideal gas mixtures
'mp1'	??	Multi-phase with separate saturation line models
'mp2'	??	Multi-phase built on table lookups

3.0.1, the default installation includes the classes listed in Table 5.1.

When PYroMat is first imported, it calls `reg.regload()` with no arguments, which causes it to discover a list of possible registry directories from the `config['reg_dir']` entry. If the `config['reg_recursive']` is set to `True`, the `reg.regload()` function will also descend into sub-directories. Calling `reg.regload()` at any time causes it to repeat this process. See the in-line documentation for how `reg.regload()` arguments can be used to override configuration entries when manually repeating the registry load process from the command line.

The `reg.regload()` function calls `exec()` on all files with names ending in `.py` and without a leading `.` or `_`. Each class that is found to be a child of `PYroMatModel` is added to the `registry` dictionary with its name as the key. A PYroMat installation should *never* be configured in such a way that one user can load registry files from a different user, as it would constitute a serious security concern. Instead, shared registry files should be stored in administrator-controlled directories with read-only permissions, and users should be prohibited from saving their own files there.

This concern is the reason multiple registry directories can be configured. For example, on a Linux system, an administrator might configure a system-wide installation of PYroMat so that users can create their own private registries in their home directories by adding the line to a configuration file.

```
reg_dir = '~/pyromat/reg'
```

See Chapter 3 for more information on configuration.

It is also important to understand that calling `regload()` does *not* cause any instances loaded by the `dat` module to be associated with the newly loaded classes. It will be important to also make a call to the `dat.load()` function to force reloading the substance model instances. See below for more information.

5.2 The data module, `dat`

The `dat` module is responsible for maintaining the `dat.data` dictionary of all available substance models. It includes tools to discover, load, and manipulate the model data. The data dictionary is where `get()` finds the substance instances requested by the user.

5.2.1 The `load()` function

The `dat.load()` function is the most important of `dat`'s functions. It is responsible for loading all of the substance data and creating the class instances with which users will interact. When `PYroMat` is first imported, `dat.load()` is called with no arguments, which prompts it to discover a list of possible data file directories from `config['dat_dir']`. Every file with a `.hpd` file extension is loaded and used to initialize an appropriate class instance. Each instance is added to the `dat.data` dictionary with its “id” as the keyword string. See section 5.2.2 for more information.

When it is called with a directory or a path to a specific file, `dat.load()` will only open the contents of that directory or that file. This is useful for manually adding or re-loading files under development that are not yet in the data directories listed in `config`.

The `dat.load()` function also has an optional keyword, `check`, that prompts a data file integrity check when set to `True`. When run in check mode, `dat.load()` returns a dictionary with six entries describing the results of the load process. If a user has changed the data contained in the `dat.data` dictionary, added a new element, or deleted an existing

element, it will be discovered by comparing the current data dictionary against a repeated load process.

```
>>> import pyromat as pm
>>> # This does NOT affect pm.dat.data
>>> result = pm.load(check=True)
>>> result['changed']      # modified substances
>>> result['added']        # new substances
>>> result['removed']      # substances removed
>>> result['redundant']    # redundant files
>>> result['suppressed']   # excluded files
>>> result['data']         # new data dict
```

The **changed**, **added**, and **removed** elements of the **result** dict are lists of id strings for the species that are affected. This lets developers carefully inspect the impermanent changes they have made to the data during a command line session.

The **redundant** dictionary member is a dictionary of substance ids with more than one definition found in the bank of ***.hpd** files. This happens often when multiple data sources have their own contradictory models for the same substances. The keys to the **redundant** dictionary are the substance ids, and the values are lists of paths to the multiple files that specified the same substance id.

The **suppressed** dictionary member is a list of paths to files in the search path with a ***.hpd** file extension. These files that have been removed from the load process (probably to resolve a redundancy), but that the user may want to be able to locate to re-activate them.

See section [5.2.3](#) for advanced tools for working with this information.

5.2.2 Data files

Files are in the JavaScript Object Notation (JSON) data format, which only requires an ASCII character set, so any UTF-N extension will do. The file should define a dictionary with keyword names that describe the essential data elements of the substance. The sections in Chapters [6](#) and [7](#) describe the various substance classes and their required data elements, but there are also certain basic keywords used by PYroMat itself.

Table 5.2: Required data keywords in all PYroMat files

Keyword	Type	Description
'id'	str	The string used by the <code>get</code> function to recognize the species and its collection. It should be in the format <code><collection>.<formula></code>
'class'	str	The string name of the class from the <code>reg</code> module that will be used to interpret the data.
'doc'	str	A long string that is used to describe the data and cite its original source.
'atom'	dict	A dictionary with element symbols for keywords, and the numerical quantity per molecule of each for values. This is used by <code>info</code> to search for species by contents. It can also be useful for calculating the molecular weights of isotopes.

5.2.3 Tools for working with data files

For users who want to develop their own models or change existing models, the `dat` module includes tools to make that easier.

The `dat.clear()` function empties the current `dat.data` dictionary. Since successive calls to `dat.load()` merely overwrite or add to the dictionary, if users really want to start from scratch, they should call `dat.clear()` before running `dat.load()`.

The `dat.new()` function creates a new entry in the `dat.data` dictionary from a dictionary like one that might be loaded directly from a `*.hpd` file. The intent is to allow users to write scripts that build their own data dictionaries from scratch, test them in PYroMat and only save them permanently when they have been tested.

The `dat.updatefiles()` function runs `dat.load()` in check mode and allows the user to resolve differences between the current members of the `dat.data` dictionary and the currently available data files. When run verbosely, the user will be prompted with a choice of actions for each file. When run with the `verbose=False`, it will automatically operate on all findings. See the inline documentation for more information.

5.3 The utility module, utility

The `utility` module is a container for a number of back-end code to which users should not need direct access. For example, it is where all of the PYroMat error types are defined, and there are a number of back-end helper functions.

5.3.1 PYroMat error types

There are special error types defined to be unique to PYroMat. These are intended to help users write their own scripts with meaningful exceptions when things go wrong. Table 5.3 shows all of the error types and describes their use.

Table 5.3: PYroMat error types

Error Type	Description
PMAnalysisError	A numerical routine has failed. This probably means that an iteration has not converged or a calculation gave an unexpected illegal result. These are very unusual.
PMDataError	A substance data set is corrupt. This usually appears when a required data element is not defined or the data are incorrect data types. Not all of these errors are handled gracefully, so the property methods may simply crash with a basic Python exception.
PMFileError	There was an error working with a file. This is probably because the user does not have permission to work one of the *.hpd files.
PMParamError	There was a problem with an argument passed to a method or function. This is common when the user specifies a state out of the model's range or some otherwise invalid combination of properties.

5.3.2 Redundancy tools

The `utility.revive_file()` and `utility.suppress_file()` automatically add or remove a ‘ ’ at the end of a file extension to add it to or remove it from the load process. The `utility.red_repair()` is an automatic interactive file redundancy repair function that calls these if necessary. Interested users can read their inline documentation for more information.

5.3.3 Other tools

The `utility` module is also where messaging and error handling helper functions, `print_lines()`, `print_warning()`, and `print_error()` reside. These automatically add line breaks and headers in appropriate locations so all terminal communication will be in a standard format.

Finally, `load_file()` is a helper function that does back-end work for the `dat.load()` function. It is a wrapper function that loads and checks the `json` code from data files, and it returns the dictionary they create. The `dat.load()` is responsible for passing them to the appropriate class initializer and adding them to the data dictionary.

Chapter 6

Ideal Gases

The ideal gas is one in which bombarding molecules do not exert significant forces on each other except in collisions. Under these conditions, the distance between molecules (the gas's density) is unimportant for determining thermodynamic properties. Only the gas's temperature (the speed of the molecules) is important. It is worth emphasizing that transport properties (like conductivity and diffusivity) are still impacted by density.

There are two classes in PYroMat that implement the two most widely used models: the `ig` class manages the Shomate equation of state, and `ig2` manages the so-called NASA polynomials equation of state. In either case, constant-pressure specific heat, c_p , is constructed purely as a function of temperature. Here, we re-develop the thermodynamic properties from first principles to demonstrate how c_p is sufficient to calculate them.

6.1 Properties of ideal gases

6.1.1 Ideal gas law

When they are spread so sparsely that forces between molecules are small, ideal gases are well described by the relations

$$p = nkT \quad (6.1a)$$

$$p = \rho RT \quad (6.1b)$$

$$p = \bar{\rho} R_u T. \quad (6.1c)$$

Here, p and T are the pressure and temperature of the gas. The densities are expressed in number density, n , mass density, ρ , and molar density, $\bar{\rho}$. It is clear, then, that the Boltzmann constant, k , and the ideal gas constants, R and R_u , are related,

$$kN_a = RW = R_u. \quad (6.2)$$

The Boltzmann constant is exact by definition thanks to the contemporary definition of temperature (see Section 4.5.6), so this relationship provides the authoritative definitions for the gas constants, R , and R_u .

The ideal gas law is empirical in its origins, originally formulated as an amalgamation of the independent laws of Gay-Lussac, Charles, and Boyle. The kinetic theory of gases eventually provided an independent formulation based entirely in Newton's laws. Given its importance to the study of matter and its deeply intuitive nature, it is surprising that few introductory text on thermodynamics gives the subject any treatment. It is worth a brief summary here.

First, let us take that a gas is comprised of molecules that translate freely in space. They may be imagined to follow straight paths of constant velocity unless they collide with a containing wall or another molecule.

Pressure, then, is due to a series of impacts on a surface so rapid that they appear to be continuous. Pressure only has meaning as a property when the density of molecules is sufficiently high that their individual impacts are imperceptible. That means that pressure is a kind of average force determined by a series of many individual random impacts. It is possible to quantify its magnitude by describing the

individual impacts through Newtonian mechanics. If the forces due to a series of impacts in time is $F(t)$, a surface with area, A , will experience a pressure, p , based on the rate of increase of total impulse,

$$p \equiv \frac{1}{At} \int_0^t F(\tau) d\tau. \quad (6.3)$$

This formulation should have a well defined limit when t is larger than period between individual impacts and shorter than the period over which the local properties change.

In a Cartesian coordinate system with z normal to a surface and positive *into* the surface, the velocity of any single molecule will have three components, $\vec{u}_1 = u_x \hat{i} + u_y \hat{j} + u_z \hat{k}$. After an elastic collision with the surface, the velocity will be $\vec{u}_2 = u_x \hat{i} + u_y \hat{j} - u_z \hat{k}$.

The impulse imparted to the surface by one such collision will be

$$\int_0^t F d\tau = 2mu_z, \quad (6.4)$$

when m is the mass of the molecule. When collisions occur due to many identical molecules, their impulses accumulate

$$\begin{aligned} \int_0^t F d\tau &= 2m(u_{z,1} + u_{z,2} + u_{z,3} + \dots) \\ &= 2m \sum_i u_{z,i} \end{aligned} \quad (6.5)$$

Note that for a molecule to collide with the surface, its z -component velocity before the collision, u_z , must be positive. This results in a purely positive force (into the surface). Ideal gas molecules experiencing elastic collisions have no mechanism to pull on the surface.

The next step to predict the force on the surface requires a prediction for the rate at which collisions occur. Because faster moving molecules will travel more distance in the time interval, their collisions will be more numerous. Let the number density of molecules with z -component velocity between u_z and $u_z + du_z$ be $n'(u_z)du_z$. Here, n' is a population density function for a population of molecules based on one component of their velocity. So, n' has units number per volume

per velocity, and its integral over all velocities is precisely n , the total number density of molecules.

The number of molecules with a certain velocity that will strike an area, A , in time t will be determined by the size of the volume that can be traversed by molecules traveling at that velocity. Molecules traveling with z -component velocity u_z will traverse a length $u_z t$ in the time interval. So, the total volume occupied by molecules with that velocity that will strike the surface is Atu_z . The total number of collisions is $Atu_z n'(u_z) du_z$. So, the impulse becomes

$$\begin{aligned} \int_0^t F d\tau &= 2m \int_0^\infty Atu_z^2 n'(u_z) du_z \\ &= mAt \int_{-\infty}^\infty u_z^2 n'(u_z) du_z \end{aligned} \quad (6.6)$$

Note that only half of the gas's population will have a velocity component in the positive direction, and the other half will not cause pressure. Therefore, the 2 coefficient is canceled when the bounds of the integral are extended to include all molecules.

The integral is merely a calculation of the average value of u_z^2 over the population of molecules, and may be simplified to $n\langle u_z^2 \rangle$ when n is simply the total number density of the molecules and $\langle u_z^2 \rangle$ is the mean square of z -component velocity. Finally, we have obtained an expression for pressure in terms of the outward velocity component,

$$p = mn\langle u_z^2 \rangle. \quad (6.7)$$

The last simplification to this relationship comes when we assert that gas velocity statistics are isotropic; velocity statistics are the same in all directions and do not depend on the coordinate system. That implies that $\langle u_x^2 \rangle = \langle u_y^2 \rangle = \langle u_z^2 \rangle$, so

$$\langle \vec{u}^2 \rangle = \langle u_x^2 + u_y^2 + u_z^2 \rangle = 3\langle u_z^2 \rangle. \quad (6.8)$$

Therefore, pressure is proportional to the mean square of molecular translational velocity,

$$p = \frac{1}{3} mn\langle u^2 \rangle. \quad (6.9)$$

When this relationship is substituted into the ideal gas law above, it provides a purely mechanical interpretation for temperature as well,

$$\left\langle \frac{1}{2}mu^2 \right\rangle = \frac{3}{2}kT. \quad (6.10)$$

Temperature is a measure of the average *thermal* energy of the gas. Higher temperature means higher kinetic energy, and the Boltzmann constant relates the two.

This development was quick and it neglects to address mixtures of molecules of different masses. However, the same approach may be used quite intuitively to show that Dalton's law for partial pressures also follows from these basic assumptions.

6.1.2 Internal energy

What happens to heat and work as they are added to an ideal gas depends on the structure of the gas molecule. If there are chemical, atomic, or phase changes, the species can be modeled as vanishing and being replaced by a new substance with its own properties. The ideal property models are, therefore, concerned with how energy is stored in a molecule that is neither changing its fundamental structure nor changing phase. This remaining energy will be exhibited entirely as the thermal energy in (6.10) and internal vibration, rotation, or electrical motions inside the molecule, for which we have not yet accounted.

Perfect gases are ideal gases, but not all ideal gases are perfect. The molecules of an ideal gas collide elastically and have no further interactions with their surroundings. However, the molecules of a perfect gas are further assumed to neither spin nor vibrate. As a result, any molecule more complicated than a single atom does not form a perfect gas.

When a gas composed of a monoatomic molecule like argon is heated (without atomic, chemical, or phase changes) the energy can only be stored in the thermal translational energy described in (6.10). A sample of N molecules of such a gas would have thermal energy $N\frac{3}{2}kT$, so the

energy of one mole or one gram is

$$\bar{e} - \bar{e}_0 = N_a \frac{3}{2} kT = \frac{3}{2} R_u T \quad (6.11)$$

$$e - e_0 = \frac{N_a}{W} kT = \frac{3}{2} RT. \quad (6.12)$$

See section 4.5.4 for definitions of the molar mass, W , and the Avogadro number, N_a . Here, e_0 is the internal energy due to the atomic, chemical or phase changes we have not yet considered. Note that the gas constants, R_u and R , appear naturally in terms of the Boltzmann constant, k .

Non-perfect ideal gases are made of more complicated molecules, and they can store energy in more complicated ways; they vibrate, they rotate (spin), and they have means of storing energy electrically. They do this because the can; they have more degrees of freedom.

The 3 in (6.12) first appeared in (6.8) because all ideal gases are free to translate in three directions. These three thermal degrees of freedom are the only ones that contribute to measurements of temperature, but complex molecules have more degrees of freedom that do not involve translation through space of the molecule's bulk. If these are included in the internal energy as well, a molecule that is free to vibrate, rotate, and translate in n_f degrees of freedom will have an internal energy

$$e - e_0 = \frac{N_{kg}}{W} \frac{n_f}{2} kT = \frac{n_f}{2} RT. \quad (6.13)$$

The intuitive assumption might be that n_f is constant and a property of each molecule. This assumption implicitly asserts that all modes of vibration, rotation, and translation have an equal fraction of the total energy so it is called the *equipartition* assumption. It results in an elegant formulation, but this is tragically a case of profound theoretical beauty that completely fails to predict the actual behavior of molecules in gases at low temperatures.

Instead, it is necessary to stipulate that n_f is a statistical quantity that can change with the state. It can be thought of as the number of “active” degrees of freedom of each molecule. Its minimum is 3, but it can increase to a maximum that will depend on the complexity of the molecule and the energy of the collisions it endures. For example,

at very low temperatures, molecules may not vibrate much; instead they may just bang around and spin like rigid bodies (see Figure 6.1 below). However, at higher temperatures, there may be enough energy to excite vibrations inside the molecules.

Fortunately, since all ideal gases (perfect or not) are presumed to collide elastically, the equilibrium distribution of energy should depend on neither the rate of collisions nor the distance separating the molecules. Obviously, this assumption will break when molecules are packed in closely with one another (like in a liquid), but when it holds, the active degrees of freedom is only be a function of temperature.

$$e(T) - e_0 = \frac{n_f(T)}{2} RT \quad (6.14)$$

Better insights into the behavior of n_f will be seen when we examine specific heats below.

6.1.3 Enthalpy

As established above, we will construct all of the properties of the ideal gas in terms of its specific heat, but in order to derive reliable relations for the specific heats, it is important to first re-examine enthalpy.

Recall from Section 1.1.7 that the definition for enthalpy is $e + pv$. For an ideal gas, pv may be substituted with RT , so an ideal gas will have enthalpy

$$h(T) = e(T) + RT. \quad (6.15)$$

For all ideal gases, enthalpy and internal energy are only functions of temperature, and they are related by RT .

6.1.4 Specific heats

Recall from Section 1.1.11 that the constant-volume and constant-pressure specific heats are

$$c_v = \left(\frac{\partial e}{\partial T} \right)_v$$

$$c_p = \left(\frac{\partial h}{\partial T} \right)_p$$

Because both internal energy and enthalpy are only functions of temperature, this relationship is relatively simple,

$$c_v = \frac{n_f(T)}{2} R \left(1 + T \frac{n'_f(T)}{n_f(T)} \right) \approx \frac{n_f(T)}{2} R \quad (6.16)$$

$$c_p = c_v + R \quad (6.17)$$

The approximation in (6.16) only holds when $n_f(T)$ is very nearly constant.

Figure 6.1 shows the quantities $2c_v/R$ for six gases with increasing molecular complexities. Monoatomic He and Ar are perfect gas models, so they behave precisely as predicted with exactly three degrees of freedom. Meanwhile, all four polyatomic molecules start with quasi-rigid motion at low temperatures before adopting higher degrees of freedom at high temperatures.

For H_2O , rigid motion means six degrees of freedom: three coordinates of translation and three axes of rotation. For diatomic molecules like O_2 and N_2 , however, their symmetry robs them of one of their axes of rotation, so they only exhibit five degrees of freedom at low temperatures. Since CO_2 is also an axisymmetric molecule, it can be seen asymptotically approaching 5 at low temperatures as well.

6.1.5 Entropy and enthalpy revisited

Recall from Section 1.1.8 that the entropy of a pure substance changes like

$$Tds = dh - vdp.$$

For the ideal gas, this can be simplified by substituting RT/p for v and integrating to obtain

$$s - s_0 = \int c_p(T) \frac{dT}{T} - R \ln \left(\frac{p}{p^\circ} \right). \quad (6.18)$$

when p° is a reference pressure. For a perfect gas,

$$s - s_0 = c_p \ln \left(\frac{T}{T_0} \right) - R \ln \left(\frac{p}{p^\circ} \right). \quad (6.19)$$

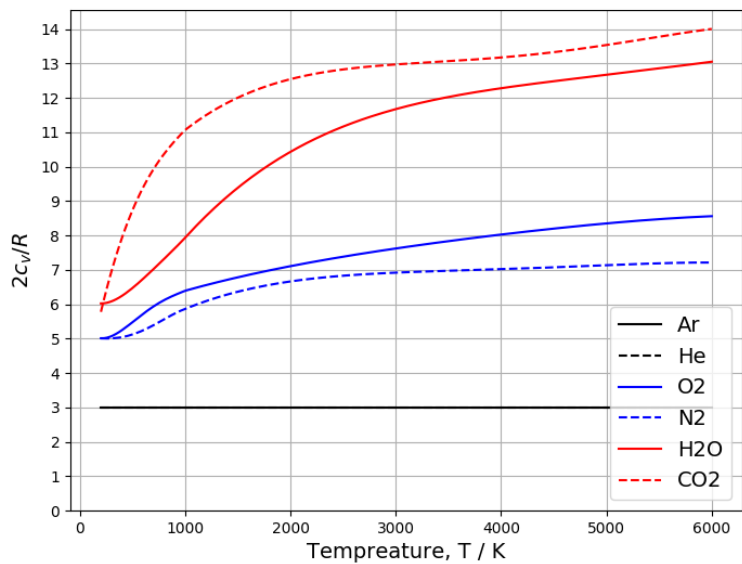


Figure 6.1: Approximate mean active degrees of freedom in selected ideal gas models

when T_0 is a reference temperature.

Evaluating the pressure portion of (6.19) is straightforward, but it will be seen below that the temperature dependence is more nuanced. For that reason, it is often treated alone,

$$s^\circ(T) = s_0 + \int c_p(T) \frac{dT}{T}, \quad (6.20)$$

where s° is the entropy at the reference pressure, p° .

An identical approach can be taken for enthalpy, but with an even simpler result.

$$h - h_0 = \int c_p(T) dT \quad (6.21)$$

For a perfect gas,

$$h - h_0 = c_p (T - T_0). \quad (6.22)$$

Enthalpy is sometimes tabulated as $h^\circ(T)$, but it is an inherent assumption associated with ideal gases, that $h(T, p) = h^\circ(T)$.

In the NIST-JANAF tables, the specific heat is calculated theoretically from the molecular and atomic structure of each atom, but specific heat is also readily validated by calorimetry. For the elements in their most stable state (e.g. diatomic hydrogen, monoatomic argon, ...), h_0 is fixed at zero at the reference conditions. The enthalpy of compounds at the reference conditions are fixed empirically to match calorimetry experiments so that the enthalpy of formation (see Section 6.1.6) is correct [1, pg 13].

The integration factor for entropy requires substantially more nuance. The introduction to the 1998 NIST-JANAF tables describes a procedure for using equilibrium data from “small groups of interrelated substances” to relate the Gibbs energy, and the entropy with it [1, pg 14 par 3]. The authors even recommend adjusting those values based on the application in detailed reaction modeling.

Since simultaneous solution has not been attempted for all substances it is necessary to fix, simultaneously when possible, certain key values for common substances. These

are then used in the usual sequential way to help fix other substances. JANAF policy is to adopt the key values recommended by the CODATA Task Group for Key Values in Chemical Thermodynamics. Exceptions to this policy only occur when additional recent data are available.

Because of the many revisions of these tables, perfect interconsistency is not always attained. However, any inconsistency of the tables is a prime concern and is cause for revision if the effects are of the same order of magnitude as the stated uncertainties. [1, pg 14]

6.1.6 Formation properties

When a substance is formed either by nuclear, chemical or phase change, energy and entropy are nearly always released or consumed. These are accounted for by the *enthalpy of formation* and *entropy of formation*. The former is the energy required to form a substance from its atomic constituents starting in their reference state, so exothermic reactions have negative enthalpies of formation. The ideal gas data do not consider nuclear reactions; only chemical reactions and phase changes.

One might imagine a reactor with a mixture of reactants flowing in and a mixture of products flowing out. In the simplest case, we should imagine the products to be made entirely of the substance we wish to study, so the reactants will be only those that are absolutely necessary for forming it and in the correct proportions. For example,



Note that the reference states for hydrogen and oxygen are *not* monoatomic hydrogen or oxygen. Instead, they are their most stable state at the reference conditions, 298.15 K and 1 bar.

Many such chemical reactions release or consume vast amounts of energy. Usually, this results in cooling or heating of the substance as it changes. When heat and work are neither added nor removed from the system, an energy balance mandates an isenthalpic process,

$$\begin{aligned} h_{\text{reactants}} &= h_{\text{products}} \\ \bar{h}_{\text{H}_2} + 0.5\bar{h}_{\text{O}_2} &= \bar{h}_{\text{H}_2\text{O}} \end{aligned} \quad (6.24)$$

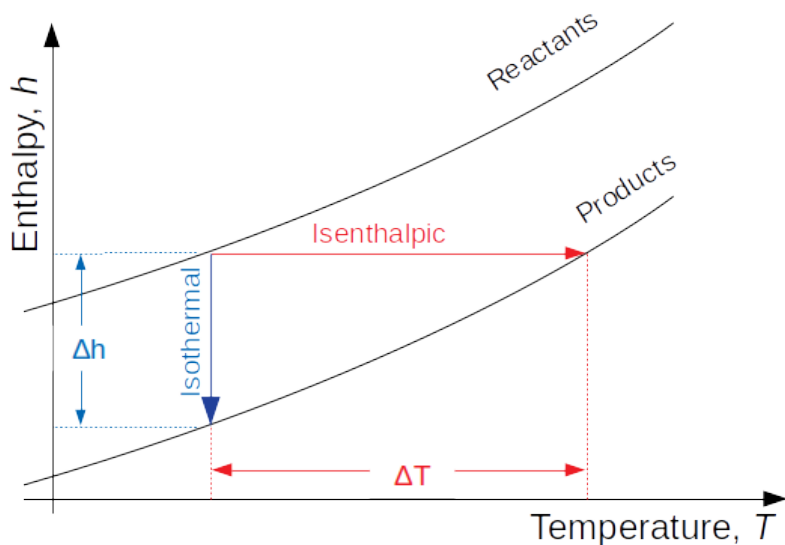


Figure 6.2: Isenthalpic and isothermal reactions on an h - T diagram.

when $\bar{h}$ is enthalpy in molar units.

This result is often counter intuitive. Since ideal gas enthalpy is only a function of temperature, it may seem like an isenthalpic process should also be isothermal. However, it is important to recall that the enthalpies of substances includes chemical energy, so these curves may include a dramatic offset from one another.

Fig. 6.2 shows enthalpy curves for a hypothetical set of reactants and products. Not only may the two have dissimilar slopes, but the curves may have large offsets separating them. When the process is isenthalpic, these offsets cause temperature changes shown in red. In the case shown, the reactants have higher enthalpy than the products, so the additional energy is absorbed thermally by the products, causing an increase in temperature. On a molecular level, the effect is like allowing two magnetic marbles to roll near one another on a flat table. Even if neither has much velocity to begin with, after they collide, they will be sent off quickly spinning and rolling. The same happens in an exothermic reaction.

If the hypothetical reactor were modified to add or extract heat so that the temperature of the reaction were constant, the process would adopt the vertical blue line instead. In an isothermal reaction, the amount of thermal energy exhibited by the substance is constant, so any release or consumption of heat will have come from a chemical reaction (including a change in the substance's active degrees of freedom, m). Therefore, the change in enthalpy between the two curves is the enthalpy consumed by the chemical reaction reaction.

In the diagram, the enthalpy is seen decreasing in order to maintain the temperature, so heat was removed, and the reaction is called exothermic. For our example with water, the enthalpy of formation would be

$$\Delta_f \bar{h}^\circ(T) = \bar{h}_{\text{H}_2\text{O}}(T) - \bar{h}_{\text{H}_2}(T) - 0.5\bar{h}_{\text{O}_2}(T) \quad (6.25)$$

The traditional notation for enthalpy of formation, $\Delta_f \bar{h}^\circ$, is cumbersome but specific. The Δ_f is traditionally used to denote “change in during formation,” and the circle denotes that the process is at the reference pressure. Enthalpy of formation is calculated isothermally, so it is also a function of the temperature at which the process is conducted.

It is important to note that the enthalpy of formation is *only* calculated relative to the constituent atoms in their reference state. The reference state is carefully defined as the atoms in their most stable state at 298.15 K and 1 bar. For hydrogen and oxygen, this is unambiguously diatomic gases, but other substances must be calculated from solid, like compounds involving magnesium.

Once enthalpy of formation is obtained in experiment, the value for h_0 from the previous chapter can be chosen to match at the experimental conditions. Then, the specific heats of species are sufficient information to extrapolate to other conditions.

However, s_0 and the entropy of formation, $\Delta_f s$, are much more difficult to obtain. The entropy of formation is usually inferred from equilibrium concentrations. A reversible reaction's equilibrium conditions are

$$0 = \Delta g = \Delta h - T_{eq} \Delta s. \quad (6.26)$$

So, there is an empirical link between the enthalpy of formation and the entropy of formation that can be used to calculate the one from the

other once the equilibrium concentrations are known. Unfortunately, these conditions occur in *systems* of groups of atoms interacting in numerous interactions and not necessarily all in the gas phase. A detailed treatment is not given here, but significant effort is required to generate values for s_0 from these data.

6.1.7 Speed of sound

In Section 1.1.12, the speed of sound, a , was defined as

$$a^2 = \left(\frac{\partial p}{\partial \rho} \right)_s. \quad (6.27)$$

Isentropic compression obeys

$$\begin{aligned} 0 &= T ds = de + p dv \\ &= dh - v dp. \end{aligned}$$

After some manipulation, these can be integrated to yield the isentropic relations for an ideal gas

$$\begin{aligned} \frac{T}{T_0} &= \left(\frac{v}{v_0} \right)^{-(\gamma-1)} = \left(\frac{\rho}{\rho_0} \right)^{\gamma-1} \\ \frac{T}{T_0} &= \left(\frac{p}{p_0} \right)^{\frac{\gamma-1}{\gamma}} \end{aligned}$$

or simply

$$\frac{p}{p_0} = \left(\frac{\rho}{\rho_0} \right)^{\gamma}. \quad (6.28)$$

Here, T_0 , p_0 , and ρ_0 represent the initial state of the gas, and the properties, T , p , and ρ is the state of the gas after isentropic compression or expansion.

Speed of sound, therefore, is obtained by differentiating pressure with respect to density,

$$\begin{aligned} a^2 &= \frac{\partial}{\partial \rho} p_0 \left(\frac{\rho}{\rho_0} \right)^{\gamma} = \gamma \frac{p_0}{\rho_0} \left(\frac{\rho}{\rho_0} \right)^{\gamma-1} = \gamma \frac{p_0}{\rho} \left(\frac{\rho}{\rho_0} \right)^{\gamma} = \gamma \frac{p}{\rho} \\ &= \gamma RT \end{aligned} \quad (6.29)$$

6.1.8 Other properties

Once $c_p(T)$ is well defined, it is also possible to evaluate internal energy,

$$e(T) = h(T) - RT, \quad (6.30)$$

Helmholtz free energy,

$$f(T, p) = h(T) - (R + s(T, p))T, \quad (6.31)$$

Gibbs energy,

$$g(T, p) = h(T) - Ts(T, p), \quad (6.32)$$

constant-volume specific heat,

$$c_v(T) = c_p(T) - R, \quad (6.33)$$

specific heat ratio,

$$\gamma(T) = \frac{c_p(T)}{c_p(T) - R}, \quad (6.34)$$

and others.

6.1.9 Properties of mixtures

The discussion so far has applied only to properties of pure ideal gases. The composition of a gas mixture is conventionally defined in either mass or molar quantities of the constituent pure gases. In this section, we establish methods by which properties can be calculated from the properties of the components.

In extensive units, a volume, V , might contain many individual gas species, each with total mass, m_i , or total mole count, N_i . They are related by the pure gas's molecular weight,

$$m_i = W_i N_i. \quad (6.35)$$

Mass and mole fractions are the fractions of a gas composed of a single pure substance. The total mass and count of gas in the volume is merely the sum of all the constituents, $m = \sum m_i$, and $N = \sum N_i$.

These let us more conveniently express the composition in mass and mole fractions, respectively,

$$y_i \equiv \frac{m_i}{\sum m_k} \quad (6.36a)$$

$$\chi_i \equiv \frac{N_i}{\sum N_k}. \quad (6.36b)$$

Note that the fractional quantities, y_i and χ_i , are not dependent on the choice of units for mass or mole count. Observe that, by definition, $\sum y_i = \sum \chi_i = 1$.

Mixture density is the total mass or mole count of all species divided by the volume they occupy.

$$\rho \equiv \frac{\sum m_i}{V} \quad (6.37a)$$

$$\bar{\rho} \equiv \frac{\sum N_i}{V} \quad (6.37b)$$

If ρ_i and $\bar{\rho}_i$ were the densities of constituent i at the same temperature and pressure as the total mixture, then it is important to emphasize that m_i/V and N_i/V are *not* ρ_i and $\bar{\rho}_i$. That question is addressed below.

Molecular weight of a gas mixture has the same definition as the molecular weight of a pure gas; it is the mass per mole count of total gas. It can be calculated from the constituent molecular weights using either mole fractions or mass fractions,

$$W \equiv \frac{\sum m_i}{\sum N_j} \quad (6.38a)$$

$$\begin{aligned} &= \frac{\sum N_i W_i}{\sum N_j} \\ &= \sum \chi_i W_i \end{aligned} \quad (6.38b)$$

$$\begin{aligned} &= \frac{\sum m_i}{\sum m_j / W_j} \\ &= \left(\sum \frac{y_j}{W_j} \right)^{-1}. \end{aligned} \quad (6.38c)$$

Partial pressure, p_i , is the pressure force exerted on a surface due only to collisions of a single constituent, i . Section 6.1.1 will help understand what is meant by this idea. It is the pressure force that would be measured if all other constituent gases were removed.

$$\begin{aligned} p_i &\equiv \frac{N_i}{V} R_u T \\ &= \chi_i \frac{N}{V} R_u T \\ &= \chi_i \bar{\rho} R_u T \end{aligned} \tag{6.39}$$

The same pressure may be calculated from mass fraction,

$$\begin{aligned} p_i &= \frac{N_i}{V} R_u T \\ &= \frac{m_i}{W_i V} \rho R_u T \\ &= y_i \rho R_i T \end{aligned} \tag{6.40}$$

The ideal gas constant, R_i , is the mass-based gas constant for only that constituent, $R_i = R_u / W_i$.

Total pressure, p , is the pressure force actually experienced by a surface due to all of the constituent gases. By definition,

$$\begin{aligned} p &= \sum p_i \\ &= \bar{\rho} R_u T \end{aligned} \tag{6.41}$$

Observe, also, that

$$\chi_i = \frac{p_i}{p}. \tag{6.42}$$

The total pressure can also be calculated in terms of mass density. Using (6.38c),

$$\begin{aligned} p &= \sum p_i \\ &= \sum y_i \rho R_i T \\ &= \sum \frac{y_i}{W_i} \rho R_u T \\ &= \rho R T. \end{aligned} \tag{6.43}$$

The total mixture gas constant appears naturally.

The mixture gas constant, R , appears naturally when calculating total pressure from total mass density, and it is calculated in precisely the same way as a pure gas constant, but the mixture molecular weight is used instead,

$$R \equiv \frac{R_u}{W} \quad (6.44a)$$

$$\begin{aligned} &= \frac{R_u}{\sum \chi_i W_i} \\ &= \left(\frac{\chi_i}{R_i} \right)^{-1} \end{aligned} \quad (6.44b)$$

$$\begin{aligned} &= R_u \sum \frac{y_i}{W_i} \\ &= \sum y_i R_i \end{aligned} \quad (6.44c)$$

Densities can be calculated from the same properties of the component species using the ideal gas law. For mass density,

$$\begin{aligned} \rho &= \frac{p}{RT} \\ &= \frac{p}{\sum y_i R_i T} \\ &= \left(\sum \frac{y_i}{\rho_i(T, p)} \right)^{-1} \end{aligned} \quad (6.45a)$$

$$\begin{aligned} &= \frac{p}{(\sum \chi_i / R_i)^{-1} T} \\ &= \sum \chi_i \rho_i(T, p) \end{aligned} \quad (6.45b)$$

For molar density, no such complexity is needed, since the universal gas constant is the same for all gases,

$$\bar{\rho} = \frac{p}{R_u T}$$

Internal energy, enthalpy, entropy, and other bulk properties are merely the sum of all the values contributed by each of the constituent gases. By definition, ideal gases in a mixture do not affect

one another, so each is evaluated at the pressure it *would* exhibit if all other gases were removed; the partial pressure, p_i .

$$\begin{aligned}\left(\sum m_j\right) \phi(T, p) &= \sum m_i \phi_i(T, p_i) \\ \left(\sum N_j\right) \bar{\phi}(T, p) &= \sum N_i \bar{\phi}_i(T, p_i)\end{aligned}$$

Therefore,

$$\begin{aligned}\phi(T, p) &= \sum y_i \phi_i(T, p) \\ \bar{\phi}(T, p) &= \sum \chi_i \bar{\phi}_i(T, p_i)\end{aligned}$$

In ideal gases, only entropy is affected by mixing. We leave the discussion of the entropy of mixing for Section 6.2.3.

This identity may be applied to internal energy, enthalpy, entropy, and all the properties derived from them, so

$$e(T) = \sum y_i e_i(T) \quad (6.46a)$$

$$h(T) = \sum y_i h_i(T) \quad (6.46b)$$

$$s(T, p) = \sum y_i s_i(T, p_i) \quad (6.46c)$$

and

$$\bar{e}(T) = \sum \chi_i \bar{e}_i(T) \quad (6.47a)$$

$$\bar{h}(T) = \sum \chi_i \bar{h}_i(T) \quad (6.47b)$$

$$\bar{s}(T, p) = \sum \chi_i \bar{s}_i(T, p_i). \quad (6.47c)$$

6.2 The ideal gas collection

There are several classes that implement various data models for ideal gas properties. Their interfaces are all standardized so that very little difference should be apparent to the user, except that the ideal gas mixture class has some extra methods associated with its composition.

All property methods accept any two of temperature, density, or pressure to specify the state.

6.2.1 The Shomate equation: `ig`

PYroMat's `ig` class is built on the Shomate equation for constant-pressure specific heat c_p . This is the formulation used by the NIST webbook [12]. It has the advantage of using a simple standard piece-wise formulation for specific heat, but it does suffer from certain limitations.

The Shomate equation takes the form

$$\theta = \frac{T}{T_s} \quad (6.48)$$

$$c_p(t) = c_0 + c_1\theta + c_2\theta^2 + c_3\theta^3 + \frac{c_4}{\theta^2}, \quad (6.49)$$

where the scaling temperature, T_s is 1000K for all species. The decision to scale the temperature by a large value has the effect of scaling τ so that it will not be much larger than 5 or 6. That helps reduce numerical errors in high-order polynomials.

Because of its simplicity, the Shomate equations lack the degrees of freedom to express specific heat over wide ranges, so data are usually given in piece-wise formulations. For example, tungsten dioxide (WO_2), has a set of coefficients for $298\text{K} \leq T < 1100\text{K}$ and $1100\text{K} \leq T \leq 6000\text{K}$.

The enthalpy can be explicitly calculated from (6.21),

$$\begin{aligned} h(T) &= h_0 + \int c_p(T) dT \\ &= h_0 + T_s \int c_p(\theta) d\theta \\ &= T_s \left(c_0\theta + \frac{c_1}{2}\theta^2 + \frac{c_2}{3}\theta^3 + \frac{c_3}{4}\theta^4 - \frac{c_4}{\theta} + c_5 \right). \end{aligned} \quad (6.50)$$

It is important to emphasize that h_0 is not the same as the enthalpy of formation, Δh_f° . Instead, it is merely an integration constant, which can be alternately expressed as a new coefficient, c_5 .

Because of the temperature term in the denominator, no multiple

of T_s appears in entropy when the integration is changed to θ ,

$$\begin{aligned}
 s^\circ(T) &= s_0 + \int \frac{c_p(T)}{T} dT \\
 &= s_0 + \int \frac{c_p(\theta)}{\theta} d\theta \\
 &= c_0 \ln \theta + c_1 \theta + \frac{c_2}{2} \theta^2 + \frac{c_3}{3} \theta^3 - \frac{c_4}{2\theta^2} + c_6 \quad (6.51) \\
 s(T, p) &= s^\circ(T) - R \ln \left(\frac{p}{p^\circ} \right)
 \end{aligned}$$

Just like in the enthalpy integral, a new coefficient, c_6 , has been introduced to represent the integration constant.

Table 6.1 lists the data elements that define the `ig` class. For more information on how `.hpd` files are stored, see Section 5.2.2. Most of the essential data elements are self explanatory, but the coefficients and temperature limits must have compatible sizes. Even though the `ig` class only uses seven coefficients, there are eight provided in the NIST data sets. If there are N sets of coefficients defined over N temperature ranges, there must be N arrays of eight coefficients and $N + 1$ temperature limit values. 5 For example, in a data set with two temperature ranges, the following would define a data set valid between `<T0>` and `<T2>`, and the transition between the two piece-wise data sets is at `<T1>`.

```

"Tlim": [<T0>, <T1>, <T2>]
"C": [
  [ <c0>, <c1>, <c2>, <c3>, <c4>, <c5>, <c6>, <c7> ],
  [ <c0>, <c1>, <c2>, <c3>, <c4>, <c5>, <c6>, <c7> ],
]

```

6.2.2 The NASA polynomial: `ig2`

The so-called “NASA polynomials” are a piece-wise empirical formulation to the specific heat of an ideal gas. They are taken from [13] predate the latest formulation of the NIST-JANAF tables, and are even used for reference. Unlike the Shomate equation, there is no $1/t^2$ term, there is no attempt to scale temperature prior to evaluating the

Table 6.1: HPD data file elements for the `ig` class

Name	Type	Description
<code>id</code>	<code>str</code>	Unique substance identifier string
<code>class</code>	<code>= 'ig'</code>	Class identifier string
<code>doc</code>	<code>str</code>	Documentation string
<code>atoms</code>	<code>dict</code>	Keys are elemental atoms, and values are their integer quantities in the substance.
<code>mw</code>	<code>float</code>	The molecular weight.
<code>Tlim</code>	<code>list</code>	A sorted list of $N + 1$ floating point temperatures. Middle values define the boundaries between piece-wise coefficient ranges. High and low values define the limits of the model.
<code>C</code>	<code>nested list</code>	The value, $C[i][j]$, corresponds to coefficient c_j in the temperature interval <code>Tlim[i]</code> to <code>Tlim[i+1]</code> .
<code>TAB</code>	<code>nested list</code>	Optional table of truth values published by NIST used for validation

polynomial, and properties are scaled with respect to the ideal gas constant.

$$c_p(T) = R (c_0 + c_1T + c_2T^2 + c_3T^3 + c_4T^4) \quad (6.52)$$

There are nearly identical formulations for enthalpy,

$$h(T) = R \left(c_0T + \frac{c_1}{2}T^2 + \frac{c_2}{3}T^3 + \frac{c_3}{4}T^4 + \frac{c_4}{5}T^5 + c_5 \right), \quad (6.53)$$

and entropy

$$s^\circ(T) = R \left(c_0 \ln(T) + c_1T + \frac{c_2}{2}T^2 + \frac{c_3}{3}T^3 + \frac{c_4}{4}T^4 + c_6 \right). \quad (6.54)$$

Here, just as in the Shomate equations, c_5 and c_6 are introduced as integration constants in enthalpy and entropy.

There is some significant overlap in the species offered by the two classes. For substances that have available models for both, PYroMat actually installs with data for both models. However, the Shomate equation substances are suppressed from loading so the older NASA polynomials take precedence for two reasons:

- Most of the NASA models have wider ranges of validity than the Shomate models.
- Some of the Shomate data have been found to suffer from discontinuity errors at the piecewise boundaries.

Otherwise, the two have been found to be sufficiently equivalent that most users will not find a reason to be aware of the distinction.

Table 6.2 lists the data elements that define the `ig2` class. For more information on how `.hpd` files are stored, see Section 5.2.2. Most of the essential data elements are self explanatory, but the coefficients and temperature limits must have compatible sizes. Even though the `ig2` class only uses seven coefficients, there are eight provided in the NIST data sets. If there are N sets of coefficients defined over N temperature ranges, there must be N arrays of eight coefficients and $N + 1$ temperature limit values.

For example, in a data set with two temperature ranges, the following would define a data set valid between `<T0>` and `<T2>`, and the transition between the two piece-wise data sets is at `<T1>`.

Table 6.2: HPD data file elements for the `ig2` class

Name	Type	Description
<code>id</code>	<code>str</code>	Unique substance identifier string
<code>class</code>	<code>= 'ig2'</code>	Class identifier string
<code>doc</code>	<code>str</code>	Documentation string
<code>atoms</code>	<code>dict</code>	Keys are elemental atoms, and values are their integer quantities in the substance.
<code>pref</code>	<code>float</code>	Reference pressure in Pascals
<code>mw</code>	<code>float</code>	The molecular weight.
<code>Tlim</code>	<code>list</code>	A sorted list of $N + 1$ floating point temperatures: Middle values define the boundaries between piece-wise coefficient ranges. High and low values define the limits of the model.
<code>C</code>	<code>nested list</code>	The value, $C[i][j]$, corresponds to coefficient c_j in the temperature interval $Tlim[i]$ to $Tlim[i+1]$.

```
"Tlim": [<T0>, <T1>, <T2>]
"C": [
  [<c0>, <c1>, <c2>, <c3>, <c4>, <c5>, <c6>, <c7>],
  [<c0>, <c1>, <c2>, <c3>, <c4>, <c5>, <c6>, <c7>],
]
```

6.2.3 The ideal gas mixture: `igmix`

The `PYroMat` ideal gas mixture class defines methods to calculate properties of a static mixture of ideal gases. For improved computational efficiency, properties like the mixture molecular weight, mole fractions, and mass fractions, are calculated ahead of time and stored in class instances for later use.

Section 6.1.9 already shows how properties of a mixture can be calculated from the properties of the constituent gases. The implementation of all but entropy is quite straightforward. In all of the ideal gas codes, the entropy at the reference pressure, s° , is calculated in

a separate efficient internal method. For that reason, the `igmix` class calls on these methods directly and then treats the pressure dependence separately.

When calculating in molar units,

$$\begin{aligned}\bar{s}(T, p) &= \sum \chi_i \bar{s}(T, p_i) \\ &= \sum \chi_i \bar{s}_i^\circ(T) - \sum \chi_i R_u \ln \left(\frac{p \chi_i}{p_i^\circ} \right),\end{aligned}$$

where p_i is the partial pressure of constituent i and p_i° is the reference pressure of constituent i .

The partial pressure is merely $p \chi_i$, but the reference pressure for each substance may vary depending on the data source. We deal with this in the following way.

$$\begin{aligned}\sum \chi_i \ln \left(\frac{p \chi_i}{p_i^\circ} \right) &= \sum \chi_i (\ln p + \ln \chi_i - \ln p_i^\circ) \\ &= \left(\ln p - \sum \chi_i \ln p_i^\circ \right) + \sum \chi_i \ln \chi_i\end{aligned}$$

When

$$p_{\text{mix}}^\circ = \exp \left(\sum \chi_i \ln p_i^\circ \right) \quad (6.55)$$

$$\bar{s}_{\text{mix}} = -R_u \sum \chi_i \ln \chi_i, \quad (6.56)$$

the entropy is calculated by

$$\bar{s}(T, p) = \bar{s}_{\text{mix}} + \sum \chi_i \bar{s}_i^\circ(T) - R_u \ln \left(\frac{p}{p_{\text{mix}}^\circ} \right),$$

The new reference pressure, p_{mix}° , is a log-weighted average of the constituent reference pressures. In the event that they are all the same, the mixture reference pressure is also the same.

Apart from $\bar{s}_{\text{mix}}$, Equation 7.1 is merely a weighted average of the entropies of the individual constituent gases. Neither a function of temperature nor pressure, the entropy of mixing, $\bar{s}_{\text{mix}}$, is a constant for a given mixture of constituents, and it represents the entropy created by dispersing gases into one another. The same gases, neatly partitioned

Table 6.3: HPD data file elements for the `igmix` class

Name	Type	Description
<code>id</code>	<code>str</code>	Unique substance identifier string
<code>class</code>	<code>= 'igmix'</code>	Class identifier string
<code>doc</code>	<code>str</code>	Documentation string
<code>bymass</code>	<code>bool</code>	Indicates whether contents are listed by mass or by mole count
<code>contents</code>	<code>dict</code>	Specifies the mixture composition: Keys are pure ideal gas id strings, values are quantities. The quantities will be normalized after load, so they do not need to add to one.

into partitioned thin-walled compartments, and the systems entropy would be correctly calculated without $\bar{s}_{\text{mix}}$.

The smallest possible value for $\bar{s}_{\text{mix}}$ is zero, which is only achieved when χ_i values are all zero or unity. Otherwise, since values of χ_i are positive and less than one, its log will always be negative, and the contribution to $\bar{s}_{\text{mix}}$ positive.

The `igmix._build()` method calculates a number of mixture properties that do not depend on state, including $\bar{s}_{\text{mix}}$ and p_{mix}° , which are stashed in `_smix` and `_pref_pa` attributes respectively. The `_build()` method is also responsible for accumulating temperature limits, representing the narrowest of all the constituent gases, stored in `_Tlim`; the mixture's effective molecular weight, stored in `_mw`; and mass-based and mole-based mixture fraction dictionaries, stored in `_y` and `_x` respectively.

Chapter 7

Multi-phase substance models

Once a thermodynamic model permits changes of phase, it makes little sense to specify the state in terms of temperature and pressure. After all, boiling represents a massive isobaric and isothermal discontinuity in all other properties. On the other hand, using density requires no such discontinuity, and it even permits modeling meta-stable states. For this reason, contemporary multi-phase models use temperature and density to specify the thermodynamic state.

Historically, there have been countless approaches to the problem, beginning with the so-called “viral” terms inspired by Van der Waals forces in early work on the kinetic theory of gases. This approach began an evolution of increasingly complex variants of the ideal gas law, from which pressure can be calculated from increasingly complicated formula in terms of temperature and density. In this approach, parallel models for the specific heat are needed to establish a total substance model. The so-called “Span and Wagner” models in broad use today manage to address all of these issues in a single, albeit rather cumbersome formula.

7.1 Span and Wagner Equation of State

Both the `mp2` and its predecessor, `mp1`, use what is sometimes called the “Span and Wagner” model. Its earliest publication appears to be in 1991[14, 15], but it was in a three-part paper in 2003 that Span and Wagner make a defense for the model’s broad use for the description of the thermodynamic properties of substances across phase change boundaries[16, 17, 18]. Its most important feature is that it is generalizable to an incredible variety of substances without the need for custom formulae – all the terms merely need their own coefficients, which a set algorithm can load and use for calculation.

The models supported by PYroMat’s multi-phase classes uses a single formula for Helmholtz free energy as a function of temperature and density,

$$f(T, \rho) \equiv e(T, \rho) - Ts(T, \rho) \quad (7.1)$$

$$= h - \frac{p}{\rho} - Ts. \quad (7.2)$$

The impressive body of data on which PYroMat draws has been accumulated as follows. First, individual researchers accumulate bodies of raw experimental data describing the properties like specific heat, pressure, phase-change boundary, critical point, etc. with as great accuracy as is feasible. The team wishing to produce a free energy model must accumulate these data, compare them for consistency, and make judgments about which are to be trusted, which discarded, and which kept with some skepticism. A large bank of terms for an expansion representing $f(T, \rho)$ (described below) are used to fit raw data in a regression model, and the number of terms is iteratively reduced until the precision of the model drops to roughly match the uncertainty of the original data. In well-studied cases like cryogenic nitrogen, this may be on the order of 0.01%. In other cases, like some more esoteric refrigerants, this might be on the order of 1% or 2%.

If the uncertainty implied in the model’s calculation is of concern to the reader, it will be important to obtain the original publication to thoroughly understand the model’s strengths and weaknesses. This topic is far too deep, and PYroMat includes far too many individual substance models to go into this kind of detail on each one.

7.1.1 Nondimensionalization

These models are built from a nondimensional formulation for free energy. This means the terms can be scaled to be more or less on the order of unity, and the choice of molar versus mass density is irrelevant. Many authors use α to represent a nondimensional free energy, and we adopt the same here.

$$\alpha(\tau, \delta) = \frac{f(T, \rho)}{RT} \quad (7.3a)$$

$$\tau = \frac{T_c}{T} \quad (7.3b)$$

$$\delta = \frac{\rho}{\rho_c}. \quad (7.3c)$$

The free energy is normalized by the quantity RT , which is motivated by (6.14) from the study of ideal gases. Normalizing temperature and density by the critical point values acknowledges that the critical point is a natural scale for the important phenomena in this substance. The choice to make τ scale like the inverse of temperature is motivated by the Boltzmann and Maxwell distributions for molecular velocity, in which temperature appears in a denominator.

The formulation for dimensionless free energy, α , is split into two parts: the ideal gas part, α^o , and the residual part, α^r . So, the total free energy is

$$\alpha(\tau, \delta) = \alpha^o(\tau, \delta) + \alpha^r(\tau, \delta) \quad (7.4)$$

This approach separates the problem of needing to model the energy contained in molecular vibration and translation of a large number of independent oscillators (α^o) from the problem of modeling the effects of intermolecular forces with increasing density (α^r).

7.1.2 Ideal gas portion of free energy

The ideal gas portion of the free energy can be constructed from a specific heat model, just as other properties were for the ideal gas substances in Chapter 6. The ideal gas enthalpy is merely the integral

of specific heat, and ideal gas entropy can be similarly constructed in terms of specific heat, temperature, and density,

$$h = h_0 + \int_{T_0}^T c_p dT \quad (7.5a)$$

$$s = s_0 + \int_{T_0}^T \frac{c_p}{T} dT - R \ln \left(\frac{\rho}{\rho_0} \right) - R \ln \left(\frac{T}{T_0} \right). \quad (7.5b)$$

When these are transposed into the dimensionless parameters, τ and δ ,

$$h = h_0 - T_c \int_{\tau_0}^{\tau} \frac{c_p}{\tau^2} d\tau$$

$$s = s_0 - \int_{\tau_0}^{\tau} \frac{c_p}{\tau} d\tau - R \ln \left(\frac{\delta \tau_0}{\delta_0 \tau} \right).$$

When these are used to calculate the dimensionless free energy,

$$\begin{aligned} \alpha^o &= \frac{h - RT - Ts}{RT} = \frac{h\tau}{RT_c} - 1 - \frac{s}{R} \\ &= \frac{h_0}{RT_c} \tau - \frac{s_0}{R} - 1 + \ln \left(\frac{\delta \tau_0}{\delta_0 \tau} \right) + \left(-\tau \int_{\tau_0}^{\tau} \frac{c_p}{R\tau^2} d\tau + \int_{\tau_0}^{\tau} \frac{c_p}{R\tau} d\tau \right) \end{aligned} \quad (7.6)$$

Note that, for ease of interpretation, we have adopted a lazy notation with τ both inside and outside of the integrals. We will be more strict with our notation below.

The difficulty in formulating the specific heat of even an ideal gas is introduced in Section 1.1.6. It is, briefly, that portions of energy that a substance stores in translation (thermal energy) versus the many internal forms of vibration depends heavily on the temperature. A fully detailed model would need to include molecular vibration, electronic, magnetic energies, and quantum mechanical effects. Within the confines of the Span and Wagner format, different authors are still able to approach this differently – using formulations that range from polynomial fits to a more physics-based approach.

One of the biggest challenges in describing the specific heats of gases is found at low temperatures. Here, the different modes of oscillation

are found to have dramatically dissimilar energies (defying the equipartition assumption). The specific heat of many independent oscillators assuming a distribution of discrete energies adopts the form

$$\frac{c_p(\tau)}{R} = \dots + bm^2 \frac{\tau^2 \exp(m\tau)}{(\exp(m\tau) - 1)^2} + \dots,$$

where b represents the magnitude of the feature, and m is a dimensionless temperature (normalized by T_c) at which the feature occurs.

The integrals of specific heat can be collectively simplified by inverting the integration-by-parts procedure with a series of substitutions, so that

$$-\tau \int_{\tau_0}^{\tau} \frac{c_p(t)}{Rt^2} dt + \int_{\tau_0}^{\tau} \frac{c_p(t)}{Rt} dt = - \int_{\tau_0}^{\tau} \left[\int_{\tau_0}^t \frac{c_p(t')}{Rt'^2} dt' \right] dt. \quad (7.7)$$

This has the fortunate effect of canceling the τ^2 in the quantum component of specific heat, making its integral straightforward.

$$\begin{aligned} & - \int_{\tau_0}^{\tau} \int_{\tau_0}^t \frac{bm^2 \exp(mt')}{(\exp(mt') - 1)^2} dt' dt \dots \\ &= \int_{\tau_0}^{\tau} \left[\frac{bm}{\exp(mt) - 1} - \frac{bm}{\exp(m\tau_0) - 1} \right] dt \\ &= \int_{\tau_0}^{\tau} \frac{bm \exp(-mt)}{1 - \exp(-mt)} dt - \frac{bm}{\exp(m\tau_0) - 1} (\tau - \tau_0) \\ &= b \ln(1 - \exp(-m\tau)) - b \ln(1 - \exp(-m\tau_0)) + \dots \\ &\quad - \frac{bm}{\exp(m\tau_0) - 1} (\tau - \tau_0) \end{aligned} \quad (7.8)$$

The same approach could be applied to the integration of the polynomial, but it is probably more easily dealt with in separate terms. For

a sum of terms with arbitrary exponents,

$$\begin{aligned}
& -\tau \int_{\tau_0}^{\tau} \sum_k c_k t^{k-2} dt + \int_{\tau_0}^{\tau} \sum_k c_k t^{k-1} dt \dots \\
& = -\tau \left[c_1 \ln t + \sum_{k \neq 1} \frac{c_k t^{k-1}}{k-1} \right]_{\tau_0}^{\tau} + \left[c_0 \ln t + \sum_{k \neq 0} \frac{c_k t^k}{k} \right]_{\tau_0}^{\tau} \\
& = -\tau c_1 \ln \tau + \tau c_1 \ln \tau_0 - \left[\sum_{k \neq 1} \frac{c_k \tau^k}{k-1} \right] + \tau \left[\sum_{k \neq 1} \frac{c_k \tau_0^{k-1}}{k-1} \right] + \dots \\
& \quad + c_0 \ln \tau - c_0 \ln \tau_0 + \left[\sum_{k \neq 0} \frac{c_k \tau^k}{k} \right] - \left[\sum_{k \neq 0} \frac{c_k \tau_0^k}{k} \right]. \quad (7.9)
\end{aligned}$$

Note that this analysis does not require that values of k be limited to integers.

Collectively, these integrals motivate the form that is now commonplace in contemporary free-energy-based models,

$$\alpha^o(\tau, \delta) = \ln \delta + (c_0 + c_1 \tau) \ln \tau + \alpha_0^o(\tau) + \alpha_1^o(\tau), \quad (7.10a)$$

where α_0^o is a polynomial on τ ,

$$\alpha_0^o(\tau) = \sum_k c_j \tau^k, \quad (7.10b)$$

and α_1^o includes the quantum terms,

$$\alpha_1^o(\tau) = \sum_j b_j \ln(1 - \exp(-m_j \tau)). \quad (7.10c)$$

In many models, the $\tau \ln \tau$ term is omitted from Equation 7.10a, implying that there was no linear term on τ (proportional to $1/T$) in the formulation of specific heat. Because much of the complexity of the specific heat is captured by the α_1^o terms, the polynomial, α_0^o , may only contain a constant and a linear term. Still, exponentials and logarithms are numerically expensive, so this model can already be significantly more expensive than their **ig** and **ig2** equivalents.

7.1.3 Residual portion of free energy

The residual (or real-fluid) portion of free energy attempts to account for everything that the ideal gas portion does not. Many of these terms become especially important when the substance density is high, because they are responsible for quantifying the impact of the intermolecular forces ignored by the ideal gas model. The model is divided into three groups of terms,

$$\alpha^r = \alpha_0^r(\tau, \delta) + \alpha_1^r(\tau, \delta) + \alpha_2^r(\tau, \delta) \quad (7.11a)$$

Many of these terms are physically motivated, but that discussion is not included here.

The first group of terms is a series of polynomials multiplied by exponentials of powers of density,

$$\alpha_0^r = p_0(\tau, \delta) + \sum_{k=1}^K \exp(-\delta^k) p_k(\tau, \delta) \quad (7.11b)$$

$$p_k = \sum_i c_i \tau^{a_i} \delta^{b_i}. \quad (7.11c)$$

While it is not explicitly forbidden, the δ exponent, b , is neither zero nor negative in these models, so α_0^r terms with the exponential multipliers vanish near $\delta \rightarrow 0$ and $\delta \rightarrow \infty$. These terms model the complicated behaviors that occur near the phase change (when δ is on the order 1).

In most models, the first group (α_0^r) constitutes the largest number of terms (often dozens). The exponents in the polynomial expansion are sometimes integers, but they are often rationals (which permits efficient evaluation using the method described in Section 8.1), and they are rarely long decimal values.

The second group of terms, α_1^r , is a Gaussian function multiplied by powers of τ and δ . It was first introduced in 1991 by Setzmann et al. to better model the severely distorted surface that occurs near the critical point in oxygen. Most models only use a few of these terms, and they are usually centered (by ϵ and γ) near the critical point. From the 1991 paper, “The opinion has been widely held that analytic equations of state covering the whole fluid region are not able to represent the properties in the critical region.” The authors continue

to say, “We will show that our new equation is able to represent the existing experimental information on the thermodynamic properties of methane in the critical region at least as well as [competing models].” [15]

The third group, α_2^r , is a Gaussian centered on exactly the critical point, but the coefficient, Δ , is a scaled distance from the critical point. The term appears in a 1994 paper modeling carbon dioxide [20] and again in a 2014 revision to the properties of water and steam [21], but in conspicuously few others. While its formulation is not explained in detail in these sources, it allows for a more complicated shape from the term without the computational expense of additional polynomial terms. It seems to have been designed to reduce the number of Gaussian terms required to produce the desired result at the critical point.

$$\alpha_1^r = \sum_{l=0}^L c_l \delta^{d_l} \tau^{t_l} \exp \left(-a_l (\delta - \epsilon_l)^2 - b_l (\tau - \gamma_l)^2 \right) \quad (7.11d)$$

$$\alpha_2^r = \sum_{m=0}^M c_m \Delta_m^{b_m} \delta \exp \left(-C_m (\delta - 1)^2 - D_m (\tau - 1)^2 \right) \quad (7.11e)$$

$$\Delta_m = \left[1 - \tau + A_m ((\delta - 1)^2)^{1/2\beta_m} \right]^2 + B_m ((\delta - 1)^2)^{a_m}. \quad (7.11f)$$

At first glance, these formulations may appear to be an inefficient tangled mess of nested exponents. However, quantities like $\tau - 1$ and $\delta - 1$ may be positive or negative, and the exponents are not integers. It is numerically elegant to square these quantities (requiring just a multiplication operation) before raising them to decimal or fractional powers.

7.1.4 Derivatives of free energy

As we will address in the next section, it is also important to evaluate up to the second derivative of these terms. This is sufficiently cumbersome to deserve some treatment here. The efficient evaluation of derivatives of polynomials is addressed in Section 8.1, so we only need to treat the other aspects of these functional forms.

The ideal gas portion in (7.10a) has derivatives

$$\alpha_\tau^o = (c_0 - 1)\tau^{-1} + c_1(1 + \ln \tau) + \alpha_{0,\tau}^o + \alpha_{1,\tau}^o \quad (7.12a)$$

$$\alpha_\delta^o = \delta^{-1} \quad (7.12b)$$

$$\alpha_{\tau\tau}^o = -(c_0 - 1)\tau^{-2} + c_1\tau^{-1} + \alpha_{0,\tau\tau}^o + \alpha_{1,\tau\tau}^o \quad (7.12c)$$

$$\alpha_{\tau\delta}^o = 0 \quad (7.12d)$$

$$\alpha_{\delta\delta}^o = -\delta^{-2} \quad (7.12e)$$

The polynomial, $\alpha_0^o(\tau)$, and its derivatives can be evaluated separately. All that remains is to determine the derivatives of $\alpha_1^o(\tau)$.

$$\alpha_{1,\tau}^o = \sum_j \frac{b_j m_j}{\exp(m_j \tau) - 1} \quad (7.13a)$$

$$\alpha_{1,\tau\tau}^o = \sum_j \frac{b_j m_j^2 \exp(m_j \tau)}{(\exp(m_j \tau) - 1)^2} \quad (7.13b)$$

Derivatives of the residual portion in (7.11a) can be calculated individually. Much of α_0^r in (7.11c) can be calculated trivially from the derivatives of the polynomials.

$$\alpha_{0,\tau}^r = p_{0,\tau} + \sum_{k=1}^K \exp(-\delta^k) p_{k,\tau} \quad (7.14a)$$

$$\alpha_{0,\delta}^r = p_{0,\delta} + \sum_{k=1}^K \exp(-\delta^k) (p_{k,\delta} - k\delta^{k-1} p_k) \quad (7.14b)$$

$$\alpha_{0,\tau\tau}^r = p_{0,\tau\tau} + \sum_{k=1}^K \exp(-\delta^k) p_{k,\tau\tau} \quad (7.14c)$$

$$\alpha_{0,\tau\delta}^r = p_{0,\tau\delta} + \sum_{k=1}^K \exp(-\delta^k) (p_{k,\tau\delta} - k\delta^{k-1} p_{k,\tau}) \quad (7.14d)$$

$$\begin{aligned} \alpha_{0,\delta\delta}^r = p_{0,\delta\delta} + \sum_{k=1}^K \exp(-\delta^k) & \left(p_{k,\delta\delta} - k(k-1)\delta^{k-2} p_k \dots \right. \\ & \left. - 2k\delta^{k-1} p_{k,\delta} + k^2 \delta^{2(k-1)} p_k \right) \end{aligned} \quad (7.14e)$$

Derivatives of the first Gaussian terms in (7.11d) are conveniently calculated as proportional to the terms themselves. If each term were expressed as the polynomial term multiplier, p_l , and the Gaussian term, g_l , then $\alpha_1^r = \sum_l p_l g_l$. Derivatives on α_1^r can be conveniently constructed in terms of those intermediates.

$$p_l = c_l \tau^{t_l} \delta^{d_l} \quad (7.15a)$$

$$p_{l,\tau} = t_l \frac{p_l}{\tau} \quad (7.15b)$$

$$p_{l,\delta} = d_l \frac{p_l}{\delta} \quad (7.15c)$$

$$p_{l,\tau\tau} = t_l(t_l - 1) \frac{p_l}{\tau^2} \quad (7.15d)$$

$$p_{l,\tau\delta} = t_l d_l \frac{p_l}{\tau \delta} \quad (7.15e)$$

$$p_{l,\delta\delta} = d_l(d_l - 1) \frac{p_l}{\delta^2} \quad (7.15f)$$

$$g_l = \exp(-a_l(\delta - \epsilon_l)^2 - b_l(\tau - \gamma_l)^2) \quad (7.16a)$$

$$g_{l,\tau} = -2b_l(\tau - \gamma_l) g_l \quad (7.16b)$$

$$g_{l,\delta} = -2a_l(\delta - \epsilon_l) g_l \quad (7.16c)$$

$$g_{l,\tau\tau} = -2b_l g_l - 2b_l(\tau - \gamma_l) g_{l,\tau} \quad (7.16d)$$

$$g_{l,\tau\delta} = -2b_l(\tau - \gamma_l) g_{l,\delta} = -2a_l(\delta - \epsilon_l) g_{l,\tau} \quad (7.16e)$$

$$g_{l,\delta\delta} = -2a_l g_l - 2a_l(\delta - \epsilon_l) g_{l,\delta} \quad (7.16f)$$

$$\alpha_{1,\tau}^r = \sum_{l=0}^L p_{l,\tau} g_l + p_l g_{l,\tau} \quad (7.17a)$$

$$\alpha_{1,\delta}^r = \sum_{l=0}^L p_{l,\delta} g_l + p_l g_{l,\delta} \quad (7.17b)$$

$$\alpha_{1,\tau\tau}^r = \sum_{l=0}^L p_{l,\tau\tau} g_l + 2p_{l,\tau} g_{l,\tau} + p_l g_{l,\tau\tau} \quad (7.17c)$$

$$\alpha_{1,\tau\delta}^r = \sum_{l=0}^L p_{l,\tau\delta} g_l + p_{l,\tau} g_{l,\delta} + p_{l,\delta} g_{l,\tau} + p_l g_{\tau\delta} \quad (7.17d)$$

This approach to evaluating derivatives in stages helps numerical efficiency, since there are obvious ways to make repeated use of intermediate values, but it also helps with the clarity of the code. The same approach is even more valuable in the treatment of the second Gaussian terms in (7.11e). First, we turn our attention to derivatives of the distance term, Δ . For ease of notation, we will omit the subscript, m , on all of the empirical coefficients.

$$\Delta_\tau = -2 \left[1 - \tau + A ((\delta - 1)^2)^{1/2\beta} \right] \quad (7.18a)$$

$$\Delta_\delta = 2 \frac{A}{\beta} \left[1 - \tau + A ((\delta - 1)^2)^{1/2\beta} \right] ((\delta - 1)^2)^{1/2\beta-1/2} \quad (7.18b)$$

$$\Delta_{\tau\tau} = 2 \quad (7.18c)$$

$$\Delta_{\tau\delta} = 2 \frac{A}{\beta} ((\delta - 1)^2)^{1/2\beta-1/2} \quad (7.18d)$$

$$\begin{aligned} \Delta_{\delta\delta} = & 2 \left(\frac{A}{\beta} \right)^2 ((\delta - 1)^2)^{1/\beta-1} \dots \\ & + 2 \frac{A}{\beta} \left(\frac{1}{\beta} - 1 \right) \left[1 - \tau + A ((\delta - 1)^2)^{1/2\beta} \right] ((\delta - 1)^2)^{1/2\beta-1} \end{aligned} \quad (7.18e)$$

Next, we will consider the exponential term separately,

$$f(\tau, \delta) = \exp(-C(\delta - 1)^2 - D(\tau - 1)^2) \quad (7.19a)$$

$$f_\tau = -2D(\tau - 1)f \quad (7.19b)$$

$$f_\delta = -2C(\delta - 1)f \quad (7.19c)$$

$$f_{\tau\tau} = -2Df - 2D(\tau - 1)f_\tau \quad (7.19d)$$

$$f_{\tau\delta} = -2D(\tau - 1)f_\delta = -2C(\delta - 1)f_\tau \quad (7.19e)$$

$$f_{\delta\delta} = -2Cf - 2C(\delta - 1)f_\delta \quad (7.19f)$$

Finally, the α_2^r derivatives may be constructed as

$$\alpha_2^r = \Delta^b \delta f \quad (7.20a)$$

$$\alpha_{2,\tau}^r = b \Delta_\tau \Delta^{b-1} \delta f + \Delta^b \delta f_\tau \quad (7.20b)$$

$$\alpha_{2,\delta}^r = b \Delta_\delta \Delta^{b-1} \delta f + \Delta^b f + \Delta^b \delta f_\delta \quad (7.20c)$$

$$\begin{aligned} \alpha_{2,\tau\tau}^r = b \left(\Delta_{\tau\tau} \Delta^{b-1} + (b-1) \Delta_\tau^2 \Delta^{b-2} \right) \delta f \dots \\ + 2b \Delta_\tau \Delta^{b-1} \delta f_\tau + \Delta^b \delta f_{\tau\tau} \end{aligned} \quad (7.20d)$$

$$\begin{aligned} \alpha_{2,\tau\delta}^r = b \left(\Delta_{\tau\delta} \Delta^{b-1} + (b-1) \Delta_\tau \Delta_\delta \Delta^{b-2} \right) \delta f \dots \\ + b \Delta_\tau \Delta^{b-1} f + \Delta^b f_\tau + \dots \\ + b \Delta_\tau \Delta^{b-1} \delta f_\delta + \Delta^b \delta f_{\tau\delta} \end{aligned} \quad (7.20e)$$

$$\begin{aligned} \alpha_{2,\delta\delta}^r = b \left(\Delta_{\delta\delta} \Delta^{b-1} + (b-1) \Delta_\delta^2 \Delta^{b-2} \right) \delta f \dots \\ + b \Delta_\delta \Delta^{b-1} f + \Delta^b f_\delta + \dots \\ + b \Delta_\delta \Delta^{b-1} \delta f_\delta + \Delta^b \delta f_{\delta\delta} \end{aligned} \quad (7.20f)$$

7.2 Calculation of properties

The `mp1` class calculates all substance properties from temperature, density, α , and its derivatives. In this section, the formulae that are used to calculate the various properties are developed from first principles.

In this development, it will be useful to have this form of the first law:

$$T ds = de - \frac{p}{\rho^2} d\rho. \quad (7.21)$$

By definition,

$$f = e - Ts. \quad (7.22)$$

Its derivatives with respect to temperature and density may be simpli-

fied with some help from (7.21),

$$\left(\frac{\partial f}{\partial T}\right)_\rho = \left(\frac{\partial e}{\partial T}\right)_\rho - s - T \left(\frac{\partial s}{\partial T}\right)_\rho = -s \quad (7.23)$$

$$\left(\frac{\partial f}{\partial \rho}\right)_T = \left(\frac{\partial e}{\partial \rho}\right)_T - T \left(\frac{\partial s}{\partial \rho}\right)_T = \frac{p}{\rho^2}. \quad (7.24)$$

The properties below are calculated in a dimensionless form. In this way, this document is relieved from needing to be sensitive to units – including molar versus mass.

7.2.1 Pressure

The pressure (and compressibility factor) can be calculated explicitly from (7.24),

$$\begin{aligned} \frac{p}{\rho RT} &= \frac{\rho}{RT} \left(\frac{\partial f}{\partial \rho}\right)_T \\ &= \delta \alpha_\delta. \end{aligned} \quad (7.25)$$

7.2.2 Entropy

Entropy appears explicitly in (7.23), so it only needs to be transformed into terms of α . First, it is helpful to observe that derivatives with respect to temperature may be transposed into derivatives with respect to τ

$$\frac{\partial}{\partial T} = -\frac{\tau}{T} \frac{\partial}{\partial \tau}.$$

Therefore,

$$\begin{aligned} s &= -\left(\frac{\partial f}{\partial T}\right)_\rho \\ &= -\frac{\partial}{\partial T} RT\alpha \\ &= -R\alpha + R\tau\alpha_\tau \end{aligned} \quad (7.26)$$

So, normalized by R , entropy is

$$\frac{s}{R} = \tau\alpha_\tau - \alpha \quad (7.27)$$

7.2.3 Internal energy

Internal energy can be calculated in terms of a and s from (7.22) and (7.27),

$$\frac{e}{RT} = \frac{f}{RT} + \frac{s}{R} = \tau\alpha_\tau. \quad (7.28)$$

7.2.4 Enthalpy

Enthalpy is trivial to construct from (7.25) and (7.28). By definition,

$$\begin{aligned} \frac{h}{RT} &= \frac{e}{RT} + \frac{p}{\rho RT} \\ &= \tau\alpha_\tau + \delta\alpha_\delta. \end{aligned} \quad (7.29)$$

7.2.5 Gibbs free energy

Gibbs' energy is readily calculated from enthalpy and entropy; or from Helmholtz free energy.

$$\begin{aligned} \frac{g}{RT} &= \frac{h}{RT} - \frac{s}{R} = \frac{f}{RT} + \frac{p}{\rho RT} \\ &= \alpha + \delta\alpha_\delta. \end{aligned} \quad (7.30)$$

7.2.6 Specific heats

Constant-volume specific heat is obtained by differentiating internal energy by temperature.

$$\begin{aligned} c_v &= \frac{\partial e}{\partial T} = e_\tau \frac{d\tau}{dT} \\ &= -e_\tau \frac{\tau}{T} \end{aligned}$$

From (7.28), e is simply $RT_c \alpha_\tau$, so

$$\frac{c_v}{R} = -\tau^2 \alpha_{\tau\tau}. \quad (7.31)$$

Constant-pressure specific heat is not so simply obtained since pressure is not one of the independent variables of the formulation. Derivations for c_p usually begin with enthalpy, but it will become apparent

that the calculation benefits from the prior derivation for c_v . Therefore, from (1.18) we may write

$$c_p = \left(\frac{\partial e}{\partial T} \right)_p - \frac{p}{\rho^2} \left(\frac{\partial \rho}{\partial T} \right)_p.$$

However, because the **mp1** formulation is always in terms of temperature and density, derivatives with constant pressure must be transposed to derivatives on T and ρ ,

$$\left(\frac{\partial e}{\partial T} \right)_p = \left(\frac{\partial e}{\partial T} \right)_\rho + \left(\frac{\partial e}{\partial \rho} \right)_T \left(\frac{\partial \rho}{\partial T} \right)_p.$$

Of course, the same must be done for the derivative of density,

$$\left(\frac{\partial \rho}{\partial T} \right)_p = - \frac{\left(\frac{\partial p}{\partial T} \right)_\rho}{\left(\frac{\partial p}{\partial \rho} \right)_T}.$$

Substituting,

$$c_p = \left(\frac{\partial e}{\partial T} \right)_\rho - \left(\left(\frac{\partial e}{\partial \rho} \right)_T - \frac{p}{\rho^2} \right) \frac{\left(\frac{\partial p}{\partial T} \right)_\rho}{\left(\frac{\partial p}{\partial \rho} \right)_T}.$$

The first term (the derivative of internal energy with respect to temperature) is merely c_v . The second term may be simplified using (7.21), so

$$c_p = c_v - T \left(\frac{\partial s}{\partial \rho} \right)_T \frac{\left(\frac{\partial p}{\partial T} \right)_\rho}{\left(\frac{\partial p}{\partial \rho} \right)_T}.$$

Finally, this is in a form that can be conveniently substituted for

free energy. Using (7.23) and (7.24),

$$\begin{aligned}
\left(\frac{\partial s}{\partial \rho}\right)_T &= -\frac{\partial^2 f}{\partial T \partial \rho} = -\frac{R}{\rho}(\delta\alpha_\delta - \tau\delta\alpha_{\tau\delta}) \\
\left(\frac{\partial p}{\partial T}\right)_\rho &= \rho^2 \frac{\partial^2 f}{\partial T \partial \rho} = R\rho(\delta\alpha_\delta - \tau\delta\alpha_{\tau\delta}) \\
\left(\frac{\partial p}{\partial \rho}\right)_T &= 2\rho \left(\frac{\partial f}{\partial \rho}\right)_T + \rho^2 \left(\frac{\partial^2 a}{\partial \rho^2}\right)_T \dots \\
&= RT(2\delta\alpha_\delta + \delta^2\alpha_{\delta\delta})
\end{aligned}$$

So,

$$\frac{c_p}{R} = \frac{c_v}{R} + \frac{(\delta\alpha_\delta - \tau\delta\alpha_{\tau\delta})^2}{2\delta\alpha_\delta + \delta^2\alpha_{\delta\delta}} \quad (7.32)$$

7.2.7 Speed of sound

From its definition in Section 1.1.12, we write a dimensionless speed of sound

$$\begin{aligned}
\frac{a^2}{RT} &= \frac{1}{RT} \left(\frac{\partial p}{\partial \rho}\right)_s \\
&= \frac{1}{RT} \left(\frac{\partial p}{\partial \rho}\right)_T + \frac{1}{RT} \left(\frac{\partial p}{\partial T}\right)_\rho \left(\frac{\partial T}{\partial \rho}\right)_s. \quad (7.33)
\end{aligned}$$

The first term is obtained by differentiating (7.25) with respect to density, producing

$$\frac{1}{RT} \left(\frac{\partial p}{\partial \rho}\right)_T = 2\delta\alpha_\delta + \delta^2\alpha_{\delta\delta} \quad (7.34)$$

The second term in (7.33), we further sub-divide into two parts. The first part is also obtained by differentiating (7.25), giving

$$\left(\frac{\partial p}{\partial T}\right)_\rho = \rho R(\delta\alpha_\delta - \tau\delta\alpha_{\tau\delta}). \quad (7.35)$$

The second part involves the heating of the substance due to its isentropic compression. For this, we turn to

$$0 = Tds = de - \frac{p}{\rho^2}d\rho.$$

When we substitute (7.28) for e and (7.25) for p/ρ , we obtain

$$0 = -R\tau^2\alpha_{\tau\tau}dT + \frac{RT_c}{\rho_c}\alpha_{\tau\delta}d\rho - \frac{RT}{\rho}\delta\alpha_{\delta}d\rho$$

Solving for $dT/d\rho$, we obtain

$$\left(\frac{\partial T}{\partial \rho}\right)_s = -\frac{T}{\rho} \frac{\delta\alpha_{\delta} - \tau\delta\alpha_{\tau\delta}}{\tau^2\alpha_{\tau\tau}} \quad (7.36)$$

Finally, the dimensionless speed of sound is

$$\frac{a^2}{RT} = 2\delta\alpha_{\delta} + \delta^2\alpha_{\delta\delta} - \frac{(\delta\alpha_{\delta} - \tau\delta\alpha_{\tau\delta})^2}{\tau^2\alpha_{\tau\tau}}. \quad (7.37)$$

7.2.8 Liquid-vapor line

The Maxwell criteria provide a means for calculated the saturation line from a substance's properties – the two phases will be at the same temperature, pressure, and their Gibbs energy is equal. So,

$$g(T, \rho_L) = g(T, \rho_V) \quad (7.38a)$$

$$p(T, \rho_L) = p(T, \rho_V), \quad (7.38b)$$

where ρ_V and ρ_L are the vapor and liquid densities respectively.

For Newton-like numerical iteration algorithms, it is essential to provide reasonable initial guesses for ρ_L and ρ_V . Careless guesses can simply result in $\rho_L = \rho_V$, which is a classic example of a computer solution being *not wrong* but not helpful. The worst case, especially problematic near the critical point, is divergence.

7.3 The multi-phase collection

In PYroMat, the Span and Wagner models are implemented in classes `mp1` and `mp2`. These classes use wildly different approaches to establishing the state from the properties provided by the user, but there are quite a few common elements between them. See Section ?? for more information on PYroMat data, and the individual classes are described in the following sections.

7.3.1 The model coefficients

In version 3.0.1, the multi-phase coefficient organization changed significantly to make data entry faster and less error prone. Terms are organized into “groups” based on their characteristics. At the top level of the multi-phase data dictionaries, there are two entries; the `"IGgroup"` representing the ideal gas group of terms, and the `"Rgroup"` representing the residual group terms from Equation 7.4.

The `"IGgroup"` is a dictionary, which further allows entries `"logt"`, `"tlogt"`, `"group0"`, and `"group1"`, corresponding to the individual portions of Equation 7.10a. They are all optional, and their respective terms will be presumed zero if they are omitted. The `logt` and `tlogt` entries are scalar coefficients for $\log(\tau)$ and $\tau \log(\tau)$ respectively. The `group0` entry is a 1-D polynomial (see Section 8.2) defining $\alpha_0^\circ(\tau)$ from Equation 7.10b. The `group1` value is expected to be a list of two-element lists or tuples used to define the quantum terms in Equation 7.10c.

```
"IGgroup" : {  
    "logt" : <c0>,  
    "tlogt" : <c1>,  
    "group0" : <1D polynomial definition>,  
    "group1" : [[<m0>, <b0>], [<m1>, <b1>], <...>]  
}
```

Within the `"Rgroup"` dictionary, the `"group0"` member contains a list of 2D polynomials (see Section 8.1). Each a polynomial, $p_k(\tau, \delta)$, corresponds to an exponential term, $\exp(-\delta^k)p_k(\tau, \delta)$ in Equation ?? . They are listed in order, so the first polynomial definition corresponds to $k = 0$, the second to $k = 1$, and so on. The `"group1"` and `"group2"`

members are nested lists with each containing the necessary coefficients and exponents to construct the terms of α_1^r and α_2^r respectively.

```
"Rgroup" : {
  "group0" : [<p0>, <p1>, ...]
  "group1" : [
    [<t>, <d>, <b>, <a>, <gamma>, <epsilon>, <c>],
    ...
  ]
  "group2" : [
    [<a>, <b>, <beta>, <A>, <B>, <C>, <D>, <c>],
    ...
  ]
}
```

Prior to version 3.0.1, the residual and ideal gas groups required "Tscale" and "dscale" entries to define the values used to nondimensionalize the models. Their use is deprecated as it creates potentially problematic disagreements between the model's critical point and the critical point defined by the top-level data set. However, if they are specified, they are still honored for now. Future releases of PYroMat may change this behavior.

7.3.2 The mp1 class

The `mp1` class is the older of the two multi-phase classes in PYroMat. Its dictionary definition (in JSON) is in the form below. To ensure interoperable unit values, the dataset uses J for energy, kg m⁻³ for density, K for temperature, Pa for pressure, and J kg⁻¹ K⁻¹ for gas constant.

```
{
  "id"      : <substance ID string>
  "class"   : "mp2"
  "doc"     : <documentation string>
  "names"   : [<string>, <string>, ... # optional
  "inchi"   : <InChI string> # optional
  "cas"     : <CAS ID string> # optional
  "atoms"   : { <dictionary of atoms> } # optional
  "Tlim"    : [Tlow, Thigh]
  "plim"    : [plow, phigh]
  "dlim"    : [dlow, dhigh]
  "mw"      : <molecular weight>
```

```

"R"      : <Gas Constant Scalar>
"Tc"     : <critical temperature>
"dc"     : <critical density>
"pc"     : <critical pressure>
"Tt"     : <triple temperature>
"pt"     : <triple pressure>
"IGgroup" : { <ideal gas dictionary> }
"Rgroup"  : { <residual dictionary> }
"PSgroup" : { <saturation pressure model> }
"DSLgroup" : { <saturated liquid model> }
"DSVgroup" : { <saturated vapor model> }
}

```

Many original sources for equation-of-state models include polynomial fits for saturation properties that are so accurate, they can even be implemented without applying the Maxwell criteria. These are encoded in the "PSgroup", "DSLgroup", and "DSVgroup" entries as saturation properties as a function of temperature. The `mp1` class exposes these properties through methods: `ds`, `ps`, `Ts`, `es`, `hs`, and `ss`.

These formulae are calculated from dimensionless temperature,

$$\theta = \frac{T}{T_c}, \quad (7.39)$$

which is the inverse of τ . Each of the formulae for a saturation property, p , take one of four types:

0. A polynomial on θ

$$p(\theta) = \sum_k c_k \theta^{a_k} \quad (7.40)$$

This form is commonly used for liquid-solid saturation lines (not yet implemented in version 2.1.0).

1. A polynomial on $1 - \theta$

$$p(\theta) = \sum_k c_k (1 - \theta)^{a_k} \quad (7.41)$$

This form is usually used for the saturation pressure.

2. An exponential of a polynomial on $1 - \theta$

$$p(\theta) = \exp \left(\sum_k c_k (1 - \theta)^{a_k} \right) \quad (7.42)$$

This form is usually used for the liquid saturation density.

3. An exponential of a polynomial on $1 - \theta$ with a $1/\theta$ multiplier

$$p(\theta) = \exp \left(\frac{1}{\theta} \sum_k c_k (1 - \theta)^{a_k} \right) \quad (7.43)$$

This form is usually used for the vapor saturation density.

In data, there are three groups that are used to define the saturation pressure and densities.

```
"PSgroup" : {
  "fn" : <fn index>,
  "pscale" : <p scale>,
  "Tscale" : <T scale>,
  "coef" : <polynomial coefficients>
}

"DSLgroup" : {
  "fn" : <fn index>,
  "dscale" : <d scale>,
  "Tscale" : <T scale>,
  "coef" : <polynomial coefficients>
}

"DSVgroup" : {
  "fn" : <fn index>,
  "dscale" : <d scale>,
  "Tscale" : <T scale>,
  "coef" : <polynomial coefficients>
}
```

The `fn` parameter is an index (0-3) that selects the form of the formula to use. The `pscale` and `dscale` parameters are used to re-scale the dimensionless property calculated by the selected formula.

The `Tscale` parameter is used to normalize temperature to calculate θ . Finally, `coef` is a polynomial coefficient list described in Section 8.2.

These saturation models are evaluated by inner methods, `_dsV()`, `_dsL()`, and `_ps()`, which accept a temperature array as their only argument.

Saturation and ordinary property calculations are performed by inner methods, like `_e()`, `_cp()`, or `_s()`, accept temperature and density arrays, and calculate their respective states in the appropriate default units. The equivalent user-layer methods with no underscore, like `e()`, `cp()`, or `s()`, pass the user's raw arguments to `_argparse()`, which is responsible for determining the thermodynamic state the user is specifying.

If more information is needed, the in-line help for each of the methods gives a full description of its purpose and arguments.

7.3.3 The `mp2` class

The `mp2` class is the newer of the two multi-phase classes, first released in version 3.0.1. Its design was motivated by two goals. Firstly, the `mp1` class dependence on ancillary formulae to determine saturation states meant it was limited to sources that include those models. Second, there is a significant amount of redundant (and potentially contradictory) information designed into the `mp1` approach, which creates potential issues. For example, the equation of state, in theory, completely determines the saturation conditions, making the redundant saturation polynomials in (hopefully small) contradiction with the EOS model.

Its dictionary definition (in JSON) is in the form below. To ensure interoperable unit values, the dataset uses J for energy, kg m^{-3} for density, K for temperature, Pa for pressure, and $\text{J kg}^{-1} \text{K}^{-1}$ for gas constant.

```
{
  "id"      : <substance ID string>
  "class"   : "mp2"
  "doc"     : <documentation string>
  "names"   : [<string>, <string>, ... # optional
  "inchi"   : <InChI string>         # optional
  "cas"     : <CAS ID string>         # optional
```

```

    "atoms" : { <dictionary of atoms> } # optional
    "Tlim"  : [Tlow, Thigh]
    "plim"  : [plow, phigh]
    "dlim"  : [dlow, dhigh]
    "mw"    : <molecular weight>
    "R"     : <Gas Constant Scalar>      # optional
    "Tc"    : <critical temperature>
    "dc"    : <critical density>
    "Tt"    : <triple temperature>      # optional
    "IGgroup" : { <ideal gas dictionary> }
    "Rgroup" : { <residual dictionary> }
}

```

Most of the optional entries are for data features that will simply be disabled if they are not provided. For example, the `triple()` method raises an exception if the triple point temperature is not provided. The name, CAS, and InChI strings all provide data to their respective `names()`, `cas()`, and `inchi()` methods, and are not required for class instances to function properly.

When omitted, the ideal gas constant is calculated precisely from the `pm.units.const_Ru` value and the molecular mass, but allowing data authors to specify it explicitly is a vital feature for empowering the model to match precisely with the original publication. Many authors have optimized their models with rounded values for the gas constant, so to encode a model that agrees precisely with their model design, it is also important to use a truncated gas constant value.

Note that all pressure data (critical and triple) are absent, since they are determined from the equation of state model. The chief challenge posed by a theoretically minimal data set is how to implement it in a way that is not horribly slow or numerically unstable.

At retrieval using `get()` or the first time its `_argparse()` method is called, a `mp2` instance's `mp2._build()` method is called to construct a set of saturation and property look-up tables. The process takes some computation time, so it is not performed on all substances – only those retrieved for use. If necessary, the `_build()` method calls (in strict order) `_build_sattab()` then `_build_tab()`, which create instance attributes `_sattable` and `_table` respectively, which are dictionaries of property arrays.

The various property functions (especially `_argparse()`) makes heavy use of these tables to establish initial guesses very close to an acceptable answer. This allows the various iterative algorithms to abandon the `_hybrid1()` numerical tool, developed for global numerical stability in `mp1`, in favor of basic Newton-Rhapson iteration.

Constructing the saturation table is the first and slowest part of table construction. When it is complete, it contains entries, which are identically sized 1D Numpy arrays representing the columns of a traditional saturation table.

```
<substance>._sattable = {
    'T'   : <saturation temperature array>,
    'p'   : <saturation pressure array>,
    'g'   : <saturation gibbs energy>
    'dL'  : <saturated liquid density>,
    'dV'  : <saturated vapor density>,
    'eL'  : <saturated liquid int. energy>,
    'eV'  : <saturated vapor int. energy>,
    'hL'  : <saturated liquid enthalpy>,
    'hV'  : <saturated vapor enthalpy>,
    'sL'  : <saturated liquid entropy>,
    'sV'  : <saturated vapor entropy>,
    'fL'  : <saturated liquid free energy>,
    'fV'  : <saturated vapor free energy>
}
```

After initialization, the `mp2` instance has no explicit information on its saturation properties – it must deduce the table from the critical temperature and density and from the Maxwell criteria in Equations 7.38a and 7.38b. Especially near the critical point, guesses need to be extremely near to their actual values, lest they diverge. To ensure numerical stability, `_build_sattab()` calculates the table one point at a time, generating a guess by linear extrapolation from the last converged point.

Extrapolation to the next point is accomplished from derivatives of the Maxwell criteria. These represent two constraints with three variables, T , ρ_L , and ρ_V , so the tangent to the curve they form in 3-space can be calculated from their derivatives. For ease of notation,

$$g_{L,T} = \frac{\partial g(T, \rho_L)}{\partial T} \quad (7.44a)$$

$$g_{L,\rho} = \frac{\partial g(T, \rho_L)}{\partial \rho} \quad (7.44b)$$

$$p_{L,T} = \frac{\partial p(T, \rho_L)}{\partial T} \quad (7.44c)$$

$$p_{L,\rho} = \frac{\partial p(T, \rho_L)}{\partial \rho} \quad (7.44d)$$

The derivatives of the Maxwell criteria are, therefore,

$$g_{L,T} dT + g_{L,\rho} d\rho_L = g_{V,T} dT + g_{V,\rho} d\rho_V \quad (7.45a)$$

$$p_{L,T} dT + p_{L,\rho} d\rho_L = p_{V,T} dT + p_{V,\rho} d\rho_V. \quad (7.45b)$$

It is possible to calculate the differential of any two of the variables in terms of the third. For example, with respect to temperature,

$$\begin{bmatrix} g_{L,\rho} & -g_{V,\rho} \\ p_{L,\rho} & -p_{V,\rho} \end{bmatrix} \begin{bmatrix} d\rho_L \\ d\rho_V \end{bmatrix} = \begin{bmatrix} -g_{L,T} + g_{V,T} \\ -p_{L,T} + p_{V,T} \end{bmatrix} dT \quad (7.46)$$

and finally

$$\Delta = -g_{L,\rho} p_{V,\rho} + g_{V,\rho} p_{L,\rho} \quad (7.47)$$

$$\frac{d\rho_L}{dT} = \frac{1}{\Delta} [(g_{L,T} - g_{V,T}) p_{V,\rho} + (-p_{L,T} + p_{V,T}) g_{V,\rho}] \quad (7.48)$$

$$\frac{d\rho_V}{dT} = \frac{1}{\Delta} [(g_{L,T} - g_{V,T}) p_{L,\rho} + (-p_{L,T} + p_{V,T}) g_{L,\rho}] \quad (7.49)$$

Unfortunately, very near the critical point, these derivatives become singular, since all of the properties (and their derivatives) are very nearly equal. The problem is worsened, because the saturation curve is tangent to a line of constant temperature near the critical point. To avoid this problem, `_build_sattab()` uses similar derivatives with respect to vapor density, $dT/d\rho_V$ and $d\rho_L/d\rho_V$ instead, and transitions back to temperature far from the critical point.

To keep the tabulated points very nearly equally spaced throughout the table, the “distance” to the next saturation point is scaled by the

metric

$$0.02 = \sqrt{\left(\frac{\Delta T}{T_c}\right)^2 + \left(\frac{\Delta \rho_V}{\rho_c}\right)^2 + \left(\frac{\Delta \rho_L}{\rho_c}\right)^2}. \quad (7.50)$$

The “deltas” are rescaled in proportion to force the “step” size to be 0.02. Depending on the substance, this usually produces somewhere between 100 and 200 points in the saturation table.

After the deltas are calculated, Newton-Rhapson iteration is performed on the Maxwell criteria while holding vapor density constant or temperature constant. The transition between constant-density and constant-temperature iteration is made when the $-\Delta \rho_V / \rho_c < \Delta T / T_c$, indicating that the slope of the saturation curve is transitioning to be nearer to constant-density.

Finally, the temperature and density arrays are used to construct arrays of other saturated liquid and saturated vapor properties. Figure 7.1 shows the T, ρ saturation points automatically generated by this approach for acetone. The red points are saturated vapor, and the blue are saturated liquid. In this table, there are 140 individual points, including the critical and triple points.

Constructing the general property table requires a rectangular grid of temperature and density values. After `_build_tab()` is complete, the `_table` attribute of the substance instance appears as below.

```
<substance>._table = {
  'T' : <m-element 1D array>,
  'd' : <n-element 1D array>,
  'cI' : (<Tci>, <dci>),
  'p' : <m x n 2D array>,
  'e' : <m x n 2D array>,
  'h' : <m x n 2D array>,
  's' : <m x n 2D array>
}
```

The grid is constructed automatically from the saturation curve using a series of rules:

1. Density and temperature arrays must be in ascending order.

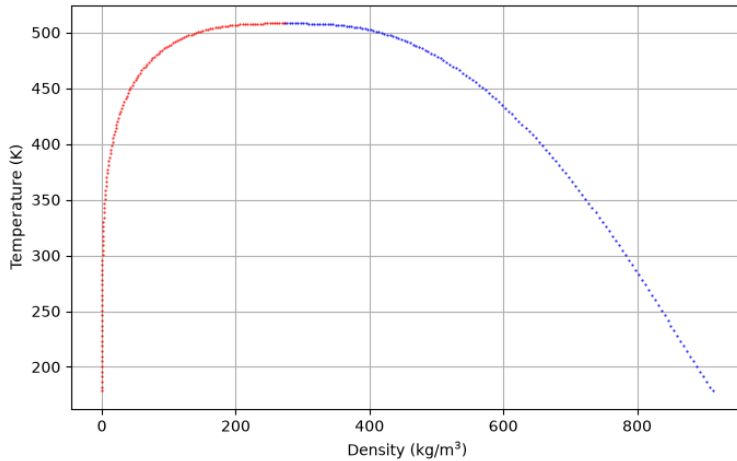


Figure 7.1: Points of the saturation table automatically generated by `mp2` for acetone.

2. No step between any two adjacent T or ρ values may be larger than 1% of the model's valid range.
3. The temperature and density arrays must span between their respective minima and maxima.
4. For each temperature below the critical temperature, the density array must contain values corresponding precisely to the liquid and vapor saturation densities at that temperature.
5. For each density between the vapor and liquid triple point densities, the temperature array must contain a temperature corresponding precisely to the corresponding saturation temperature.
6. The critical temperature and density must be represented precisely in their respective arrays.

To obey these rules, it is essential that the saturation table is already available.

From the resulting temperature and density grid values, all properties are evaluated in two-dimensional arrays, so that

$$f_{i,j} \equiv f(T_i, \rho_j) \quad (7.51)$$

The location of the critical point in the tables is recorded in the "cI" entry, as a tuple with the temperature index first and the density index second.

```
>>> ii, jj = _table['cI']
>>> Tc = _table['T'][ii]
>>> dc = _table['d'][jj]
>>> pc = _table['p'][ii, jj]
```

Thanks to rules 4 and 5 above, it is always trivial to identify grid points precisely on the saturation line. If we were to move k indices below the critical temperature, the saturation properties are

```
>>> ii, jj = _table['cI']
>>> k = 5
>>> Ts = _table['T'][ii-k]
>>> dsL = _table['d'][jj+k]
>>> dsV = _table['d'][jj-k]
>>> hsL = _table['h'][ii-k, jj+k]
>>> hsV = _table['h'][ii-k, jj-k]
```

This approach naturally produces a high density of grid lines in regions where the saturation curves cause properties to be very steep. Figure 7.2 shows the internal energy table automatically generated for acetone. There is a tight grouping of temperature grid lines near the critical point, but there is also a remarkably sharp vertical slope under “the dome” near the vapor line caused by the severely dissimilar densities between liquids and vapors far below the critical point. It only takes a small amount of vapor to significantly drop the density, while adding relatively little to the mixture’s energy. It is not until the density has diminished to a tiny fraction (often around 0.2% or so) of its liquid value that the higher vapor energy dominates. Note that this grid generation strategy naturally densely populates this region with grid lines, producing a well formed, though nearly vertical surface.

Table lookup primitives are responsible for casing out the various ways to interrogate the tables. Numpy and Scipy offer interpolation

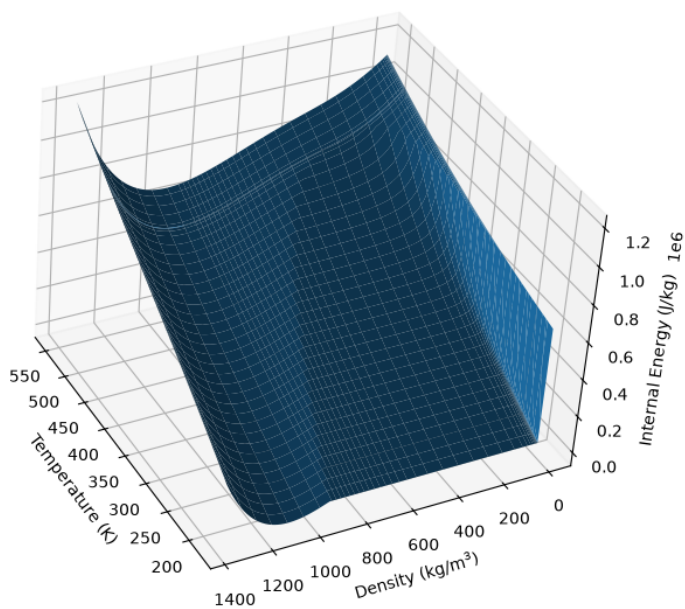


Figure 7.2: Internal energy plotted against temperature and density with the grid values automatically generated by `mp2`.

algorithms, but they require the source data to be sorted. This is fine when working from temperature and density, but they do not permit “reverse” table lookups. The “mapsearch” algorithms address this issue. Since their arrays are one-dimensional, the saturation tables use `_mapsearch1()`. The general property tables, on the other hand, use `_mapsearch2()`, `_Tmapsearch2()`, and `_dmapsearch2()`. Their inline documentation offers a detailed description of how their algorithms work.

Iterative routines in `mp2` are not in specialized methods like `_hybrid1` described in Chapter 8. Because Newton-Raphson algorithms are so simple to code, a myriad of specialized inner routines are responsible for performing iteration custom to the particular job. These are organized by which property is held constant while others are allowed to change. The saturation algorithms include `_Tsatiter()`, `_dVsatiter()`, `_dLsatiter()`, `_dsatiter()` (which preserves mixture density), `_psatiter()`, `_gsatiter()`, and `_satiter2()` (which preserves two arbitrary mixture properties). The generalized property iteration algorithms require only `_Titer()` (holding temperature constant), `_diter()` (holding density constant), and `_iter2()` (simultaneous iteration on both temperature and density).

Inner property methods in `mp2` behave fundamentally differently from their equivalents in `mp1`. Rather than accepting a state definition in terms of temperature and density, the inner methods like `_e()`, `_cp()`, `_h()`, or `_s()` accept the explicit values of $\alpha(\tau, \delta)$ and its derivatives, as returned by a new inner method, `_ff()`. Though this makes the call signatures more cumbersome, it saves countless back-end methods from redundantly performing the most computationally expensive part of the process – evaluation of α . Instead, methods that require multiple simultaneous properties (like while iterating on saturation states) merely make a single call to `_ff()` and share its output with the relevant inner property methods. All of this nuance is opaque to the user – the user layer property methods function just like the other `PYroMat` classes.

Table 7.1: Benchmarking tests comparing **mp1** against **mp2** for determining the state, t_1 , and evaluating the state properties, t_2 . Values are in microseconds.

	N		10	100	1,000	10,000	100,000
(T, d)	mp1	t_1	88	91	143	565	4,862
		t_2	1,841	1,930	2,961	9,947	110,323
	mp2	t_1	11	2,824	4,524	6,536	16,979
		t_2	741	1,530	2,152	6,258	72,040
(T, p)	mp1	t_1	14,092	15,160	22,958	80,303	843,870
		t_2	1,679	1,270	1,992	6,620	87,956
	mp2	t_1	2,358	4,268	17,889	115,795	1.16×10^6
		t_2	739	765	1,295	1,138	66,845
(s, p)	mp1	t_1	190,888	270,547	399,682	1.10×10^6	9.03×10^6
		t_2	1,347	1,006	3,757	19,521	223,192
	mp2	t_1	2,229	15,008	74,794	643,537	6.42×10^6
		t_2	748	1,570	571	4,775	61,988

7.3.4 Benchmarking

Improved computational speed was always a goal in the development of **mp2**, but it was secondary to broadening the number of substances included in PYroMat. To gauge the relative strengths and weaknesses of the two classes, benchmarking was performed by passing randomly generated property arrays in different combinations. The process of calculating properties is divided into two steps: (1) determination of the thermodynamic state in terms of temperature and density (t_1), and (2) calculation of the requested properties at that state (t_2).

The relative computational costs of these steps were measured as the average of ten repeated calls to `_argparse()`, followed by ten repeated calls to `state()` with the same datasets. The time to calculate properties was assessed as the time required for `state()` minus the time required for `_argparse()`. Tests were performed on a single core of an AMD Ryzen 9 7900 with array sizes ranging from 10 to 100,000. The results for H₂O (representing **mp1**) and N₂ (representing **mp2**) are shown in Table 7.1

In general, **mp2** scales more poorly with array size than **mp1**. This

is expected, since many of the mapsearch algorithms are serialized in for loops. When (T, ρ) property pairs are specified, no iteration is required, so the improved initial guesses of `mp2` offers no benefit. Instead, the extra overhead required to look up two-phase mixture information causes it to be somewhat slower.

It is seen that for any property combination other than (T, ρ) , the time required to determine the thermodynamic state, t_1 , dominates the computational cost. When iteration on one property is required (e.g. when (T, p) property pairs are specified), `mp2` is about five times as fast as `mp1`, slowing to break even between 1,000 and 10,000 elements. On huge datasets, `mp1`'s better vectorization makes it somewhat faster, but not much compared to `mp2`'s advantage in small datasets.

When iteration on two simultaneous properties is required (as in the case of (s, p)) `mp2` is about 100 times faster for small datasets. That advantage erodes in large data sets, but `mp1` never managed to overtake `mp2` in these tests. The `mp1` algorithms suffers severely here because it does not perform true 2D numerical inversion. For numerical stability, nested `_hybrid1` algorithms provide a horribly inefficient, albeit extremely numerically stable facsimile of 2D inversion. As a result, `mp2` is far superior in these cases.

Except for users calling the property methods inside of a higher iterative process, these magnitudes of numerical costs are mostly insignificant. Most users work on small datasets, and will not be bothered by the difference between a 1 millisecond versus a 10 millisecond calculation. This comparison is entirely for the benefit of users considering doing something a little more adventurous with these models.

7.4 Sources implemented

Table 7.2 lists the multi-phase models currently implemented in PYroMat as of version 2.2.0. Entries marked with an asterisk were not yet complete for the 2.2.0 release, but are planned to be implemented later.

Table 7.2: Sources for multi-phase data used in PYroMat. Substances with a * were not yet implemented in version 2.2.0, but they are planned for a later release.

Argon*	Ar	[22]
Carbon dioxide	CO ₂	[20]
Deuterium*	D ₂	[23]
Hydrogen*	H ₂	[24]
Nitrogen	N ₂	[25]
Oxygen	O ₂	[14]
Water	H ₂ O	[21]
R-125*	C ₂ HF ₅	[26]
R-134a	C ₂ H ₂ F ₄	[27]
R-143a*	C ₂ H ₂ F ₃	[28]
R-245fa*	C ₃ H ₃ F ₅	[29]
R-1234yf*	C ₃ H ₂ F ₄	[30]
R-1234ze	C ₃ H ₂ F ₄	[31]

Chapter 8

Numerical routines

PYroMat is designed with a number of custom back-end numerical routines. Even though low-level custom numerical routines composed in plain Python are unlikely to ever compete well with compiled numerical binaries that have been developed over decades, there are distinct advantages to this approach.

- Reducing the number of dependencies makes the installation simpler and less likely to exhibit mysterious problems on other systems,
- The installation footprint is minimal - not requiring entire tertiary libraries,
- Algorithms can be customized to conform to the inputs and outputs required by the PYroMat.
- Numerical algorithms can be (and have been) tuned specifically to provide consistent convergence for the types of numerical problems encountered in thermodynamic property evaluation.
- The PYroMat numerical routines are specially designed to iterate on *parts* of a Numpy array at a time while still benefiting from Numpy's compiled algorithm speed.

A package written in plain Python is unlikely to ever be used in numerically intense applications like computational fluid dynamics, so

we have consistently prioritized reliability over speed. Still, that's no reason to be wasteful with clock cycles! Even on arrays with thousands of elements, the most cumbersome of PYroMat property calculations reliably converge in fractions of a second in testing. The author has yet to hear of a use case for which this performance is unacceptably slow. Until that changes, this is how the PYroMat back-end will continue to do things.

8.1 Polynomials of two variables

The evaluation of the multi-phase models requires efficient evaluations of polynomials of two variables. These expansions are typically of the form

$$P(X, Y) = \sum_{i,j} c_{i,j} X^i Y^j \quad (8.1)$$

where a and b are real coefficients such that i and j are integer indices.

8.1.1 Modifying polynomials for non-integer and negative powers

Fractional and negative exponents are also possible within this framework if we were to accept input values x , and y , and adjust them according to pre-exponentials a and b ,

$$X = x^a \quad (8.2)$$

$$Y = y^b. \quad (8.3)$$

Then, if the polynomial were also multiplied by post-exponential terms, x^α , and y^β , the new polynomial formed is

$$p(x, y) = x^\alpha y^\beta P(X, Y). \quad (8.4)$$

This results in a polynomial,

$$p(x, y) = \sum_{i,j} c_{i,j} x^{ai+\alpha} y^{bj+\beta}, \quad (8.5)$$

but in a form that can be efficiently evaluated using integer exponents, i and j .

Non-integer exponents are problematic to polynomials, because they require the use of a **power** function, while integer exponents can be evaluated exclusively with multiplication. For example, consider a polynomial,

$$\begin{aligned} p(x, y) &= 2x^{-2} + x^{0.1}y + xy^2. \\ X &= x^{0.1} \\ p(x, y) &= x^{-2} (2 + X^{21}y + X^{40}y^2) \end{aligned}$$

Using scaling values $a = 0.1$ and $\alpha = -2$, the polynomial can be adjusted to be evaluated with only integer exponents.

However, this also illustrates a potential problem in the approach. For individual elements a call to Numpy's **power** function costs about double a single multiplication, but for larger arrays (when the difference is most important) the difference diverges. Figure 8.1 shows a timing study of **power**, *****, and ****** operations on arrays of random positive floating point numbers. For small arrays, the interpreter's overhead dominates the measurement, but above 100 elements, the two adopt classical power curves.

Because of the dominance of numerical overhead, repeated multiplication operations can be more expensive than a call to **power** on small arrays. However, it should be emphasized that judicious use of computational effort is really only critical on large arrays, anyway. For arrays with 100 elements, **power** operations are about 10 times as expensive, but **power** function calls on 10,000-element arrays are roughly equivalent to 100 multiplications.

This leads to a careful choice for arrays with large exponents (or tiny fractional values). The large exponents in the last example can be mitigated at the cost of added overhead and complexity by splitting the polynomial into two separate polynomials, each with its own scaling.

$$\begin{aligned} p(x, y) &= 2x^{-2} + x^{0.1}y + xy^2. \\ &= x^{-2} (2 + x^3y^2) + x^{0.1}y \end{aligned} \tag{8.6}$$

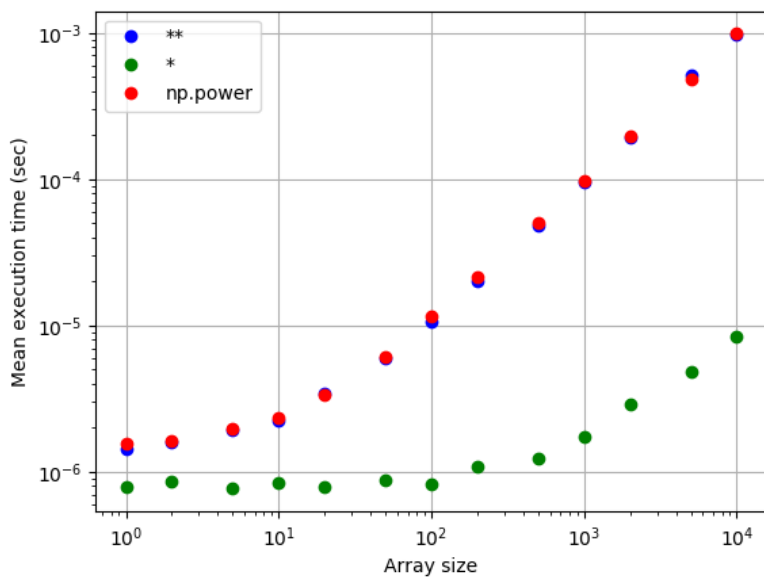


Figure 8.1: Approximate calculation time of the numpy `power` function, `**` operation, and the `*` operation on arrays of random positive floating point values.

8.1.2 Representation of polynomials in data

The `json` data representation of a polynomial on two dimensions uses a dictionary to describe pre- and post-exponents, the coefficients, and their corresponding exponents for multiple polynomial parts. Prior to version 3.0.1, polynomials were defined using a series of nested lists, but the transition to dictionaries dramatically improved human readability of the data, and made manual entry dramatically faster with fewer typos.

```
poly = {  
    # Pre- and post-exponents are optional  
    'pre': [<a>, <b>],  
    'post': [<alpha>, <beta>],  
    # The mandatory coefficient list  
    'coef': [  
        [<i0>, <j0>, <c0>],  
        [<i1>, <j1>, <c1>],  
        <...>  
    ]  
}
```

Each pair of non-negative integer exponents, i_0 , j_0 , specifies the existence of an individual term, and the corresponding c_0 indicates its floating point coefficient. The exponents must be sorted in descending order by i and then sorted by descending j ; so every i will be less than or equal to the one above, and each j will be less than the j values above with the same values of i . This approach is a “sparse” polynomial definition, where only data for the terms with non-zero coefficients are provided. By contrast a “dense” definition requires a fully populated matrix of coefficients, where each row and column corresponding to a non-negative integer power of x or y . In the sparsely populated polynomials common to the models used in PYroMat, sparse methods are easily more efficient.

It is often preferable (and sometimes essential) to specify different pre- and post-exponents for portions of a polynomial. The most common motivation comes from terms with floating point exponents that are not efficiently represented as an integer multiple of the same rational. For example $\tau^{0.221}$ and $\tau^{0.5}$ might be represented with a pre-

exponent of 0.001, resulting in terms with exponents 221 and 500. Such high exponents are numerically problematic and horribly inefficient. In these cases, sub-polynomial dictionaries can be arranged inside a list – each dictionary may specify its own pre- and post-exponents.

```
poly = [# This begins the list of sub-polynomials
        {# This begins the first sub-polynomial
          # Pre- and post-exponents are optional
          'pre': [<a>,<b>],
          'post': [<alpha>, <beta>],
          # The mandatory coefficient list
          'coef': [
              [<i0>, <j0>, <c0>],
              [<i1>, <j1>, <c1>],
              <...>
          ]
        },
        {# This begins the second sub-polynomial
          'pre': [<a>,<b>],
          'post': [<alpha>, <beta>],
          # The mandatory coefficient list
          'coef': [
              [<i0>, <j0>, <c0>],
              [<i1>, <j1>, <c1>],
              <...>
          ]
        },
        <...more sub-polynomials...>
    ]
```

It is usually a good idea to minimize the number of sub-polynomials in a data model whenever possible. It may be prudent to experiment with different realizations of the same polynomial to see which is most computationally efficient while still giving accurate results. The latter becomes an issue when pre-exponents shrink to be very tiny. Terms like the one below can impart significant numerical error, so the added time for an extra floating point exponent might be worth it.

$$\left(x^{\text{tiny number}}\right)^{\text{HUGE INTEGER}}$$

8.1.3 Efficient evaluation of the polynomial

As we have seen in the previous section, a polynomial can be adjusted so that

$$p(x, y) = \sum_{i,j} c_{i,j} x^i y^j \quad (8.7)$$

where all values of i and j are non-negative integers. However, evaluating each term individually requires two expensive calls to a `pow` function and two floating point multiplications.

The widely accepted method for evaluating a polynomial of one variable is to construct a recursive expansion

$$q(y) = c_0 + y(c_1 + y(c_2 + y(\dots \quad (8.8)$$

If there are n coefficients, then this amounts to only n multiplications with no `pow` calls. In order to extend this algorithm to two variables, more elegant notation will be helpful. If we name the intermediate value calculated in the process of these recursions q_j , then a polynomial with n terms implies the series

$$q_n = c_n \quad (8.9)$$

$$q_j(y) = c_j + y q_{j+1}(y) \quad (8.10)$$

$$q_0(y) = q(y). \quad (8.11)$$

This is a series beginning with $j = n$ and proceeding backwards to $j = 0$. The final function in the series, q_0 is also the desired polynomial, $q(y)$. In practice, there is no need to keep the old values of q , so a single register may be used to hold the latest value.

How can this be extended to a polynomial of two variables? We may consider the polynomials to be nested; the evaluation of a polynomial on y determines the individual coefficients for a polynomial on x .

$$p(x, y) = \sum_i q_i(y) x^i \quad (8.12)$$

$$q_i(y) = \sum_j c_{i,j} y^j \quad (8.13)$$

We only need a minor modification to the intermediate values for the x polynomial since there will be a separate expansion for each value of i . If there are n j terms,

$$q_{i,n}(y) = c_{n,j} \quad (8.14a)$$

$$q_{i,j}(y) = c_{i,j} + y q_{i+1,j}(y) \quad (8.14b)$$

$$q_{i,0}(y) = q_i(y). \quad (8.14c)$$

If there are m x -terms,

$$p_m(x, y) = q_m(y) \quad (8.15a)$$

$$p_i(x, y) = q_i(x) + y p_{i+1}(x, y) \quad (8.15b)$$

$$p_0(x, y) = p(x, y). \quad (8.15c)$$

8.1.4 Efficient evaluation of derivatives

The partial derivatives of the polynomial can be efficiently evaluated along with the polynomial itself. To relax the already cumbersome notation, the functional dependencies (y) and (x, y) will be dropped. For the purpose of thermodynamic property evaluation, only the first two derivatives are needed. We will denote the partial derivatives with subscripts so that

$$\frac{\partial p_{i,j}}{\partial x} = p_{i,j|x} \quad \frac{\partial^2 p_{i,j}}{\partial x^2} = p_{i,j|xx}.$$

Let us begin with the simpler task of calculating the derivatives of q_i with respect to y .

$$q_{i,n|y} = 0 \quad (8.16a)$$

$$q_{i,j|y} = q_{i+1,j} + y q_{i+1,j|y} \quad (8.16b)$$

$$q_{i,0|y} = q_{i|y} \quad (8.16c)$$

$$q_{i,n|yy} = 0 \quad (8.17a)$$

$$q_{i,j|yy} = 2q_{i,j+1|y} + y q_{i,j+1|yy} \quad (8.17b)$$

$$q_{i,0|yy} = q_{i|yy} \quad (8.17c)$$

The derivatives on p are constructed somewhat differently because they can be in both x and y . Beginning with y ,

$$p_{n|y} = 0 \quad (8.18a)$$

$$p_{j|y} = q_{i|y} + x p_{j+1|y} \quad (8.18b)$$

$$p_{0|y} = p_y \quad (8.18c)$$

$$p_{m|yy} = 0 \quad (8.19a)$$

$$p_{i|yy} = q_{i|yy} + x q_{i+1|yy} \quad (8.19b)$$

$$p_{0|yy} = p_{yy} \quad (8.19c)$$

The derivatives on x appear

$$p_{m|x} = 0 \quad (8.20a)$$

$$p_{i|x} = p_{i+1} + x p_{i+1|x} \quad (8.20b)$$

$$p_{0|x} = p_x \quad (8.20c)$$

$$p_{n|xx} = 0 \quad (8.21a)$$

$$p_{i|xx} = 2p_{i+1|x} + x p_{i+1|xx} \quad (8.21b)$$

$$p_{0|xx} = p_{xx} \quad (8.21c)$$

Finally, the cross-term (both x and y) appears

$$p_{n|xy} = 0 \quad (8.22a)$$

$$p_{i|xy} = p_{i+1|y} + y p_{i+1|xy} \quad (8.22b)$$

$$p_{0|xy} = p_{xy} \quad (8.22c)$$

8.1.5 Implementation of the algorithm

In practice, this cumbersome notation can be drastically simplified in code because it is not necessary to distinguish between the subscripts

of p and q , provided care is taken not to overwrite a value before it is needed.

In most practical polynomials of two variables of order m on x and order n on y , there are $(m+1)(n+1)$ possible coefficients, but many (if not most) of them may be zero. As a result, storing the coefficient array in a 2D array (or matrix) format is not efficient. Instead, PYroMat takes an approach closer to coordinate sparse matrix storage.

If we have one-dimensional arrays of polynomial coefficients, c_k , and exponents, i_k and j_k , the polynomial will be constructed as

$$p(X, Y) = \sum_k^{N-1} c_k X^{i_k} Y^{j_k}. \quad (8.23)$$

In this way, the polynomial

$$p(X, Y) = -0.1X^2 + XY + 0.5Y^2 - Y - 0.2 \quad (8.24)$$

may be represented by

$$i = [2, 1, 0, 0, 0] \quad (8.25)$$

$$j = [0, 1, 2, 1, 0] \quad (8.26)$$

$$c = [-0.1, 1, 0.5, -1, 0.2] \quad (8.27)$$

For the algorithm to function efficiently, it is reasonable to impose some prior sorting of the exponent values. Since the series developed in the previous section requires that we interact with higher-order terms first, let us assert that the polynomial should be expressed in order of descending exponents on X and then Y .

In Algorithm 1, an outer loop over the range of values of i and an inner loop on the range of values of j considers all of the possible terms in the polynomial. If coefficients are absent from the arrays (if the i, j pair is not found), then the term is not included in the polynomial (the coefficient is zero). Starting with the maximum value for each exponent, the indices are reduced incrementally until the i, j combination corresponding to the next row (k) is found.

Algorithm 1 Efficient evaluation of a polynomial of two variables

```
1: procedure _POLY2( $x, y, i, j, c$ ) ▷  $i, j, c$  are arrays
2:    $p, p_x, p_y, p_{xx}, p_{xy}, p_{yy} \leftarrow 0$  ▷ Initialize the results with zero
3:    $i_{max} \leftarrow i[0]$  ▷ Detect the maximum  $i$  value
4:    $k \leftarrow 0$  ▷  $k$  is our place in the coefficient array
5:   for  $ii = i_{max}$  to 0 do ▷ All possible  $i$  values
6:     if  $k < N$  and  $i[k] == ii$  then ▷ Is there an  $x^{ii}$  term?
7:        $j_{max} \leftarrow j[k]$  ▷ Detect the maximum  $j$  value
8:        $q, q_y, q_{yy} \leftarrow 0$  ▷ Initialize the inner  $q$  result
9:       for  $jj = j_{max}$  to 0 do ▷ All possible  $j$  values
10:         $q_{yy} \leftarrow 2q_y + yq_{yy}$ 
11:         $q_y \leftarrow q + yq_y$ 
12:        if  $k < N$  and  $a_k$  is  $i$  and  $b_k$  is  $j$  then ▷ Is there a
           $x^{ii}y^{jj}$  term?
13:           $q \leftarrow c_k + yq$ 
14:           $k \leftarrow k + 1$ 
15:        else ▷ Term not in polynomial
16:           $q \leftarrow yq$ 
17:        end if
18:      end for
19:       $p_{yy} \leftarrow q_{yy} + xp_{yy}$ 
20:       $p_{xx} \leftarrow 2p_x + xp_{xx}$ 
21:       $p_{xy} \leftarrow p_y + xp_{xy}$ 
22:       $p_x \leftarrow p + xp_x$ 
23:       $p_y \leftarrow q_y + xp_y$ 
24:       $p \leftarrow q + xp$ 
25:    else ▷ Term not in polynomial
26:       $p_{yy} \leftarrow xp_{yy}$ 
27:       $p_{xx} \leftarrow 2p_x + xp_{xx}$ 
28:       $p_{xy} \leftarrow p_y + xp_{xy}$ 
29:       $p_x \leftarrow p + xp_x$ 
30:       $p_y \leftarrow xp_y$ 
31:       $p \leftarrow xp$ 
32:    end if
33:  end for
34:  return  $p, p_x, p_y, p_{xx}, p_{xy}, p_{yy}$ 
35: end procedure
```

8.2 Polynomials in one dimension

Polynomials of one dimension benefit from the same approach as two-dimensional polynomials, but the algorithm is far simpler. All the formulae and algorithms that apply to q in the last section may simply be repurposed for a single dimensional polynomial.

The only topic that really requires special treatment is the format used in data. One-dimensional polynomials are identical to two-dimensional polynomials, except (1) the pre- and post-exponent values are scalars instead of two-element lists, and (2) each term only needs two parameters instead of three.

```
poly = {  
    # Pre- and post-exponents are optional  
    'pre': <a>,  
    'post': <alpha>,  
    # The mandatory coefficient list  
    'coef': [  
        [<i0>, <c0>],  
        [<i1>, <c1>],  
        <...>  
    ]  
}
```

Just like with 2-dimensional polynomials, 1D polynomials allow sub-polynomials, so multiple pre- and post-exponentials can be used within the same polynomial.

```
coef = [  
    {  
        # sub-polynomial 1  
    },  
    {  
        # sub-polynomial 2  
    }  
]
```

8.3 Iteration with `iter1`

Most of the classes have a member that implements some variation of the Newton-Rhapon variant `_iter1` method. It is fast, and it has excellent convergence characteristics in most applications except the numerically irksome cases found in the multi-phase models.

Given a function on one variable, f , the `_iter1` algorithm seeks a value, x , between upper and lower limits, x_{max} and x_{min} so that $f(x)$ is equal to a reference value, y .

Given a guess, x_k , traditional Newton-Rhapon iteration calculates a next guess, x_{k+1} by extrapolating linearly,

$$\Delta x = \frac{y - f(x_k)}{f'(x_k)} \quad (8.28)$$

$$x_{k+1} = x_k + \Delta x. \quad (8.29)$$

This is simply repeated until $y - f(x_k)$ is sufficiently small.

However, in problems with substantial nonlinearities, the iteration can diverge badly like in Figure 8.2. Worse still, these intermediate guesses can even wander to illegal property values.

The `_iter1` algorithm addresses the problem by allowing the calling property method to establish upper and lower boundaries on the value, x . If a guess wanders outside of the valid range, the size of the step, Δx , is halved repeatedly until the next guess is back in the valid range. The algorithm is usually initialized externally with some first guess for x .

Algorithm 2 gives a simplified version of the algorithm implemented in PYroMat. See the in-line documentation and comments for a more detailed description.

8.4 Iteration with `hybrid1`

Multi-phase properties present severe numerical challenges since the phase change represents an abrupt jump in properties, and the iteration is inherently two-dimensional. The `_iter1` algorithm prevents the solution from diverging, but it often fails to converge in multi-phase substance inversion problems.

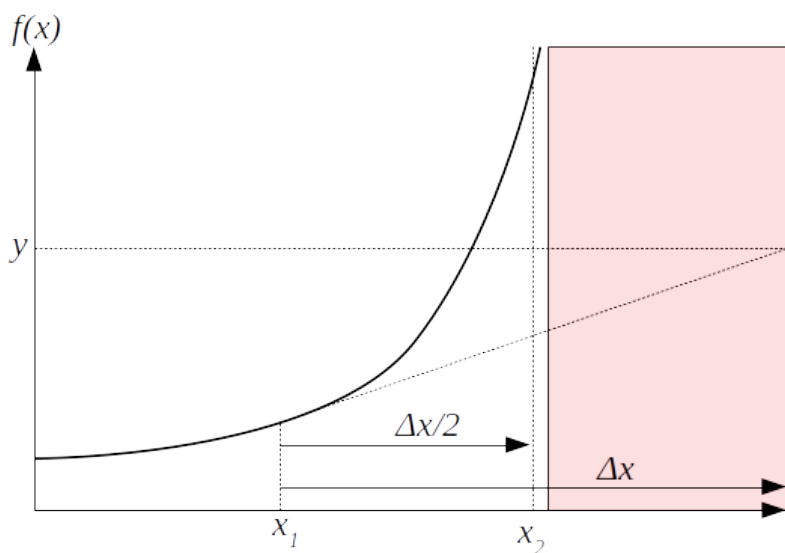


Figure 8.2: An example of a system where classical Newton iteration diverges to illegal values. The

Algorithm 2 ITER1: Modified bounded Newton-Rhapson iteration,
 $y = f(x)$

procedure _ITER1($f, y, x_{min}, x_{max}, x, \epsilon$)

$x_k \leftarrow x$ ▷ Initialize the current guess

$y_k \leftarrow f(x)$ ▷ Evaluate the function

$y'_k \leftarrow f'(x)$

while $|y - y_k| > \epsilon$ **do** ▷ While error is large

$\Delta x \leftarrow (y - y_k)/y'_k$ ▷ Calculate step size

$x_{k+1} \leftarrow x_k + \Delta x$ ▷ Calculate the next guess

▷ If x_k is out of bounds

while $x_{k+1} > x_{max}$ or $x_{k+1} < x_{min}$ **do**

$\Delta x \leftarrow \Delta x/2$ ▷ Halve Δx

$x_{k+1} \leftarrow x_k + \Delta x$ ▷ Update the guess

end while

$x_k \leftarrow x_{k+1}$ ▷ Accept the new guess

$y_k \leftarrow f(x_k)$ ▷ Evaluate the function

$y'_k \leftarrow f'(x_k)$

end while

return x_k

end procedure

8.4.1 Bisection iteration

Bisection is a classic approach to nasty inversion problems where Newton iteration fails. If upper and lower bounds, x_a and x_b are found so that $f(x_a) < y < f(x_b)$, then a continuous function must have a value somewhere between x_a and x_b such that $f(x) = y$. If we were to divide the domain in half $x_c = (x_a + x_b)/2$ and evaluate the function there, its value will either be above or below y , so one of the two values could be replaced by x_c . In this way, the domain between x_a and x_b is reliably cut in half with every iteration step, no matter how bizarrely $f(x)$ behaves.

This process just has to be repeated until the distance between the upper and lower bounds have shrunk to be so small that the numerical uncertainty is acceptable. Figure 8.3 shows three steps using this process. The vertical space occupied by each blue box represents the shrinking uncertainty for the value of $f(x)$ and the horizontal space represents the shrinking uncertainty in x .

While Newton iteration might converge in only a few steps on a nearly linear function, bisection can take tens of steps depending on the ratio of the initial domain and the acceptable error range. Since the domain is divided by two every time, it is easy to calculate the number of iterations to obtain a certain domain size.

$$N = \log_2 \frac{|x_b - x_a|}{\epsilon} \quad (8.30)$$

If the initial domain were a temperature range of 0 to 1000 Kelvin, and the acceptable error were .01 Kelvin, 17 iterations are required. If the acceptable error were tightened to .001 Kelvin, 20 iterations are required!

8.4.2 The hybrid1 candidates

Literature on numerical methods is littered with different flavors of hybrid algorithms that try to benefit from the guaranteed convergence of bisection and the speed of Newton-Raphson. This algorithm is specifically designed for application on the types of functions that appear in thermodynamic properties.

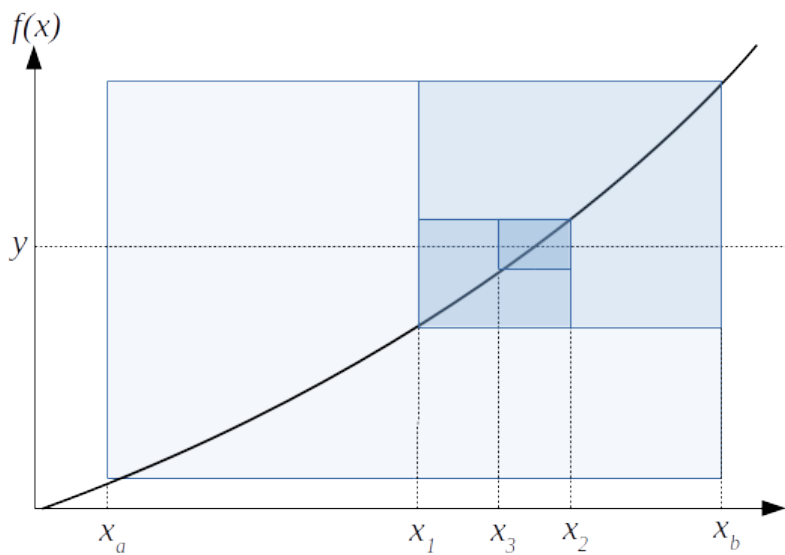


Figure 8.3: A depiction of the bisection iteration algorithm on a hypothetical function.

This hybrid approach begins by bracketing a solution with guesses, x_a and x_b , just like a bisection algorithm. Then, we calculate three candidates for the next guess, x_c :

1. Newton iteration from x_a :

$$x_1 = x_a + \frac{y - f(x_a)}{f'(x_a)} \quad (8.31)$$

2. Newton iteration from x_b :

$$x_2 = x_b + \frac{y - f(x_b)}{f'(x_b)} \quad (8.32)$$

3. Bisection iteration from x_a and x_b :

$$x_3 = \frac{x_a + x_b}{2} \quad (8.33)$$

These three parallel processes for calculating candidates are shown in Figure 8.4. As the next sections will explore, the next step is to select one. Once the function is evaluated there, the boundaries can be updated (just like in bisection) and convergence can be tested.

One could pose dozens of ideas for how information about these three candidates might be used to select a best guess, but only two are implemented in PYroMat. They are discussed in the sections below.

It may appear that each iteration requires two function evaluations, but that is not so. The next guess will be used to replace either the upper or lower bound (and its candidate). The function values and the candidate for the other bound can be retained.

8.4.3 Standard candidate selection

Once the candidate guesses are calculated, the next step is to select a single candidate for the next step of iteration. In general, the Newton steps at the extremes of the domain are unlikely to be very good until the algorithm is very close to the solution. If the two Newton iteration steps produce a guess very close to each other, there is a good chance that they can be trusted. Otherwise, it is probably best to rely on the bisection step. Above all, it is vital that no guess ever lie outside of the boundaries where a solution is known to exist.

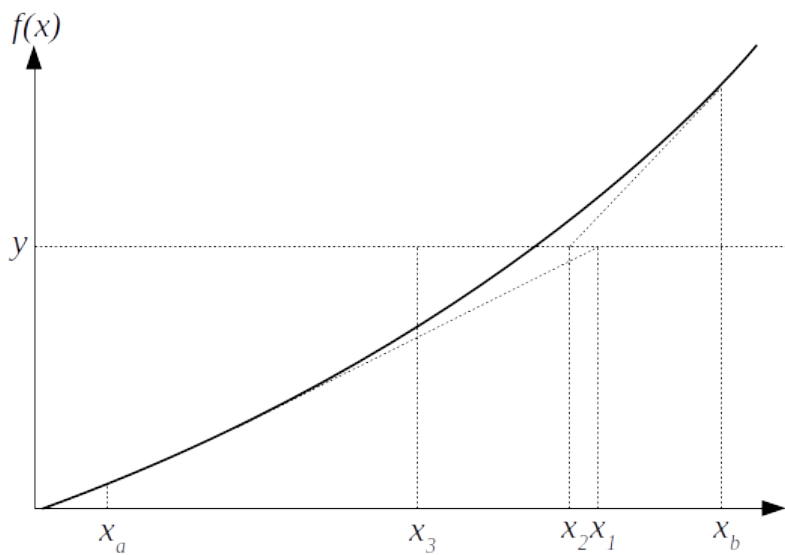


Figure 8.4: A depiction of the bisection iteration algorithm on a hypothetical function.

In standard candidate selection, the median (middle) of the three guesses is selected. If the median lies outside of x_a and x_b , then the next guess reverts to the bisection. In this way, if the two guesses at the edge agree, the more conservative of the two (the one closer to the center of the domain) will be selected. If they disagree or if they both diverge from the domain, the algorithm reverts to bisection.

The standard candidate selection process gives excellent performance virtually everywhere in multi-phase property domains except very near the critical point. Testing revealed that standard candidate selection is vulnerable to slow convergence near the critical point due to the extreme numerical stiffness there. It should be emphasized that the algorithm still converges; just very slowly.

The problem is similar to one that happens in the secant iteration algorithm. If a solution lies in a numerically stiff regime like in Figure 8.3, the boundary at the stiff edge will produce a guess very close to itself while the other guess will produce a guess that diverges outside the boundary. Median candidate selection will always favor the candidate at the stiff edge even though it only barely moves from the one prior.

8.4.4 Paranoid candidate selection

So-called “paranoid” candidate selection utterly distrusts the Newton method if either of the guesses lie outside of the domain. The idea is that if the domain is so nonlinear that either of the guesses leaves the domain, the algorithm should revert to bisection until it looks like the function is locally linear.

In practice, paranoid hybrid iteration is slower than standard hybrid iteration. However, in testing, it has never failed to converge on any of the multi-phase models. Under the most difficult conditions, it might take ten or more iterations to converge. In better conditions, it typically converges in five or so iterations.

Algorithm 3 HYBRID1: Hybrid bisection and Newton iteration

```
procedure _HYBRID1( $f, y, x_a, x_b, \epsilon_x, \epsilon_y$ )  
     $y_a \leftarrow f(x_a)$  ▷ Evaluate  $f$  at the initial bounds  
     $y'_a \leftarrow f'(x_a)$   
     $y_b \leftarrow f(x_b)$   
     $y'_b \leftarrow f'(x_b)$   
    if  $y_a > y_b$  then  
         $x_a, y_a, y'_a \leftrightarrow x_b, y_b, y'_b$  ▷ Swap the bounds  
    end if  
    if NOT  $y_a < y < y_b$  then  
        return “ERROR: The solution is not bracketed”  
    end if  
     $x_1 \leftarrow x_a + (y - y_a)/y'_a$  ▷ The first candidates  
     $x_2 \leftarrow x_b + (y - y_b)/y'_b$   
     $x_3 \leftarrow (x_a + x_b)/2$   
    while  $|x_b - x_a| > \epsilon_x$  do ▷ Halt if the interval is small  
         $x_c \leftarrow \text{median}(x_1, x_2, x_3)$  ▷ Select the median  
        if  $x_c \leq \min(x_a, x_b)$  OR  $x_c \geq \max(x_a, x_b)$  then ▷ If out of  
            bounds...  
                 $x_c \leftarrow x_3$  ▷ ...use bisection instead  
        end if  
         $y_c \leftarrow f(x_c)$  ▷ Evaluate  $f$  at the new guess  
         $y'_c \leftarrow f'(x_c)$   
        if  $|y_c - y| < \epsilon_y$  then ▷ Test for Newton convergence  
            return  $x_c$   
        end if  
        if  $y_c < y$  then ▷ If this is a lower bound  
             $x_a \leftarrow x_c$  ▷ Replace the lower bound  
             $y_a \leftarrow y_c$   
             $y'_a \leftarrow y'_c$   
             $x_1 \leftarrow x_a + (y - y_a)/y'_a$  ▷ A new lower candidate  
            ...
```

Algorithm 4 HYBRID1: Hybrid bisection continued

...
else ▷ If this is an upper bound
 $x_b \leftarrow x_c$ ▷ Replace the upper bound
 $y_b \leftarrow y_c$
 $y'_b \leftarrow y'_c$
 $x_2 \leftarrow x_b + (y - y_b)/y'_b$ ▷ A new upper candidate
end if
 $x_3 = (x_a + x_b)/2$ ▷ A new bisection candidate
end while
return x_3 ▷ Convergence by bisection
end procedure

Algorithm 5 HYBRID1: Paranoid hybrid iteration

```
procedure _HYBRID1( $f, y, x_a, x_b, \epsilon_x, \epsilon_y$ )  
     $y_a \leftarrow f(x_a)$  ▷ Evaluate  $f$  at the initial bounds  
     $y'_a \leftarrow f'(x_a)$   
     $y_b \leftarrow f(x_b)$   
     $y'_b \leftarrow f'(x_b)$   
    if  $y_a > y_b$  then  
         $x_a, y_a, y'_a \leftrightarrow x_b, y_b, y'_b$  ▷ Swap the bounds  
    end if  
    if NOT  $y_a < y < y_b$  then  
        return "ERROR: The solution is not bracketed"  
    end if  
     $x_1 \leftarrow x_a + (y - y_a)/y'_a$  ▷ The first candidates  
     $x_2 \leftarrow x_b + (y - y_b)/y'_b$   
     $x_3 \leftarrow (x_a + x_b)/2$   
    while  $|x_b - x_a| > \epsilon_x$  do ▷ Halt if the interval is small  
        ▷ If either guess is out of bounds...  
        if  $x_1 \notin (x_a, x_b)$  OR  $x_2 \notin (x_a, x_b)$  then  
             $x_c \leftarrow x_3$  ▷ ...use bisection instead  
        else  
             $x_c \leftarrow \text{median}(x_1, x_2, x_3)$  ▷ Select the median  
        end if  
         $y_c \leftarrow f(x_c)$  ▷ Evaluate  $f$  at the new guess  
         $y'_c \leftarrow f'(x_c)$   
        if  $|y_c - y| < \epsilon_y$  then ▷ Test for Newton convergence  
            return  $x_c$   
        end if  
        if  $y_c < y$  then ▷ If this is a lower bound  
             $x_a \leftarrow x_c$  ▷ Replace the lower bound  
             $y_a \leftarrow y_c$   
             $y'_a \leftarrow y'_c$   
             $x_1 \leftarrow x_a + (y - y_a)/y'_a$  ▷ A new lower candidate  
            ...
```

Algorithm 6 HYBRID1: Paranoid hybrid iteration continued

```

    ...
    else
         $x_b \leftarrow x_c$ 
         $y_b \leftarrow y_c$ 
         $y'_b \leftarrow y'_c$ 
         $x_2 \leftarrow x_b + (y - y_b)/y'_b$ 
    end if
     $x_3 = (x_a + x_b)/2$ 
end while
return  $x_3$ 
end procedure

```

▷ If this is an upper bound
 ▷ Replace the upper bound
 ▷ A new upper candidate
 ▷ A new bisection candidate
 ▷ Convergence by bisection

Bibliography

- [1] M. W. Chase, *NIST-JANAF thermochemical tables fourth edition; part I, Al-Co*. Gaithersburg, Maryland 20899-0001: American Chemical Society, American Institute of Physics, and National Institute of Standards and Technology, 1998.
- [2] “Inchi trust home page.” InChI Trust, <https://www.inchi-trust.org/>. accessed: Apr, 2022.
- [3] “Cas home page.” Chemical Abstracts Service, American Chemical Society, <https://www.cas.org>. accessed: Apr, 2022.
- [4] L. V. Judson, “Weights and measures standards of the united states, a brief history,” Tech. Rep. SP447, National Bureau of Standards, Washington DC, 20402, Mar 1976.
- [5] M. B. D. L. Grye, M. W. Foerster, M. R. Siegel, M. V. V. Lang, M. W. Marek, and M. L. B. D. Zagon, “Resolution2: Declaration on the unit of mass and on the definition of weight; conventional value of g_n ,” in *Proceedings of the 3rd CGPM*, (Paris), 1901.
- [6] M. DeBroglie, R. Vieweg, G. Zickner, U. Stille, E. Padelt, and E. Blechschmidt, “Resolution4: Definition of the standard atmosphere,” in *Proceedings of the 10th CGPM*, (Paris), 1954.
- [7] B. Inglis, J. Ulrich, and M. Milton, *The International System of Units*. Creative Commons: BIPM, 9th ed., 2019.
- [8] A. Thompson and B. N. Taylor, “Guide for the use of the international system of units,” Tech. Rep. SP811, NIST, Gaithersburg, MD 20899, Mar 2008.

- [9] A. V. Astin, H. A. Karo, and F. H. Mueller, “Refinement of values for the yard and the pound,” in *Federal Register*, (Washington DC), National Bureau of Standards, Jul 1959.
- [10] K. Butcher, L. Crown, and E. J. Gentry, “The international system of units (si) - conversion factors for general use,” Tech. Rep. SP1038, Weights and Measures Division, NIST, May 2006.
- [11] J. R. Rumble, *CRC Handbook of Chemistry and Physics*. Boca Raton, FL: Taylor and Francis, 98th ed., 2018.
- [12] National Institute for Standards and Technology, <https://webbook.nist.gov/>. accessed: Jun, 2021.
- [13] B. J. McBride, S. Gordon, and M. A. Reno, “Coefficients for calculating thermodynamic and transport properties of individual species,” Tech. Rep. 4513, NASA, 1993.
- [14] R. B. Stewart, R. T. Jacobsen, and W. Wagner, “Thermodynamic properties of oxygen from the triple point to 300 k with pressures to 80 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 20, no. 5, p. 917, 1991.
- [15] U. Setzmann and W. Wagner, “A new equation of state and tables of thermodynamic properties for methane converging the range from the melting line to 625 k at pressures up to 1000 k,” *Journal of Physical and Chemical Reference Data*, vol. 20, p. 1061, 1991.
- [16] R. Span and W. Wagner, “Equations of state for technical applications. i. simultaneously optimized functional forms for nonpolar and polar fluids,” *International Journal of Thermophysics*, vol. 24, no. 1, pp. 1–39, 2003.
- [17] R. Span and W. Wagner, “Equations of state for technical applications. ii. results for nonpolar fluids,” *International Journal of Thermophysics*, vol. 24, no. 1, pp. 41–109, 2003.
- [18] R. Span and W. Wagner, “Equations of state for technical applications. iii. results for polar fluids,” *International Journal of Thermophysics*, vol. 24, no. 1, pp. 111–162, 2003.

- [19] REFPROP, Software, <https://www.nist.gov/srd/refprop>.
- [20] R. Span and W. Wagner, “A new equation of state for carbon dioxide covering the fluid region from the triple-point temperature to 1100 k at pressures up to 800 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 25, p. 1509, 1994.
- [21] T. Petrova, “Revised release on the iapws formulation 1995 for the thermodynamic properties of ordinary water substance for general and scientific use,” tech. rep., 2014.
- [22] C. Tegeler, R. Span, and W. Wagner, “A new equation of state for argon covering the fluid region for temperatures from the melting line to 700 k at pressures up to 1000 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 28, p. 779, 1999.
- [23] I. A. Richardson, J. W. Leachman, and E. W. Lemmon, “Fundamental equation of state for deuterium,” *Journal of Physical and Chemical Reference Data*, vol. 43, p. 013103, 2014.
- [24] J. W. Leachman, R. T. Jacobsen, S. G. Penoncello, and E. W. Lemmon, “Fundamental equations of state for parahydrogen, normal hydrogen, and orthohydrogen,” *Journal of Physical and Chemical Reference Data*, vol. 38, p. 721, 2009.
- [25] R. Span, E. W. Lemmon, R. T. Jacobsen, W. Wagner, and A. Yokozeki, “A reference equation of state for the thermodynamic properties of nitrogen for temperatures from 63.151 to 1000 k and pressures to 2200 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 29, no. 6, pp. 1361–1433, 2000.
- [26] C.-C. Piao and M. Noguchi, “An international standard equation of state for thermodynamic properties of hfc-125 (pentafluoroethane),” *Journal of Physical and Chemical Reference Data*, vol. 27, p. 775, 1998.
- [27] R. Tilner-Roth and H. D. Baehr, “An international standard formulation for the thermodynamic properties of 1,1,1,2-tetrafluoroethane (hfc-134a) for temperatures from 170 k to 455 k

- and pressures up to 70 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 23, p. 657, 1994.
- [28] E. W. Lemmon and R. T. Jacobsen, “An international standard formulation for the thermodynamic properties of 1,1,1-trifluoroethane (hfc-143a) for temperatures from 161 to 450 k and pressures to 50 mpa,” *Journal of Physical and Chemical Reference Data*, vol. 29, no. 4, pp. 521–552, 2000.
- [29] R. Akasaka, Y. Zhou, and E. W. Lemmon, “A fundamental equation of state for 1,1,1,3,3-pentafluoropropane (r-245fa),” *Journal of Physical and Chemical Reference Data*, vol. 44, p. 013104, 2015.
- [30] M. Richter, M. O. McLinden, and E. W. Lemmon, “Thermodynamic properties of 2,3,3,3-tetrafluoroprop-1-ene (r1234yf): vapor pressure and p-rho-t measurements and an equation of state,” *Journal of Chemical and Engineering Data*, vol. 56, pp. 3254–3264, 2011.
- [31] M. Thol and E. W. Lemmon, “Thermodynamic equation of state for the thermodynamic properties of trans-13,3,3-tetrafluoropropene [r-1234ze(e)],” *International Journal of Thermophysics*, vol. 37, no. 28, pp. 1–16, 2016.